

Pisa-Pontedera, March-May, 2026

UNIPi-SSSA-IMT MSc in Bionics Engineering



Scuola Superiore
Sant'Anna

Neuromorphic Engineering

Calogero M. Oddo (calogero.oddo@santannapisa.it)

Associate Professor of Bioengineering

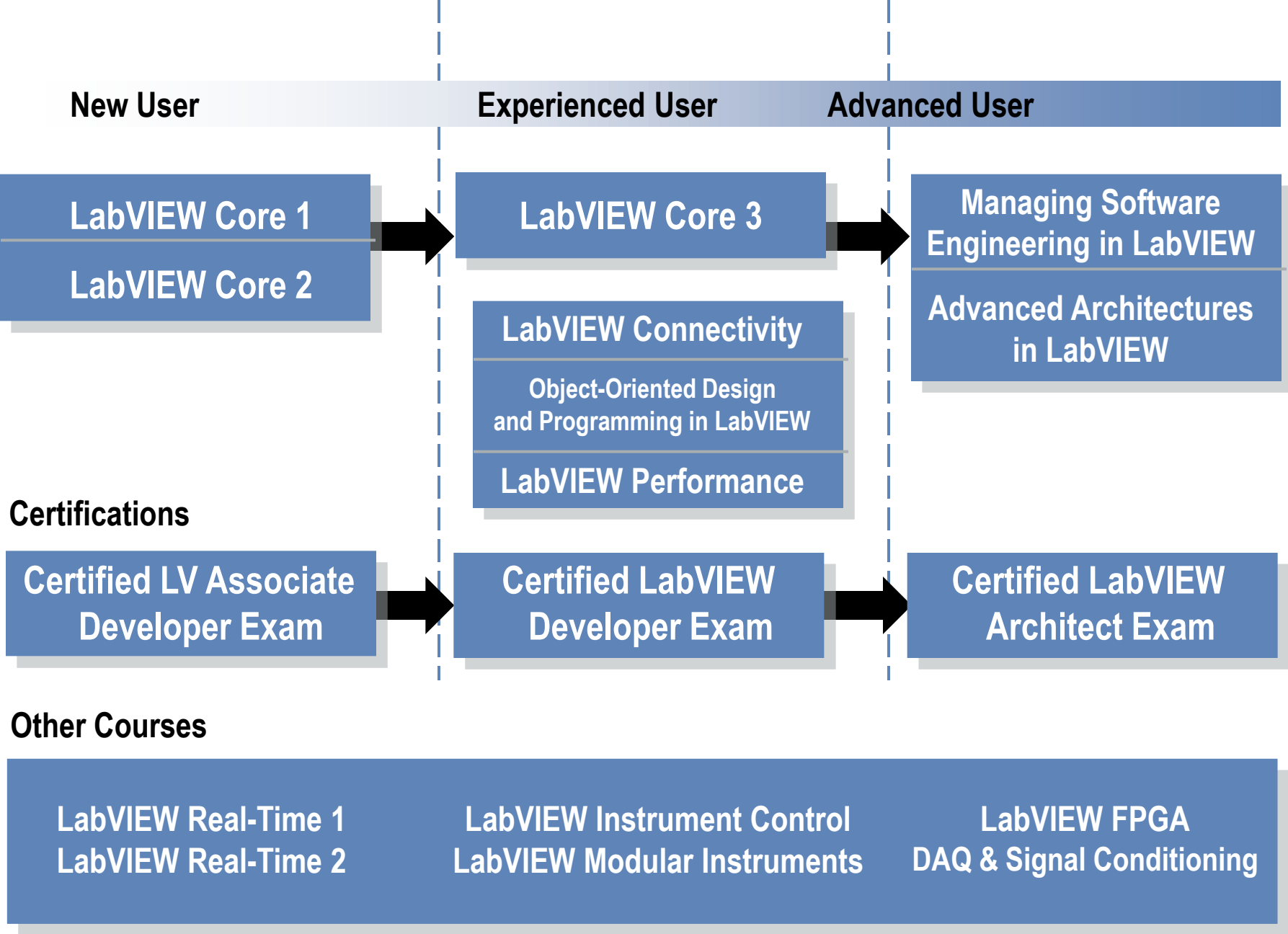
Head of the Neuro-Robotic Touch Laboratory



The BioRobotics Institute
Scuola Superiore Sant'Anna, Pisa, Italy

Labview installation

- v2025 Q3
- Install 32-bit version, not the 64-bit one!
- Code: D11L1101332
- Only for academic use



Navigating LabVIEW

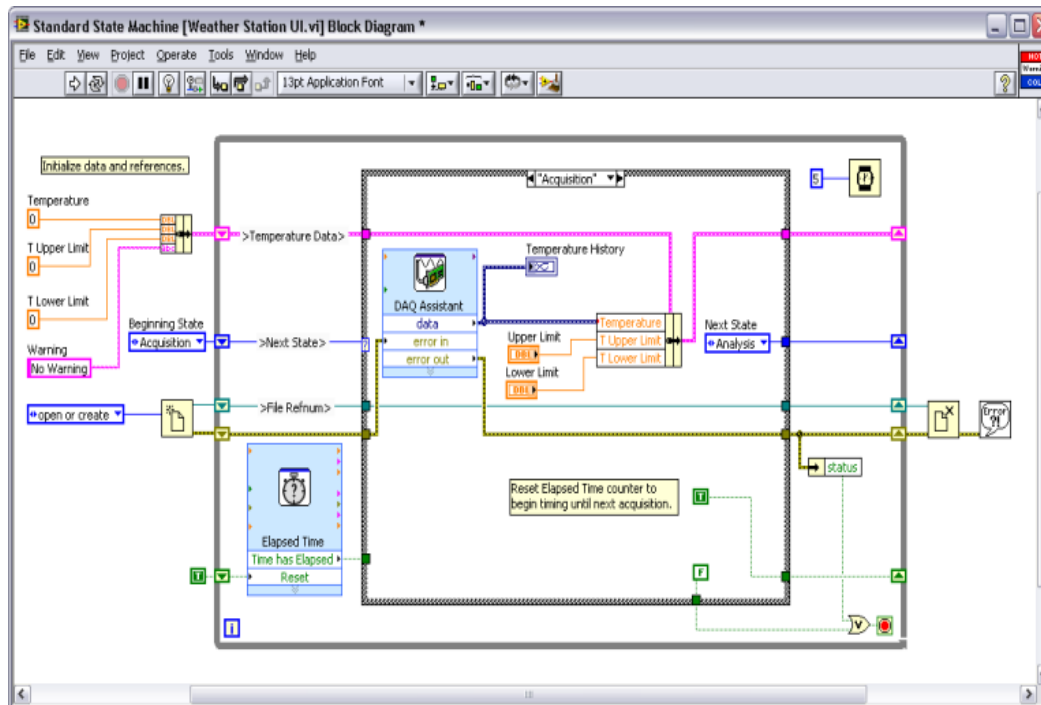
TOPICS

- A. What Is LabVIEW?
- B. Project Explorer
- C. Parts of a VI
- D. Front Panel
- E. Block Diagram
- F. Searching for Controls, VIs and Functions
- G. Selecting a Tool
- H. Dataflow
- I. Building a Simple VI

A. What Is LabVIEW?

What Is LabVIEW?

— A graphical programming environment used to develop sophisticated measurement, test, and control systems.



LabVIEW:

- Interfaces with wide variety of hardware
- Scales across different targets and OSs
- Provides built-in analysis libraries

LabVIEW Language Characteristics

LabVIEW Core 1 Topics

- Graphical
- Dataflow-oriented
- Compiled
- Multi-platform
- Synchronous

LabVIEW Core 2 and Later Topics

- Event-driven
- Multi-threaded
- Object-oriented
- Multi-target
- Memory-Managed

B. Project Explorer

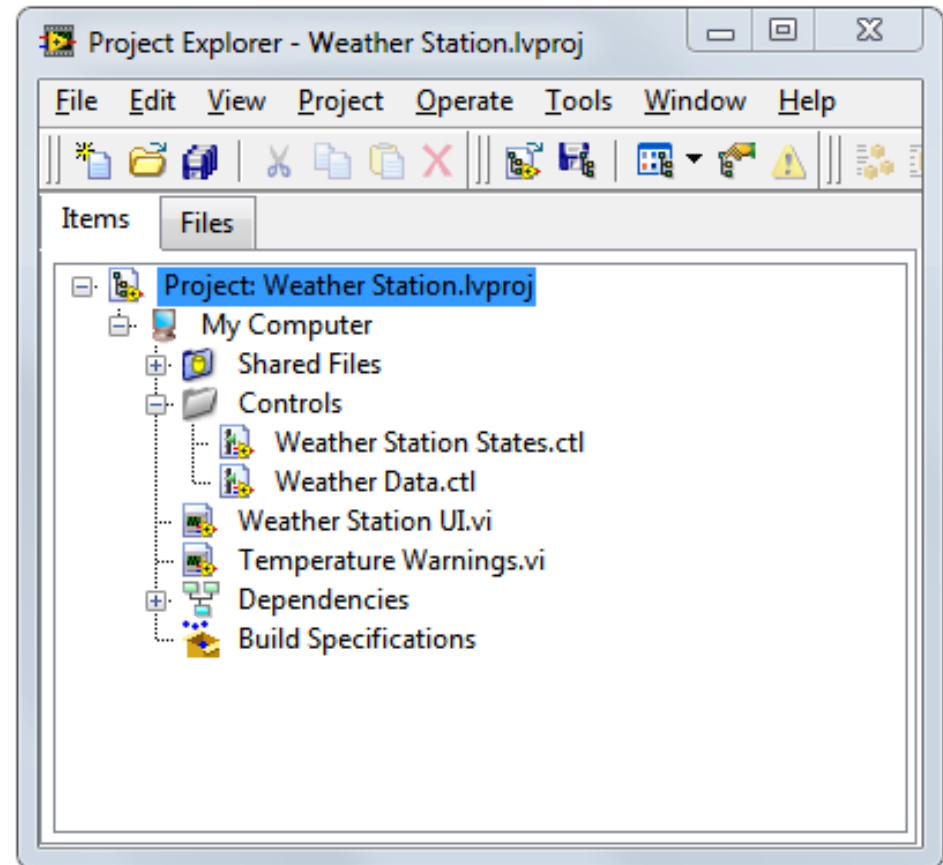
Project Explorer Window

Files Types

Project Folders

Project Explorer

- Find, access, and organize project files
- Prevent, detect, and resolve incorrect links
- Deploy or download files to targets
- Manage code for build options
 - Executables, installers, and zip files
- Integrate with source code control providers



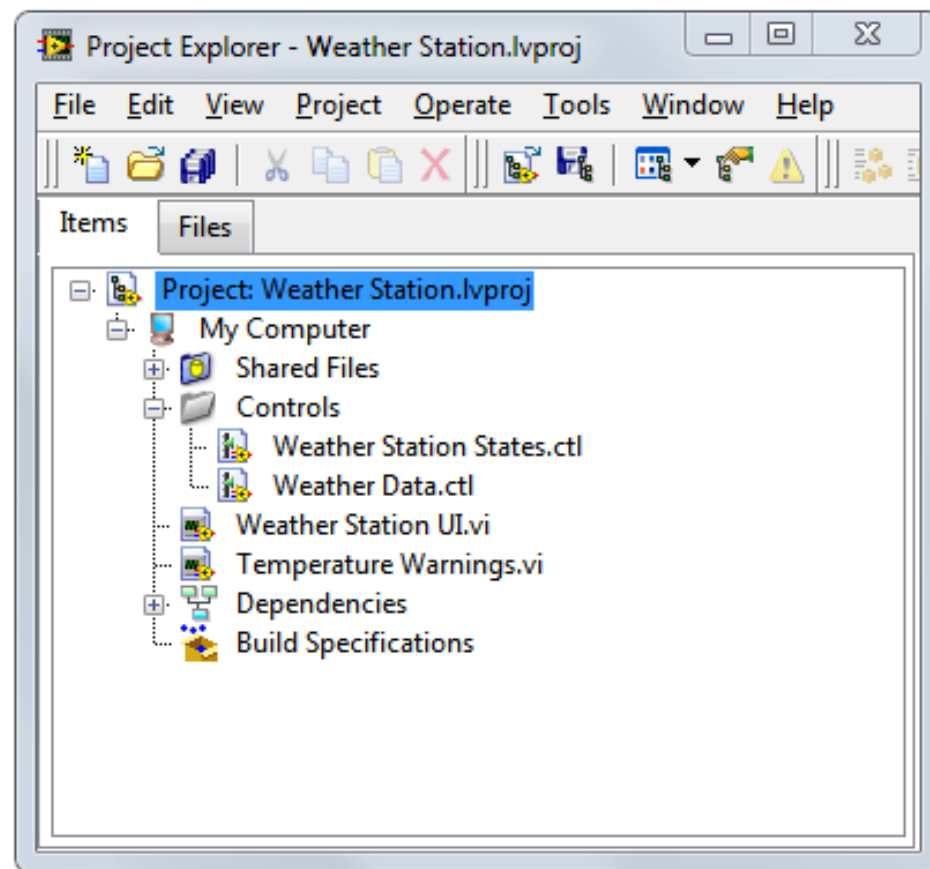
LabVIEW Files

Common LabVIEW file extensions:

LabVIEW project — .lvproj

Virtual instrument (VI) — .vi

Custom control — .ctl



Adding Folders to a Project



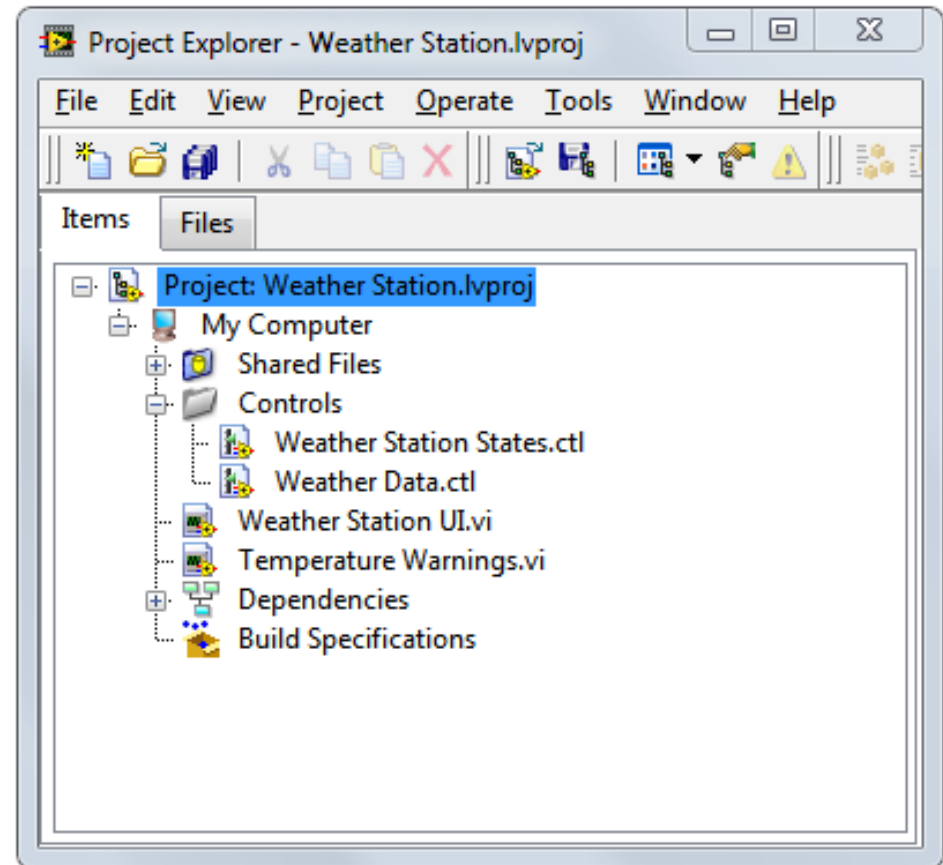
Virtual folder

- Organizes project items and does not represent files on disk



Auto-populating folder

- Adds a directory on disk to the project
- LabVIEW continuously monitors and updates the folder according to changes made in the project and on disk



C. Parts of a VI

Front Panel

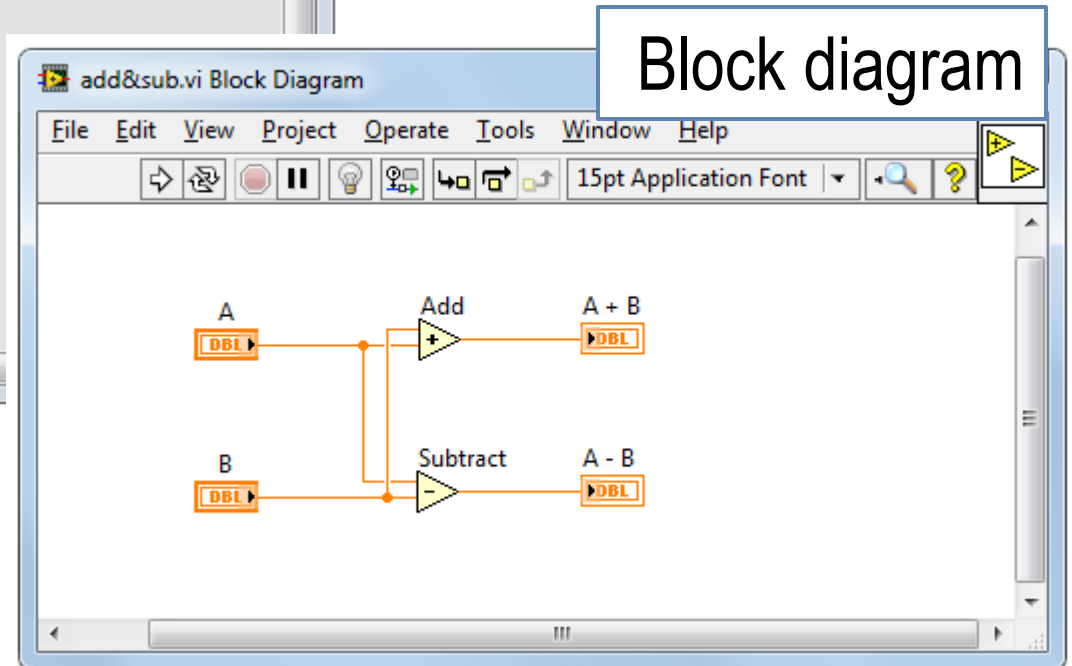
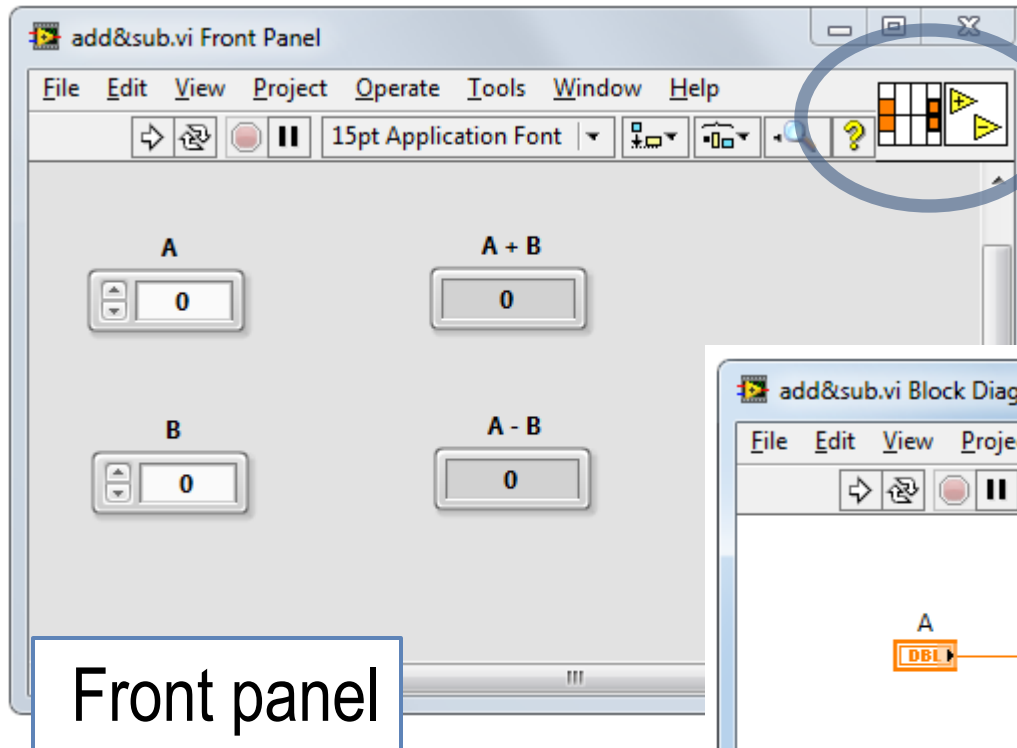
Block Diagram

Icon

Connector Pane

Parts of a VI

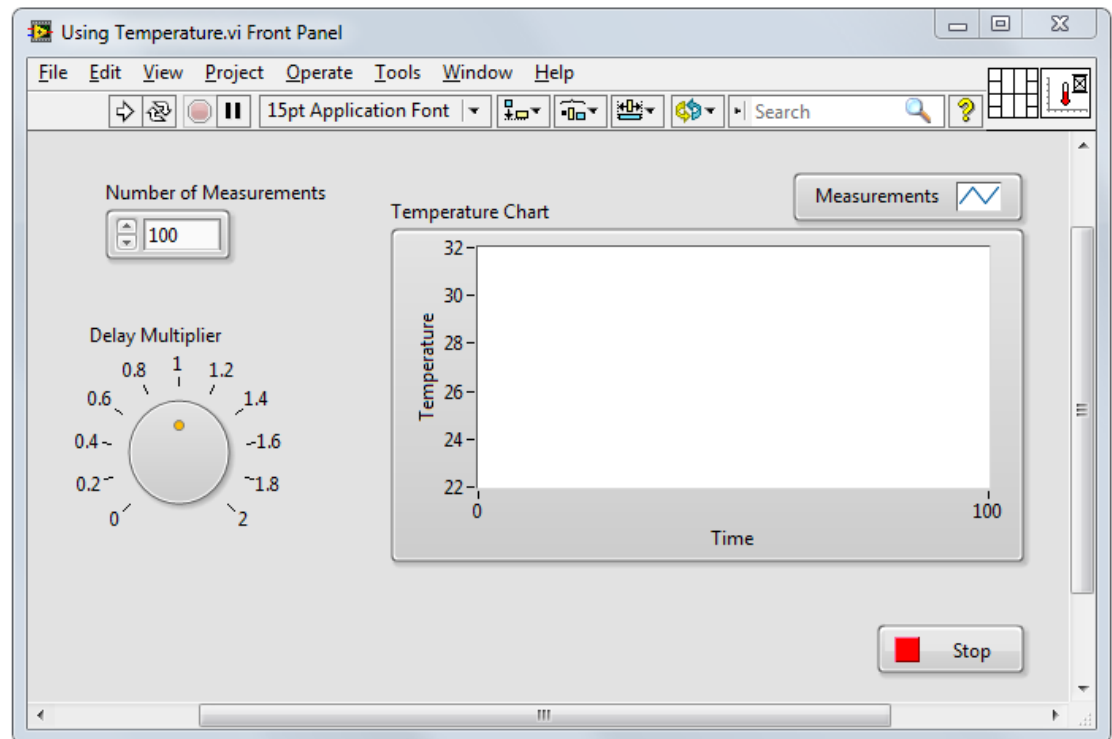
VIs have 3 main components:



Parts of a VI – Front Panel

Front Panel – User interface for the VI

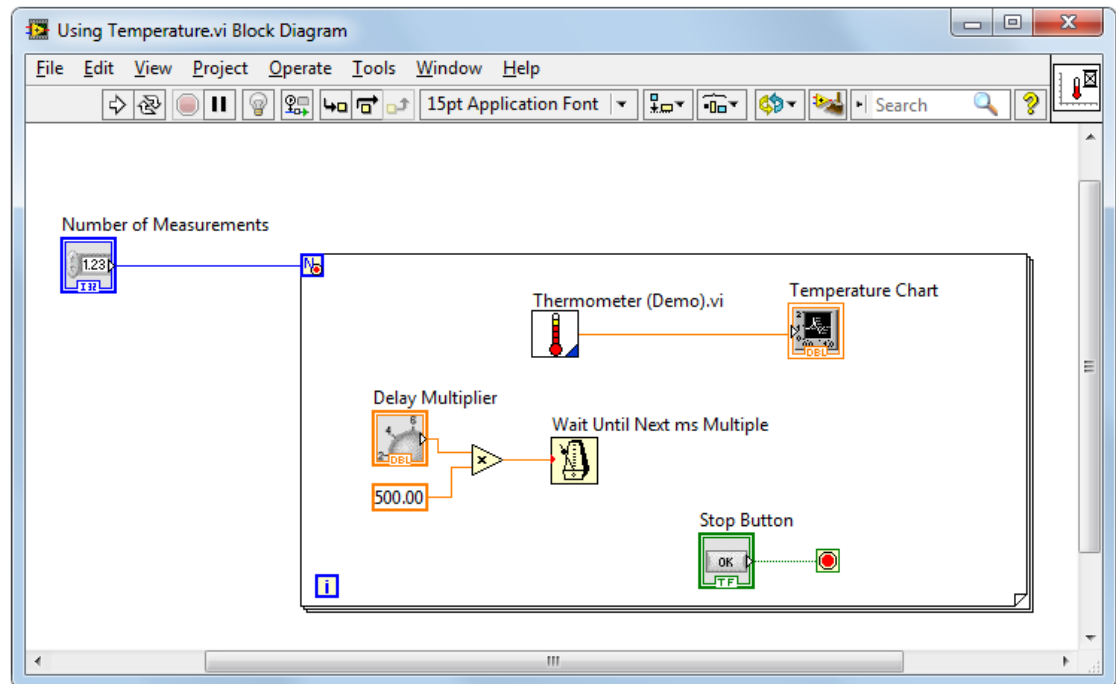
You build the front panel with controls (inputs) and indicators (outputs).



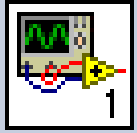
Parts of a VI – Block Diagram

Block Diagram – Contains the graphical source code

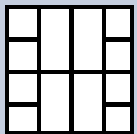
Front panel objects appear as terminals on the block diagram.



Parts of a VI – Icon/Connector Pane



Icon – Graphical representation of a VI



Connector Pane – Map of the inputs and outputs of a VI

Icons and connector panes are necessary to use a VI as a subVI.

- A subVI is a VI that appears on the block diagram of another VI.
- A subVI is similar to a subroutine or function in a text-based programming language.

Using the Project Explorer and Starting a VI

- Open an existing project.
- Identify the parts of a VI.
- Create a new project.
- Add and remove files and folders.
- Create a new VI.

DEMONSTRATION

D. Front Panel

Controls and Indicators

Object Styles

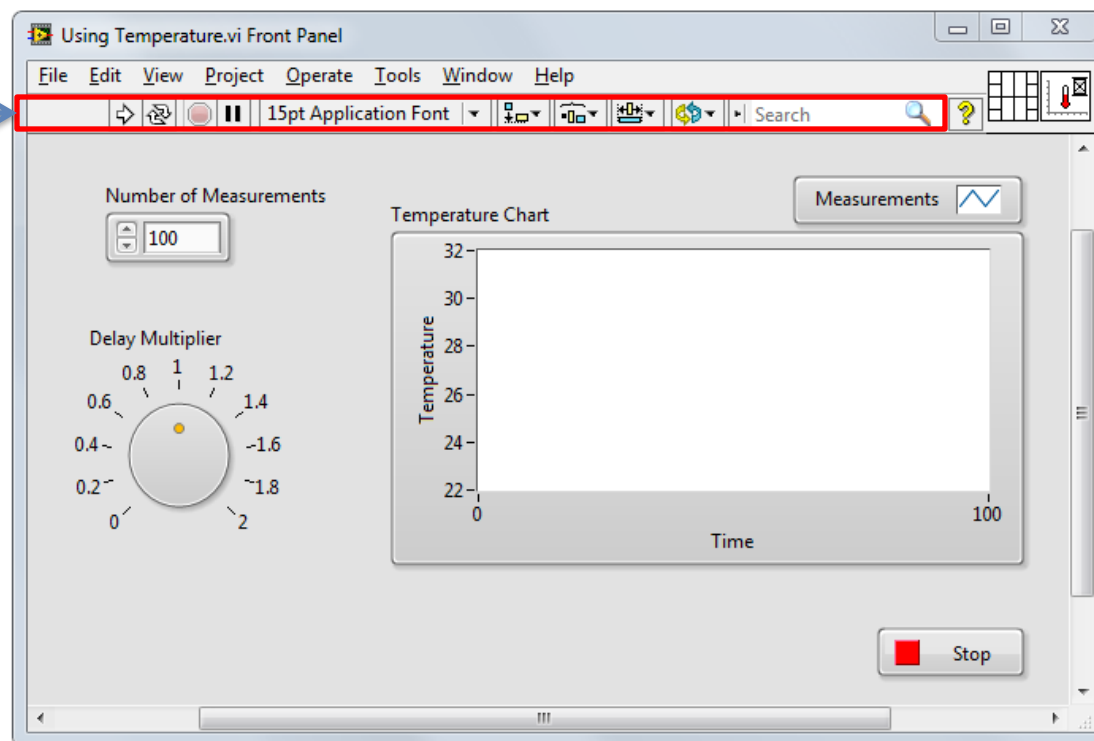
Object Types

- Boolean

- Numeric

- String

Front Panel



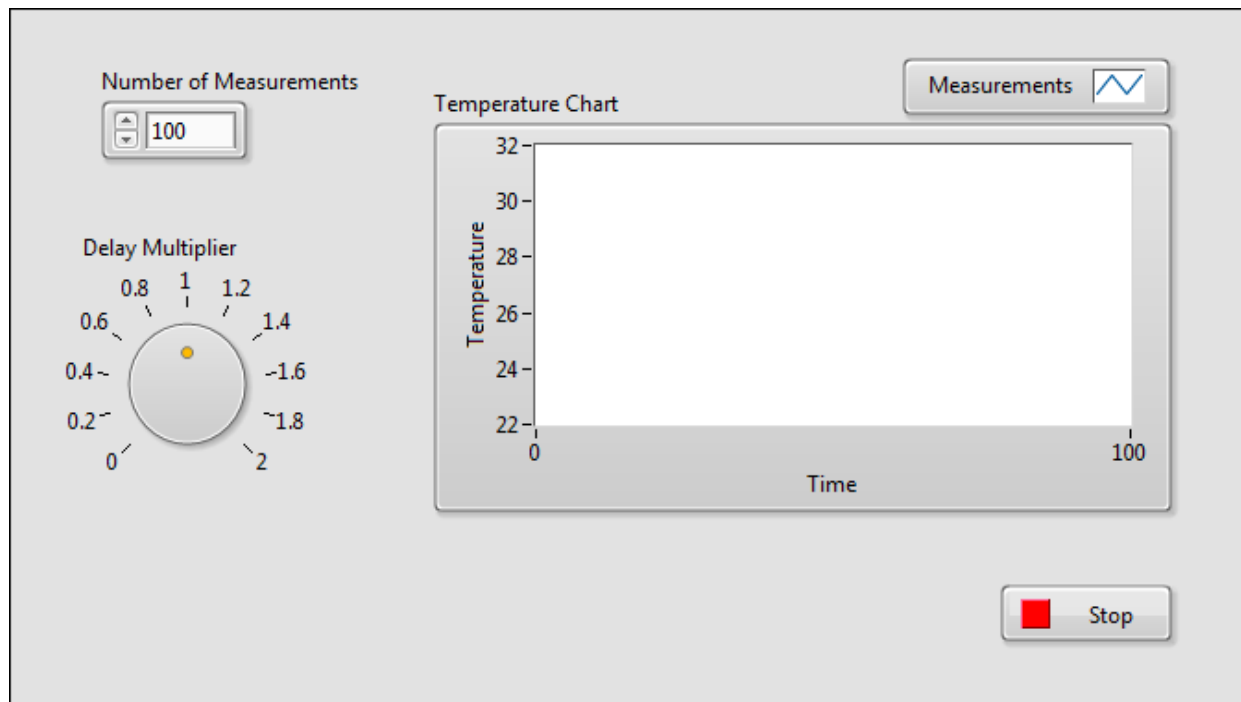
Controls and Indicators

Controls

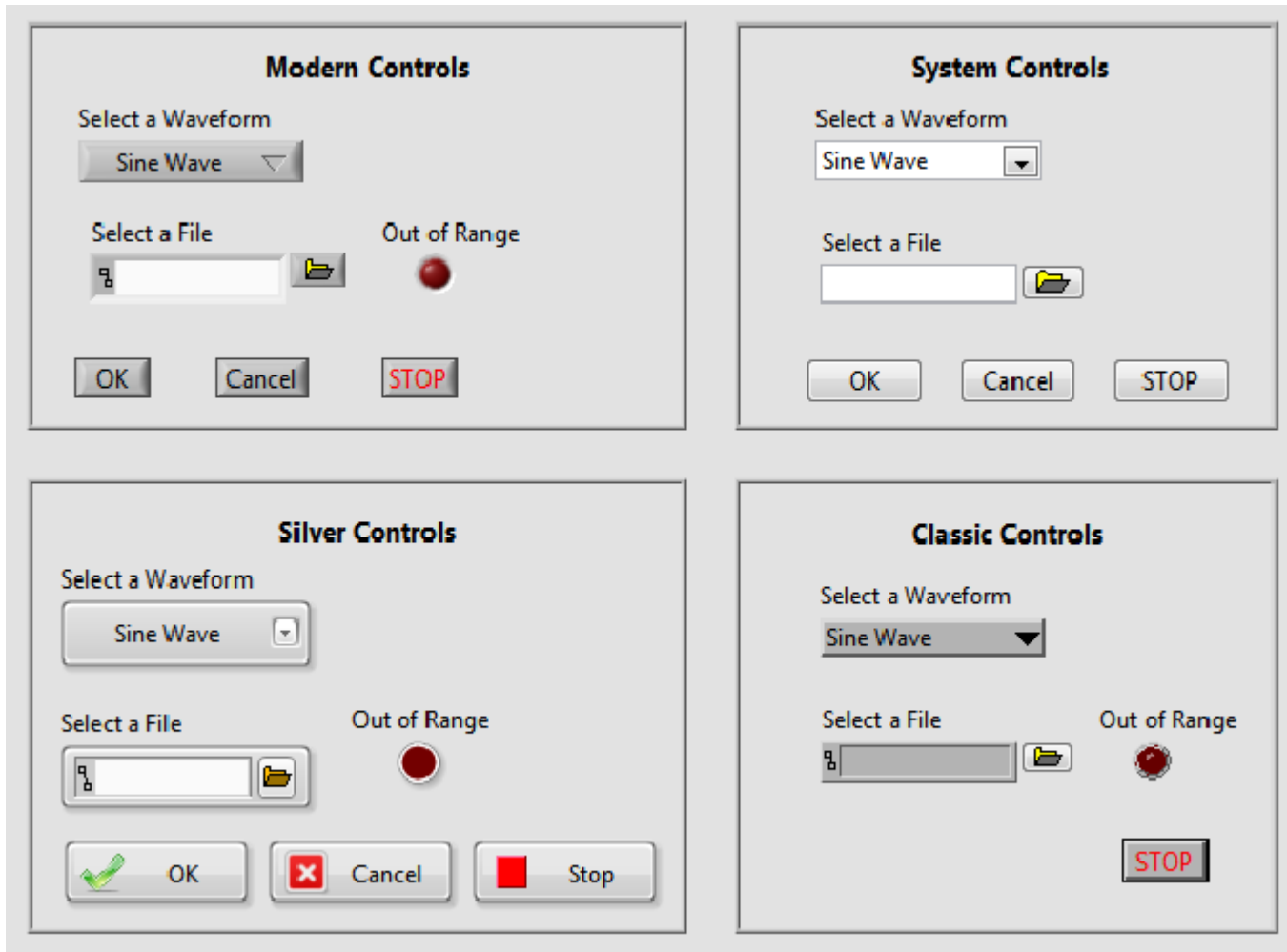
- Input devices
- Knobs, buttons, slides
- Supply data to the block diagram

Indicators

- Output devices
- Graphs, LEDs
- Display data the block diagram acquires or generates

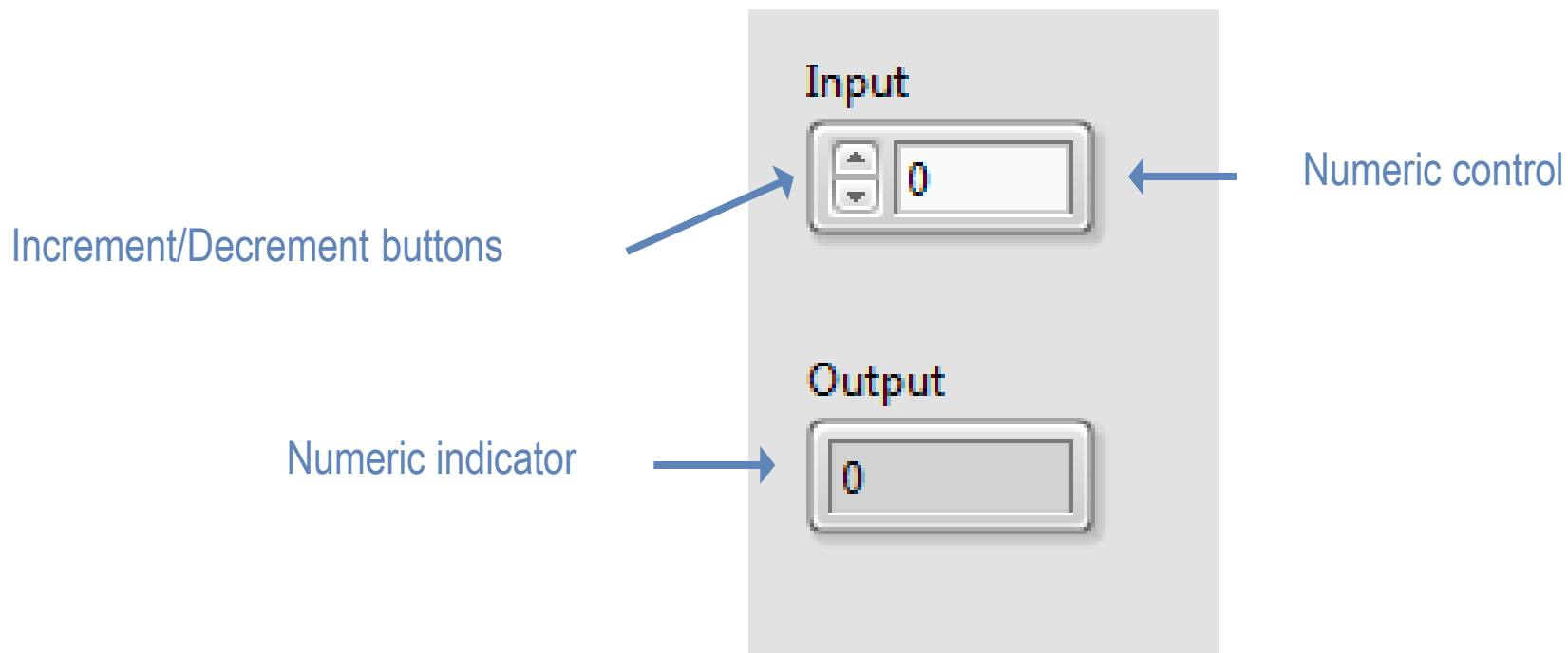


Front Panel Object Styles



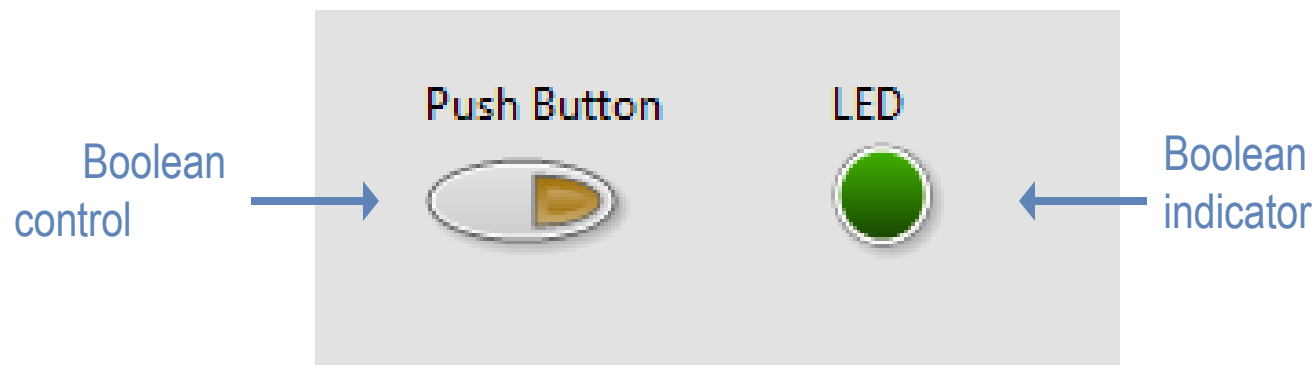
Numeric Controls and Indicators

The numeric data in a control or indicator can represent numbers of various types, such as integer or floating-point.



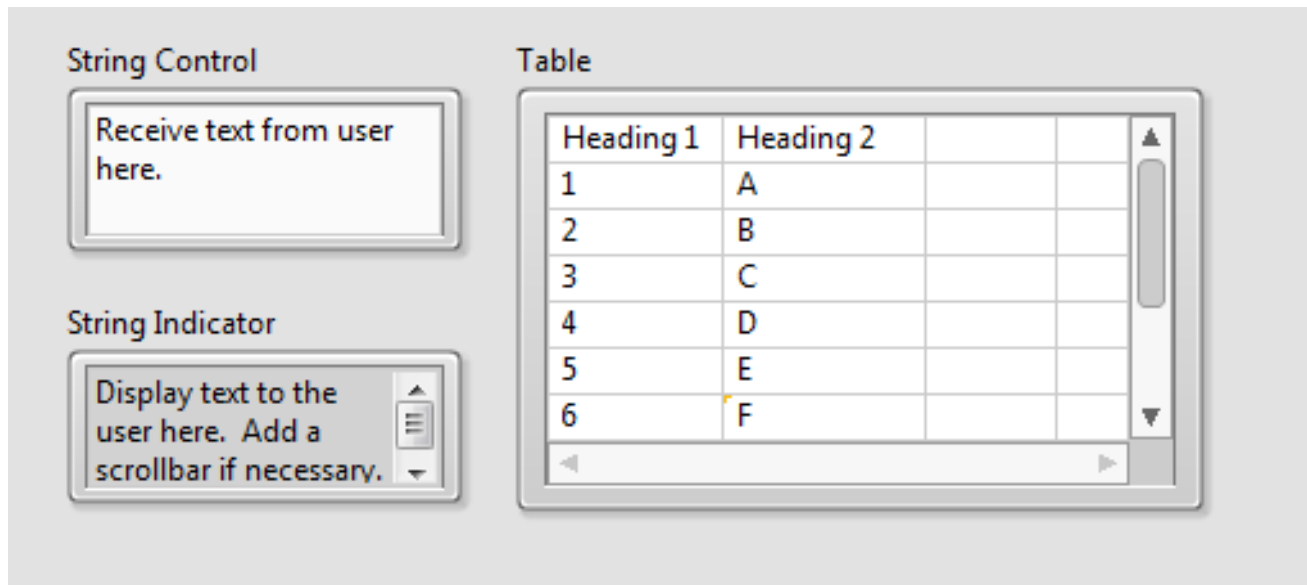
Boolean Controls and Indicators

- The Boolean data type represents data that has only two options, such as True/False or On/Off.
- Use Boolean controls and indicators to enter and display Boolean (TRUE/FALSE) values.
- Boolean objects simulate switches, push buttons, and LEDs.



Strings

- The string data type is a sequence of ASCII characters .
- Use string controls to receive text from the user, such as a password or user name.
- Use string indicators to display text to the user.



E. Block Diagram

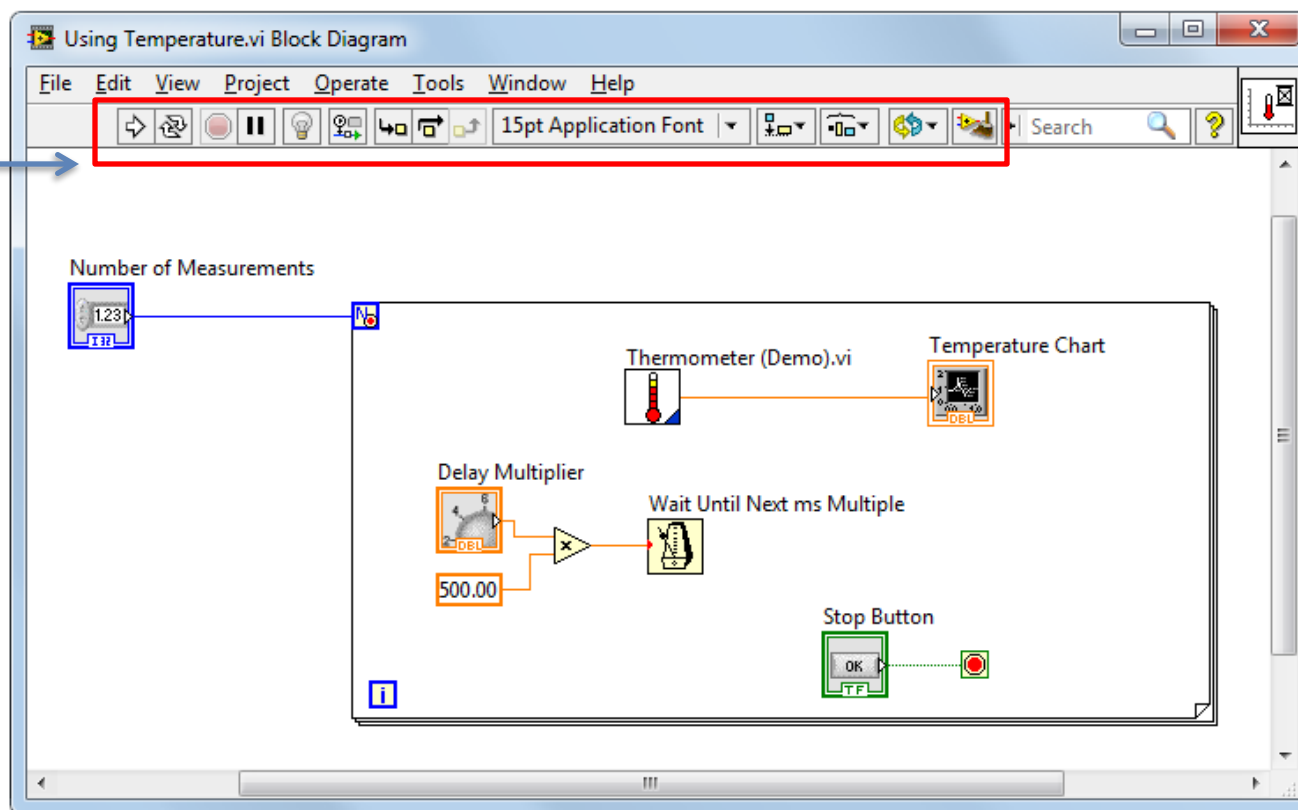
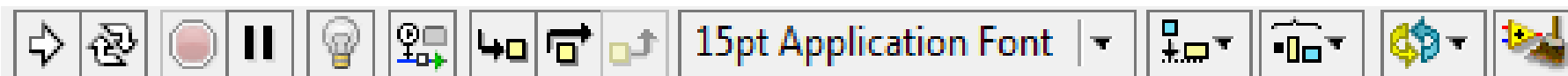
Terminals

Nodes

Wires

Context Help

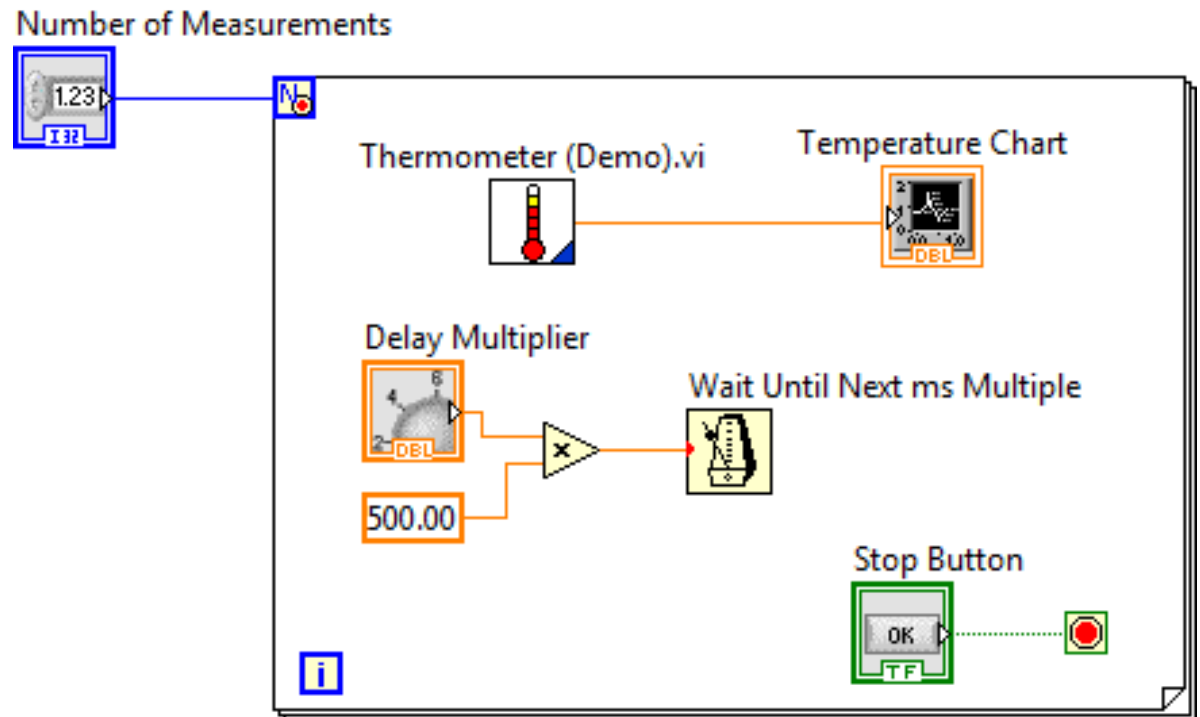
Block Diagram



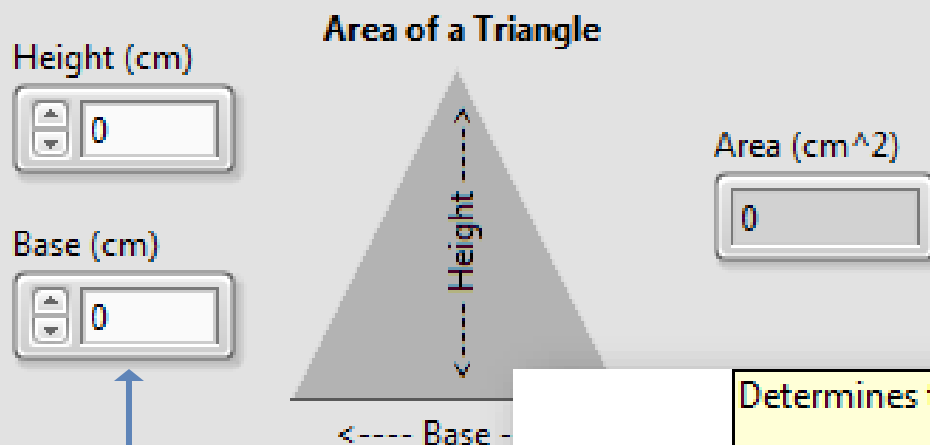
Block Diagram

Block diagram items:

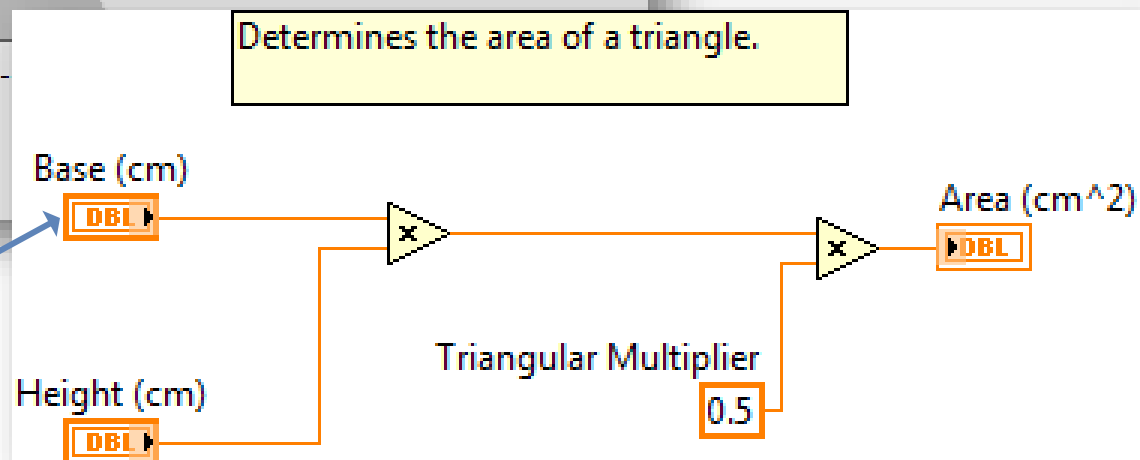
- Terminals
- Constants
- Nodes
 - Functions
 - SubVIs
 - Structures
- Wires
- Free labels



Terminals



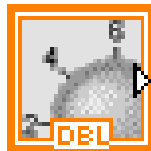
Same label name



Terminals for Front Panel Objects

- Terminals are:
 - Entry and exit ports that exchange information between the front panel and block diagram.
 - Analogous to parameters in text-based programming languages.
- Double-click a terminal to locate the corresponding front panel object.

Input



Input



View Terminals as Icons

- By default, View as Icon option is enabled.
- Deselect View as Icon for a more compact view.

Numeric



Numeric



Boolean



String



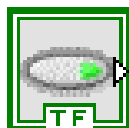
Numeric



Numeric



Boolean



String



String



Visible Items ▶

Find Control

Make Type Def.

Hide Control

Change to Indicator

Change to Constant

Description and Tip...

String Palette ▶

Create ▶

Data Operations ▶

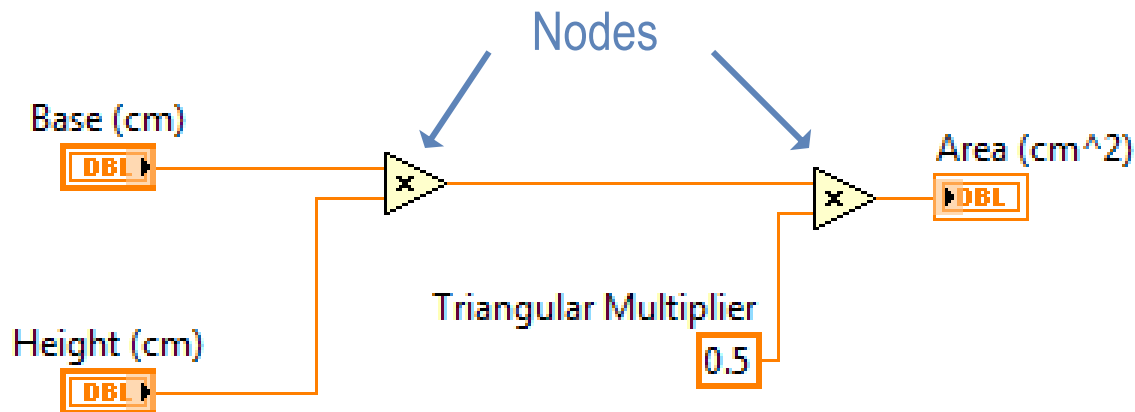
Advanced ▶

✓ View As Icon

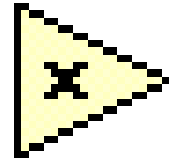
Properties

Nodes

Nodes are objects on the block diagram that have inputs and/or outputs and perform operations when a VI runs.



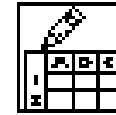
Function Nodes



- Functions are:
 - Fundamental operating elements of LabVIEW.
 - Do not have front panels or block diagrams, but do have connector panes.
 - Has a pale yellow background on its icon.
- Double-clicking a function only selects the function.
- Functions do not open like VIs and subVIs.

SubVI Nodes

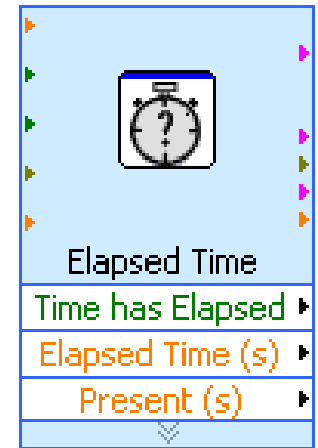
Write To Spreadsheet File.vi



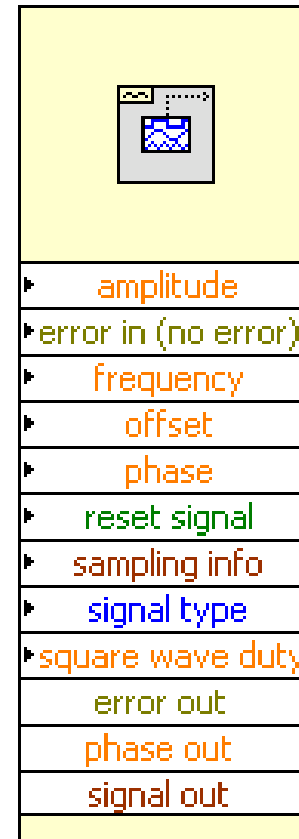
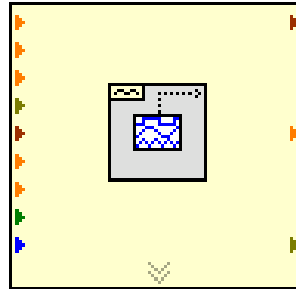
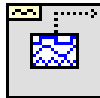
- SubVIs :
 - Are VIs that you use on the block diagram of another VI.
 - Have front panels and block diagrams.
 - Use the icon from the upper-right corner of the front panel as the icon that appears when you place the subVI on a block diagram.
- When you double-click a subVI, the front panel and block diagram open.
- Any VI has the potential to be used as a subVI.

Express VIs

- Express VIs:
 - Are a special type of subVI.
 - Require minimal wiring because you configure them with dialog boxes.
 - Save each configuration as a subVI.
- Icons for Express VIs appear on the block diagram as icons surrounded by a blue field.



Node View Options



Wires

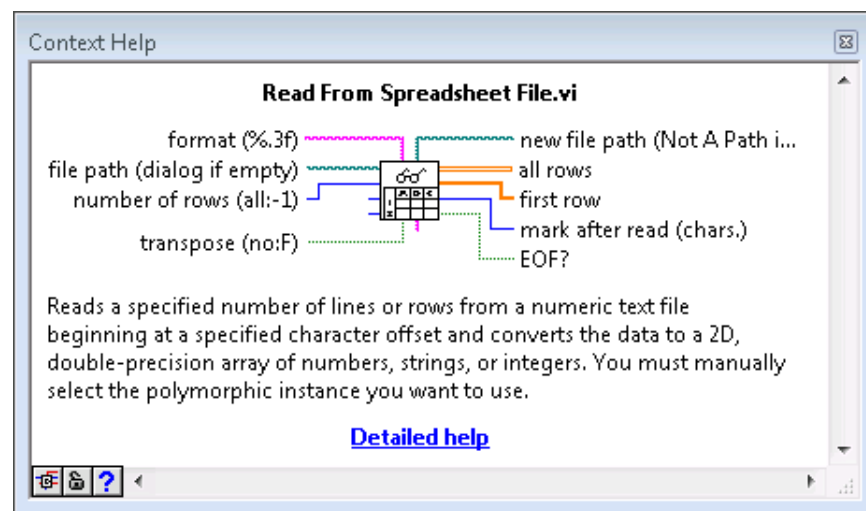
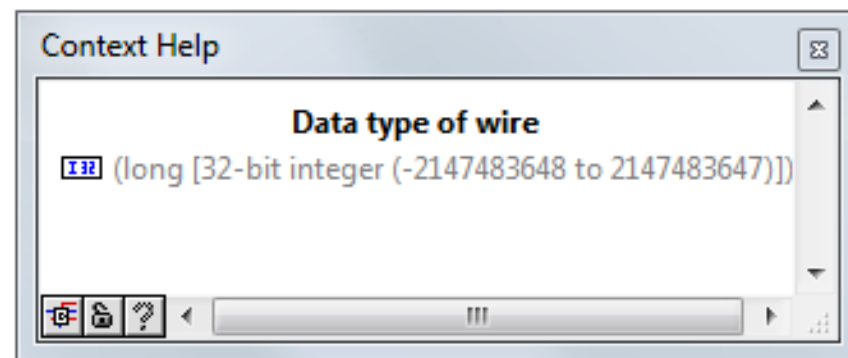
- Wires transfer data between block diagram objects.
- Wires are different colors, styles, and thicknesses, depending on their data types.
- A broken wire appears as a dashed black line with a red X in the middle.



	Floating-point	Integer	String	Boolean
Scalar				
1-D Array				
2-D Array				

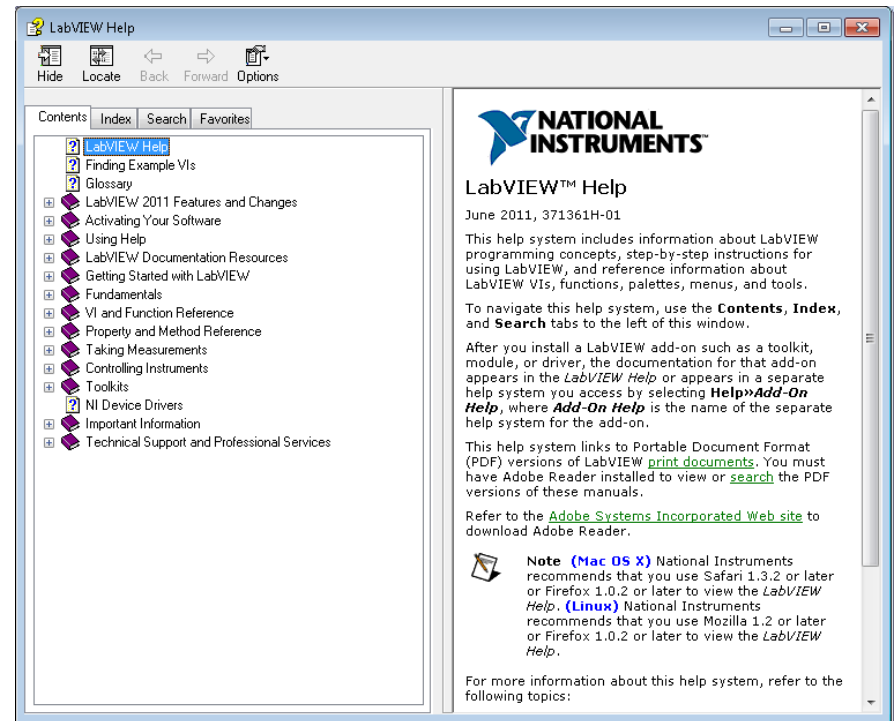
Context Help

- Displays basic information about wires and nodes when you move the cursor over an object.
- Can be shown or hidden in the following ways.
 - Select **Help»Show Context Help** from the LabVIEW menu.
 - Press <Ctrl-H>.
 - Click the following button on the toolbar: 



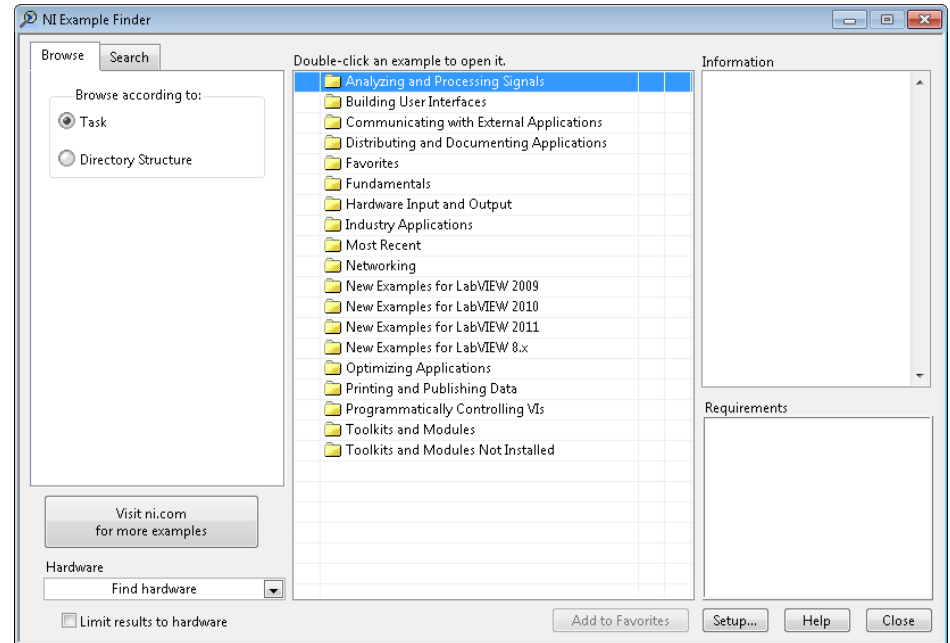
LabVIEW Help

- Contains detailed descriptions and instructions for most palettes, menus, tools, VIs, and functions.
- Can be accessed by:
 - Selecting **Help» LabVIEW Help** from the menu.
 - Clicking the **Detailed help** link in the **Context Help** window.
 - Right-clicking an object and selecting **Help** from the shortcut menu.



Examples

- LabVIEW includes hundreds of example VIs.
- Use NI Example Finder to browse and search installed examples.
 - Select **Help»Find Examples** in the menu.
- Click the example buttons in *LabVIEW Help* topics.



Open example



Find related examples

F. Searching for Controls, VIs, and Functions

Palettes

Quick Drop

NI Global Search

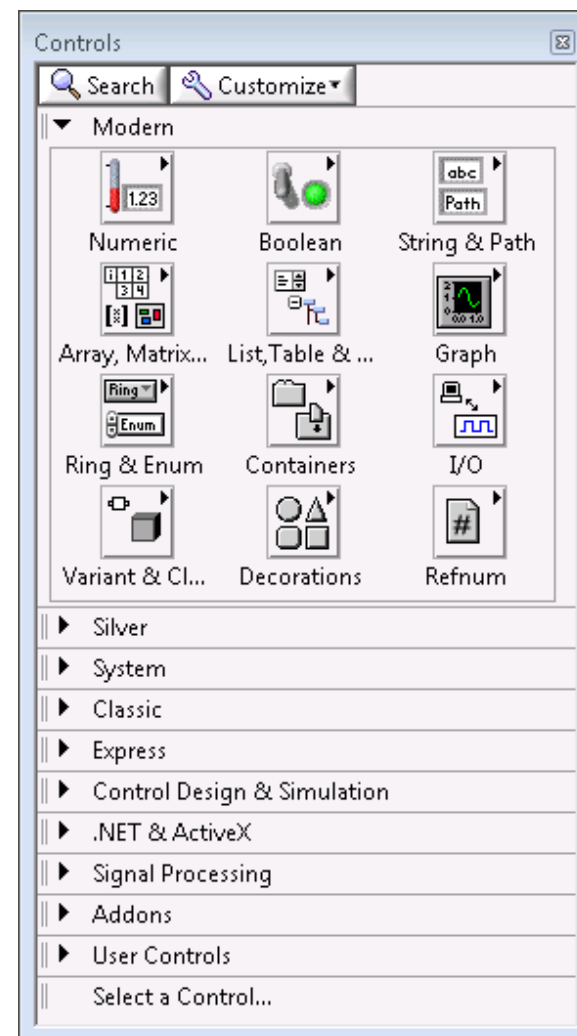
Searching for Controls, VIs, and Functions

Ways to find controls, VIs, and functions:

- Search or navigate the palettes.
 - Controls palette
 - Functions palette
- Search by name of object.
 - Quick Drop dialog box
- Search palettes, *LabVIEW Help*, and `ni.com`.
 - Search text box in toolbar

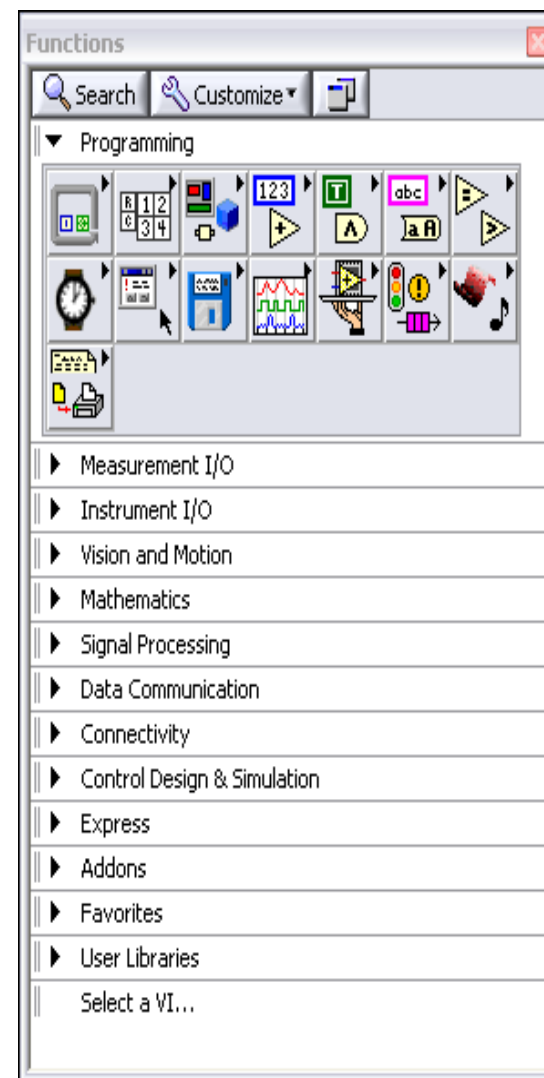
Controls Palette

- Contains the controls and indicators you use to create the front panel.
- Navigate the subpalettes or use the **Search** button to search the Controls palette.



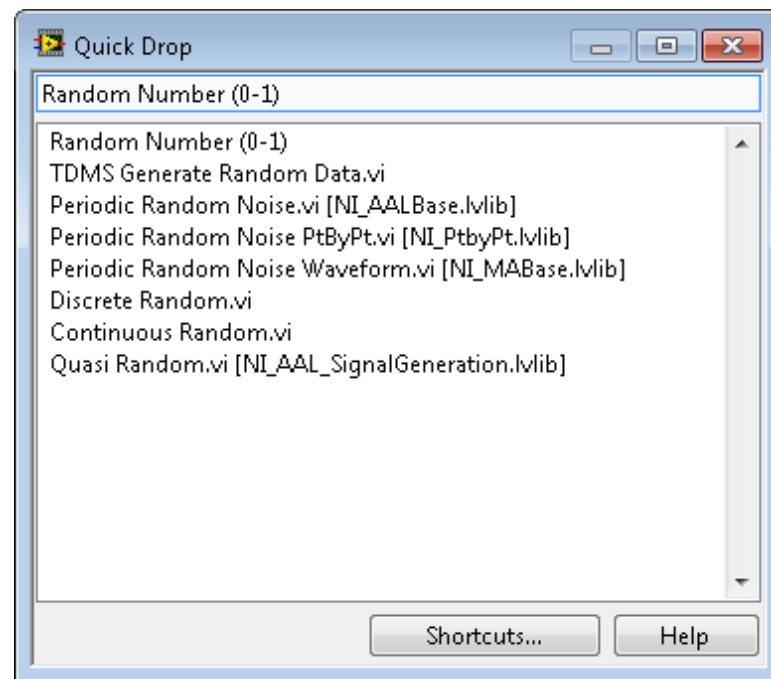
Functions Palette

- Contains the VIs, functions, and constants you use to create the block diagram.
- Navigate the subpalettes or use the **Search** button to search the Functions palette.



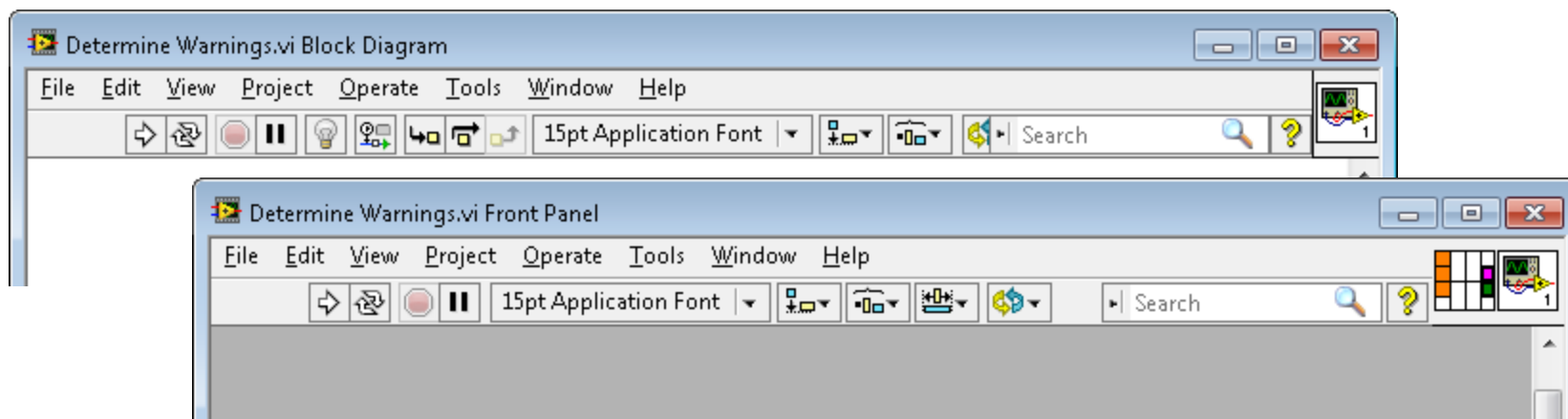
Searching with Quick Drop

- Lets you quickly find controls, functions, VIs, and other items by name.
- Press the <Ctrl-Space> keys to display the Quick Drop dialog box.



Global Search

Use the Search bar in the top right of the front panel and block diagram windows to search palettes, *LabVIEW Help*, and *ni.com*.



Search for Controls, VIs, and Functions

- Configure palettes to customize visible palettes.
- Search and navigate the palettes.
- Search for help using global search.
- Use Quick Drop to search by name.

DEMONSTRATION

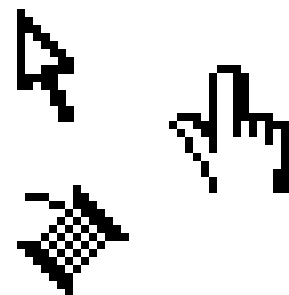
G. Selecting a Tool

Selecting a Tool

Block Diagram Clean-Up

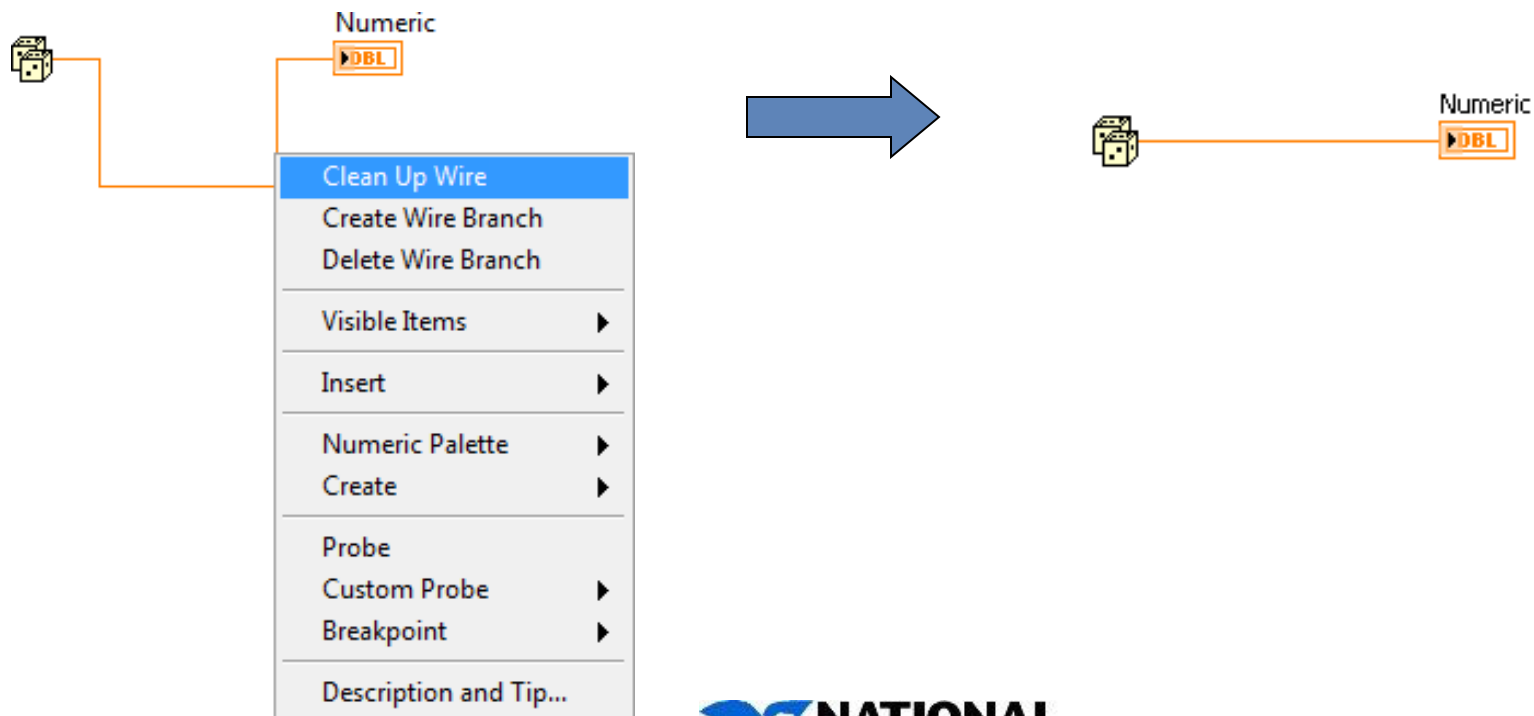
Selecting a Tool

- A tool is a special operating mode of the mouse cursor.
- Create, modify, and debug VIs using the tools provided by LabVIEW.
- By default, LabVIEW automatically selects tools based on the context of the cursor.
- If you need more control, use the **Tools** palette to select a specific tool.
 - Select **View»Tools Palette** to open the **Tools** palette.



Wiring Tips

- Press <Ctrl-B> to delete all broken wires.
- Right-click and select **Clean Up Wire** to reroute the wire.

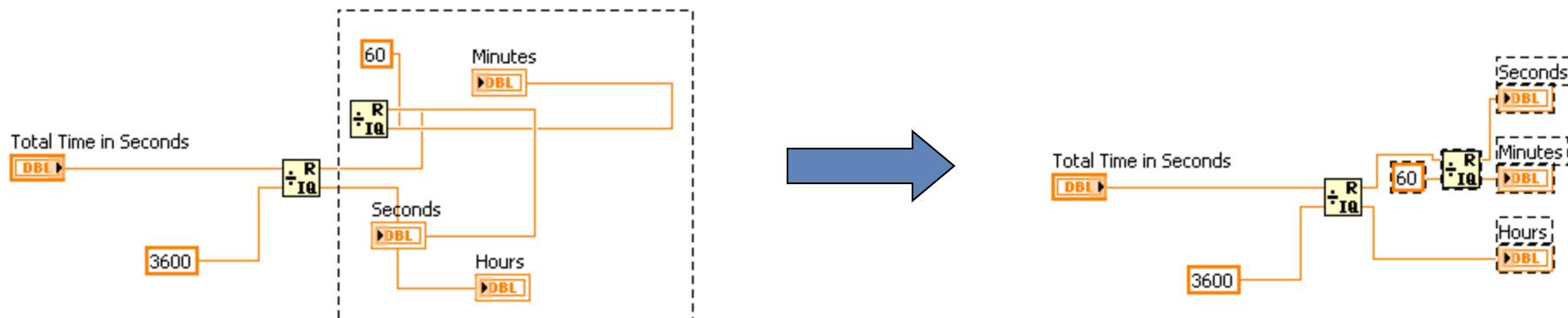


Wiring Tips – Clean Up Diagram



Use the Clean Up Diagram tool to reroute multiple wires and objects and to improve readability.

1. Select a section of your block diagram.
2. Click the Clean Up Diagram button on the block diagram toolbar.



Cloning and Moving Items

- Clone an object in LabVIEW using the following steps:
 1. Select the Positioning tool.
 2. Press the <Ctrl> key while clicking an object.
 3. Drag the copy to new location.
- Move an object using the following steps:
 1. Select the Positioning tool.
 2. Click and drag the object to new location.

Note: Avoid cutting and pasting objects as this can impact related items.
For example, cutting and pasting a block diagram terminal also moves the front panel object.

Selecting, Editing, Resizing and Wiring

- Select item to move, copy, or delete
- Edit text
- Resize an object
- Wire terminals and nodes

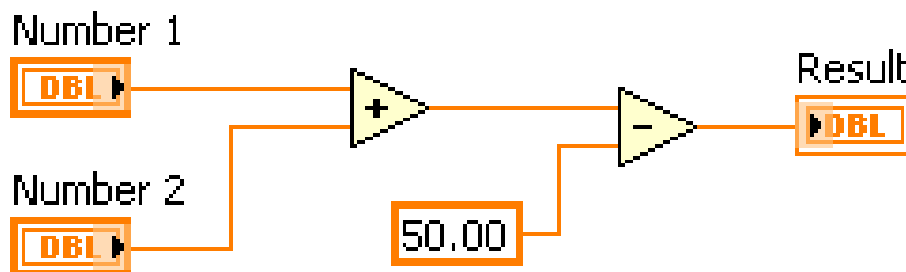
DEMONSTRATION

H. Dataflow

Dataflow

LabVIEW follows a dataflow model for running VIs.

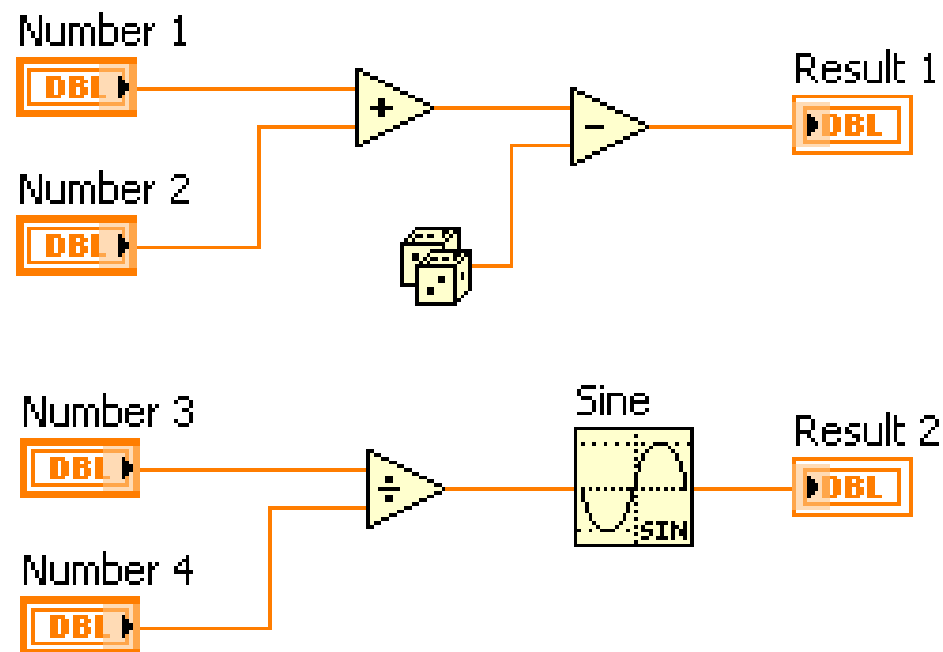
- A node executes only when data are available at all of its required input terminals.
- A node supplies data to the output terminals only when the node finishes execution.



Dataflow – Quiz

Which node executes first?

- a) Add
- b) Subtract
- c) Random Number
- d) Divide
- e) Sine

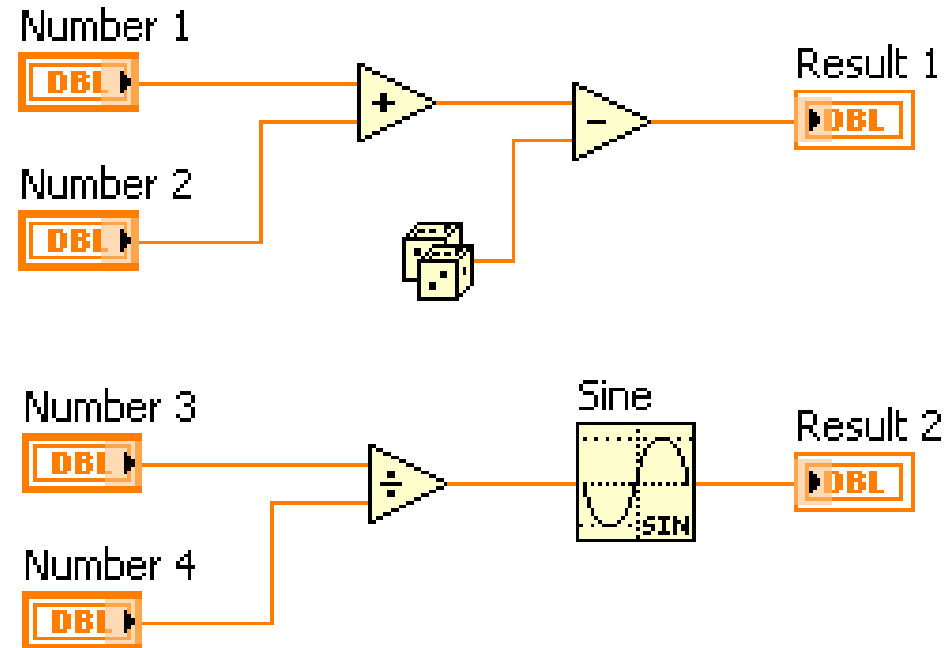


Dataflow – Quiz Answers

No single correct answer.

Which node executes first?

- a) Add – **Possibly**
- b) Subtract – **Definitely not**
- c) Random Number – **Possibly**
- d) Divide – **Possibly**
- e) Sine – **Definitely not**



Group Exercise

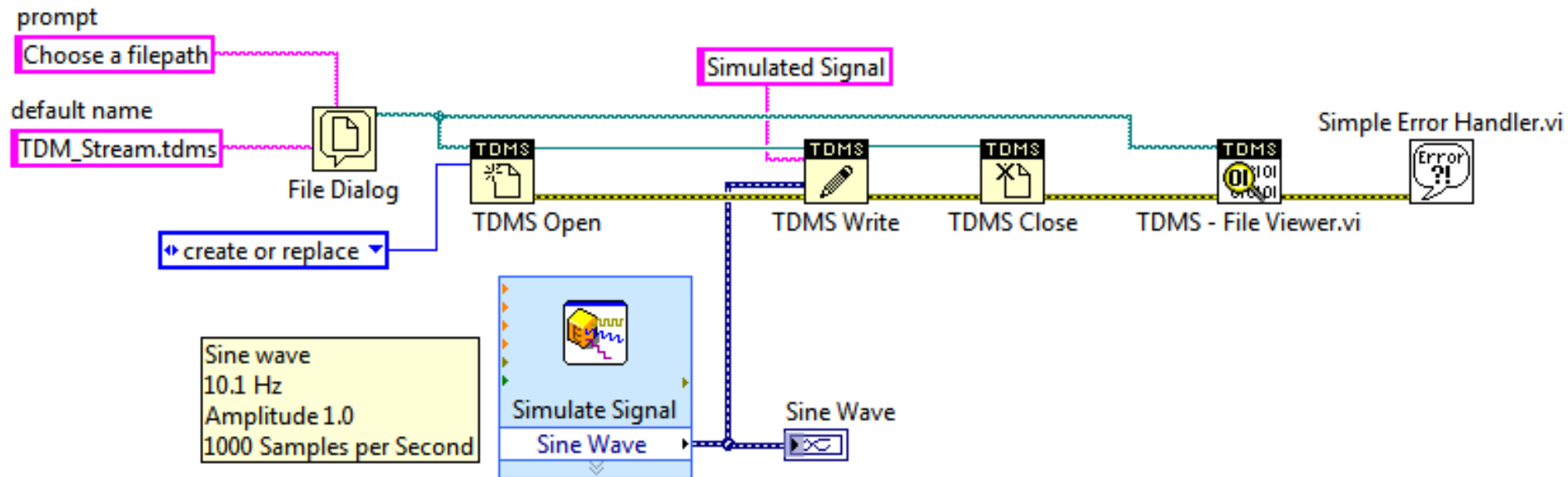
Concept: Dataflow

Identify dataflow execution order in the following block diagrams.

GOAL

Group Exercise

Concept: Dataflow

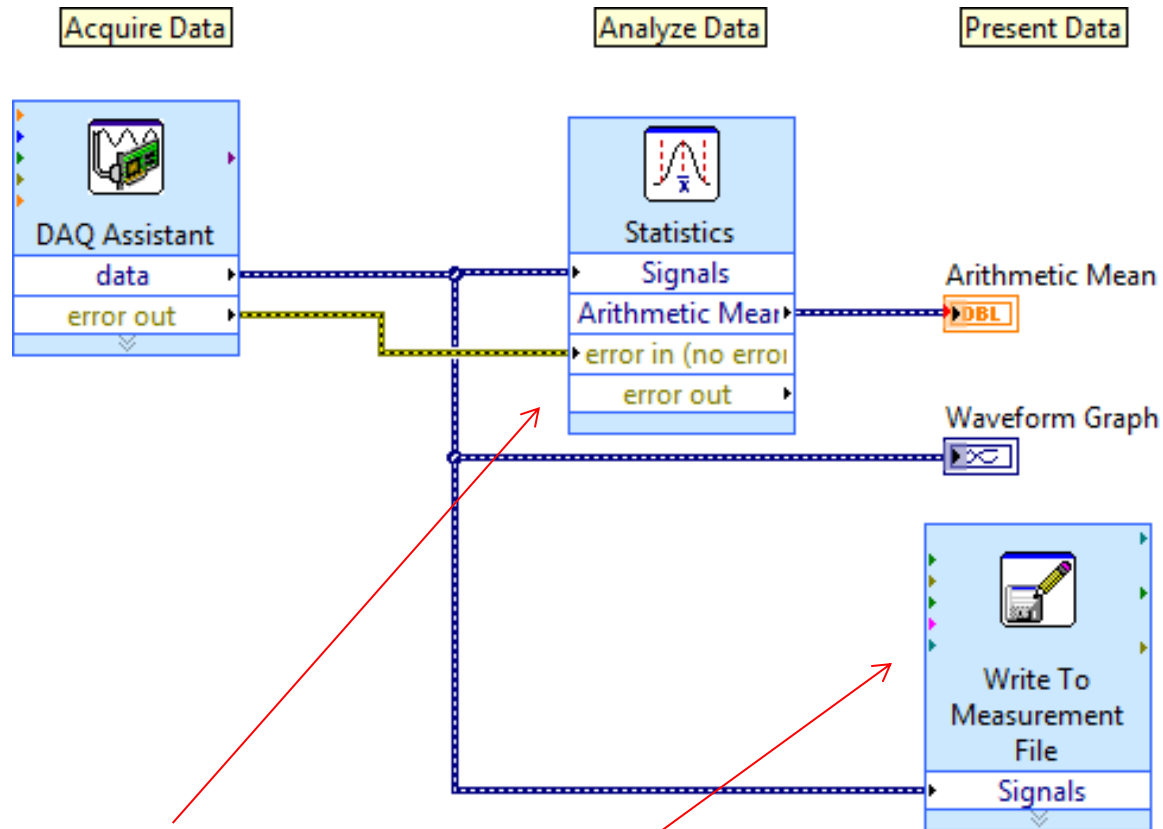


Which node executes first? Last?
Where are the data dependencies?

DISCUSSION

Group Exercise

Concept: Dataflow

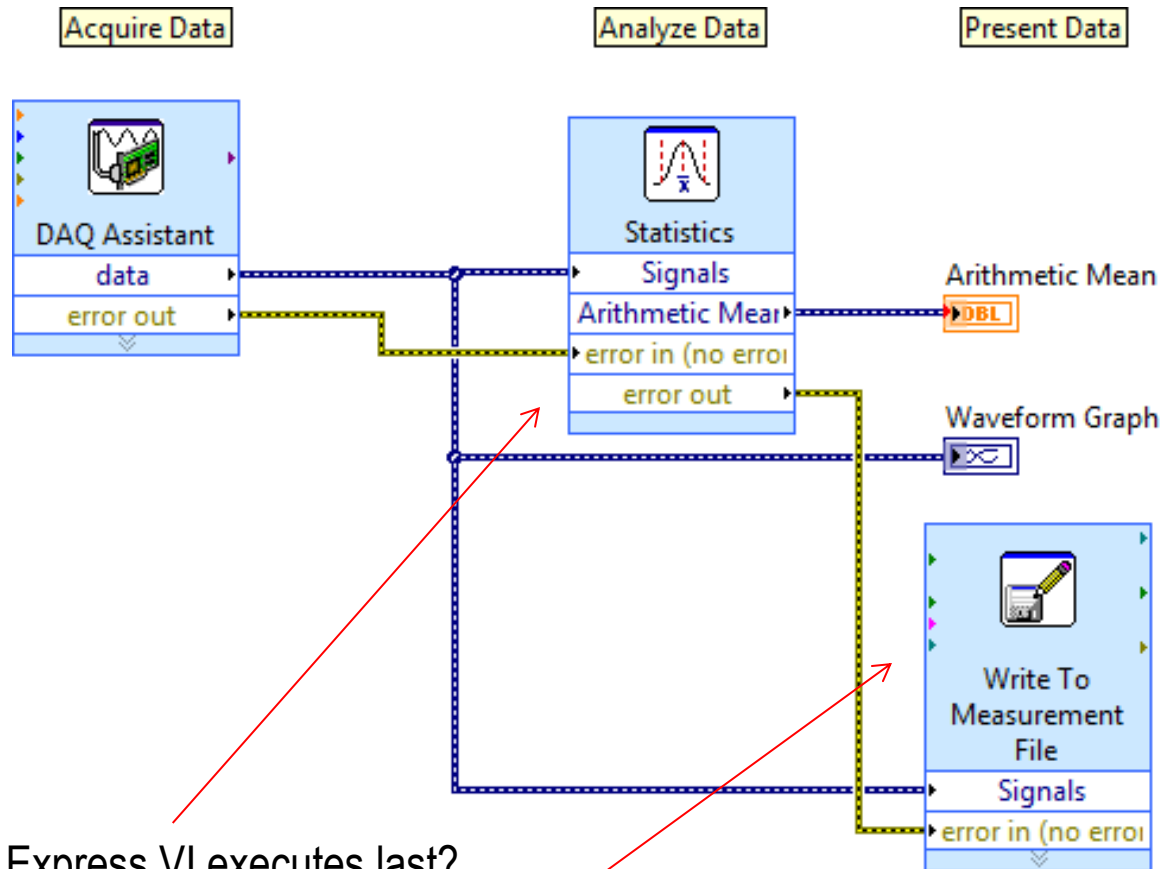


Which Express VI executes last?

DISCUSSION

Group Exercise

Concept: Dataflow

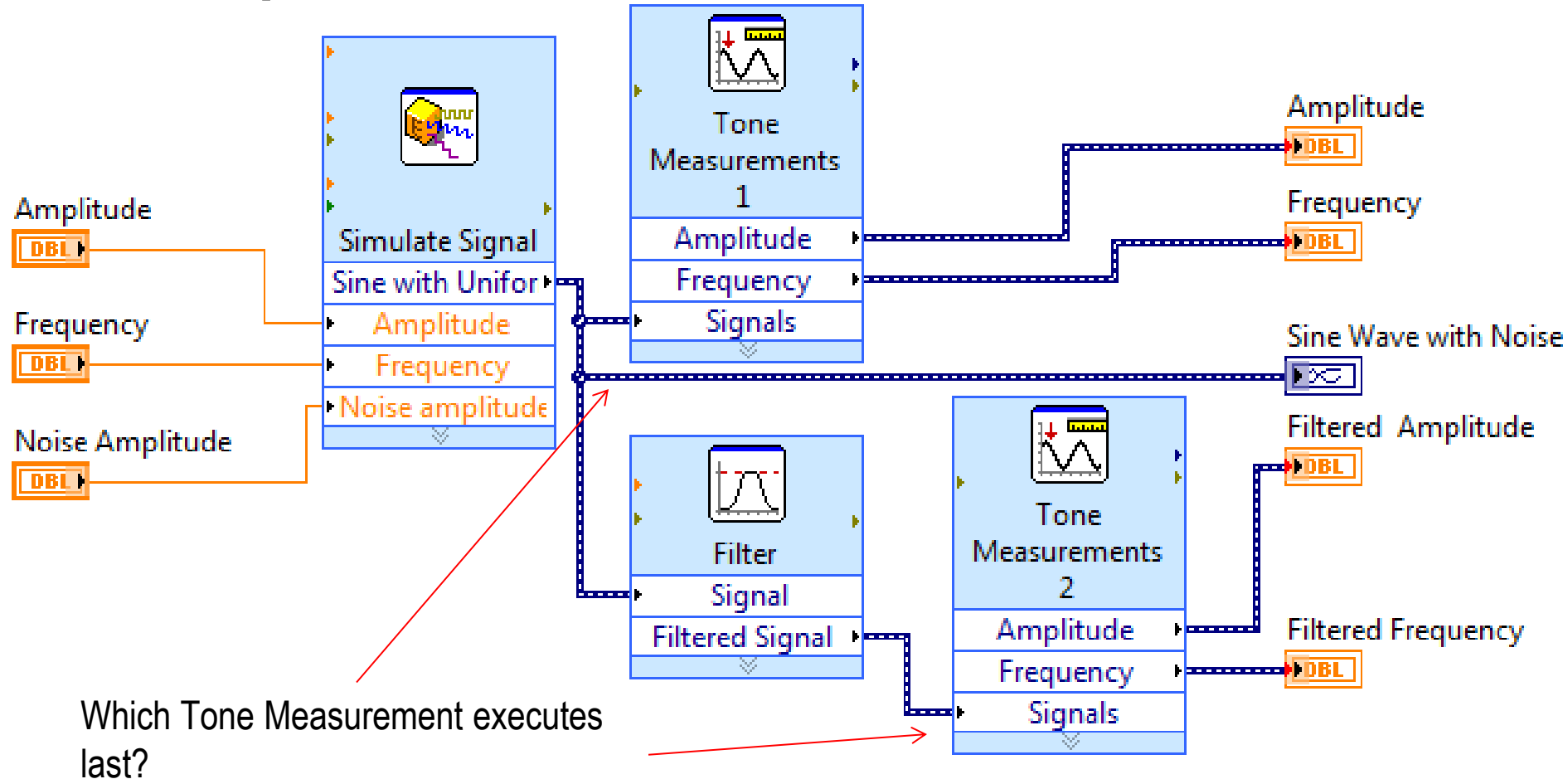


Which Express VI executes last?

DISCUSSION

Group Exercise

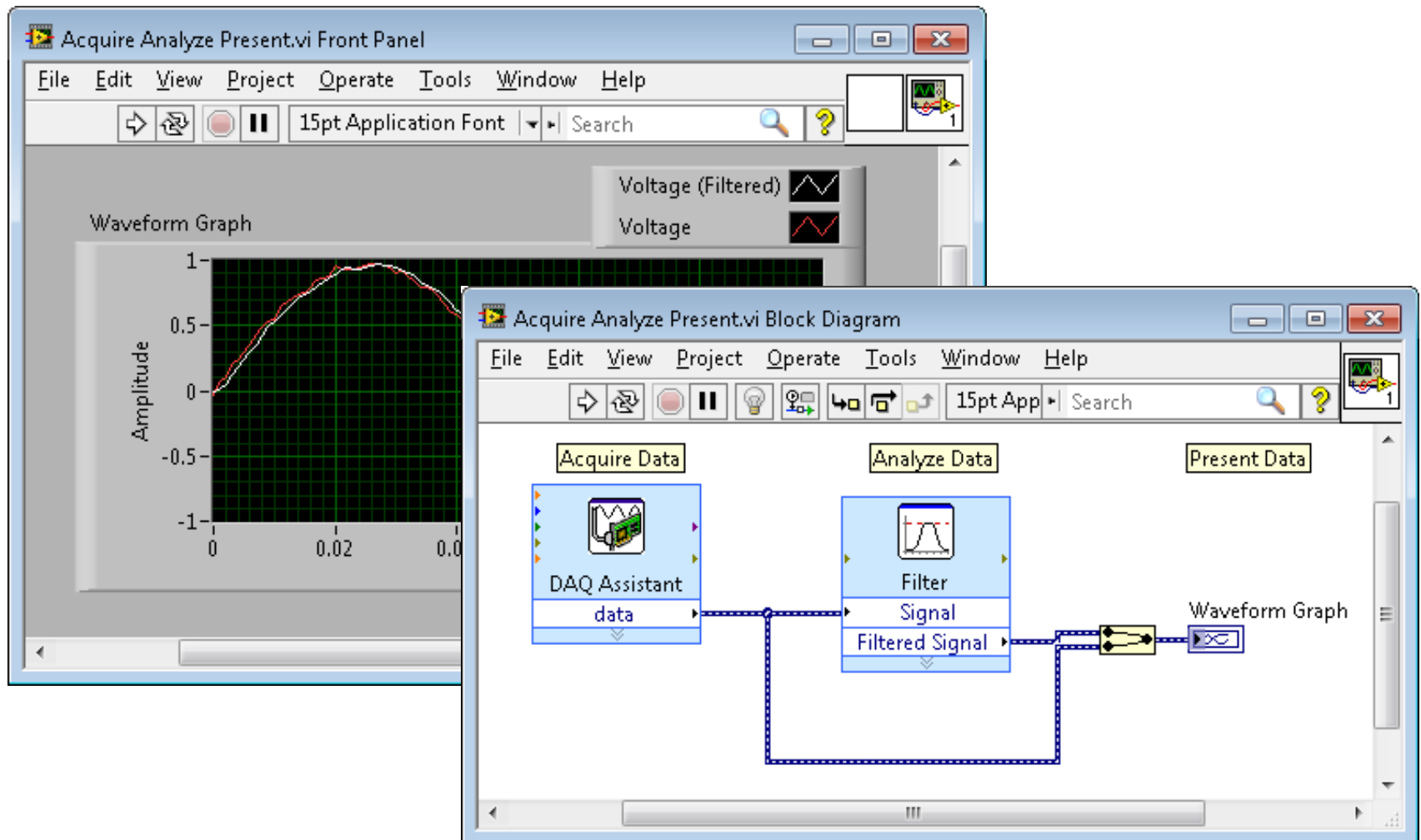
Concept: Dataflow



DISCUSSION

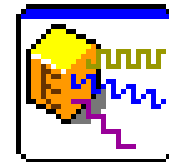
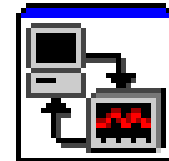
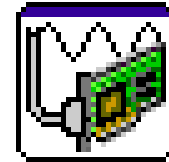
I. Building a Simple VI

Building a Simple VI



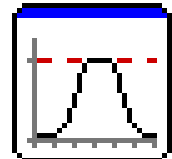
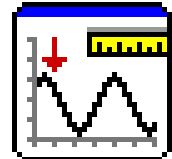
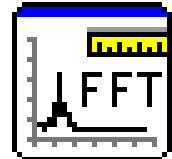
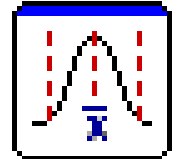
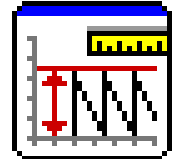
Acquire Express VIs

- DAQ Assistant Express VI
- Instrument I/O Assistant Express VI
- Simulate Signal Express VI
- Read from Measurement File Express VI



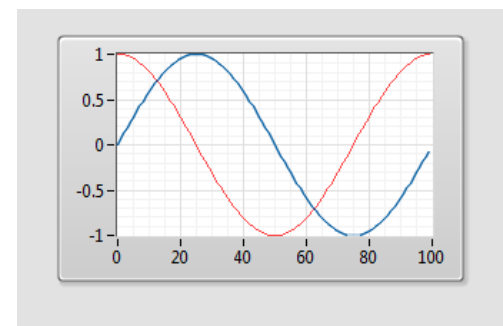
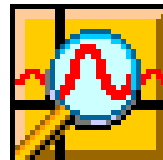
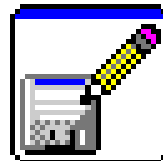
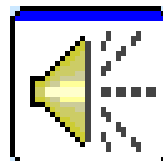
Analyze Express VIs

- Amplitude and Level Measurements Express VI
- Statistics Express VI
- Spectral Measurements Express VI
- Tone Measurements Express VI
- Filter Express VI



Present Express VIs and Indicators

- Display Message Express VI
- Play Waveform Express VI
- Report Express VI
- Write Measurement File Express VI
- DIAdem Report Express VI



Building and Running a VI

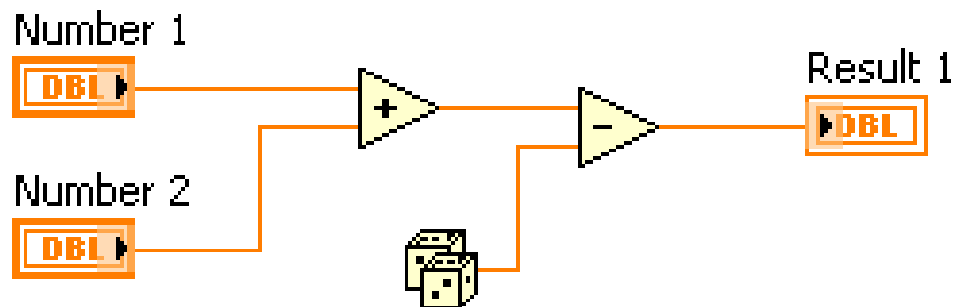
1. Place Express VI on the block diagram.
2. Configure the dialog box that opens.
3. Wire Express VIs together.
4. Save and run the VI.

The **Run** button appears broken when the VI you are creating or editing contains errors.



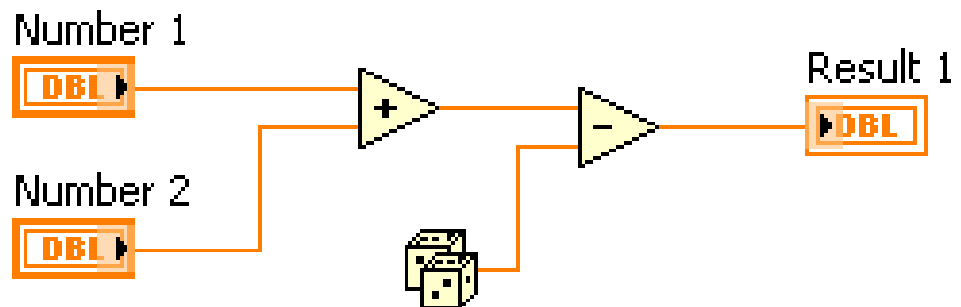
Summary—Quiz

1. Which function executes first:
Add or Subtract?
 - a) Add
 - b) Subtract
 - c) Unknown



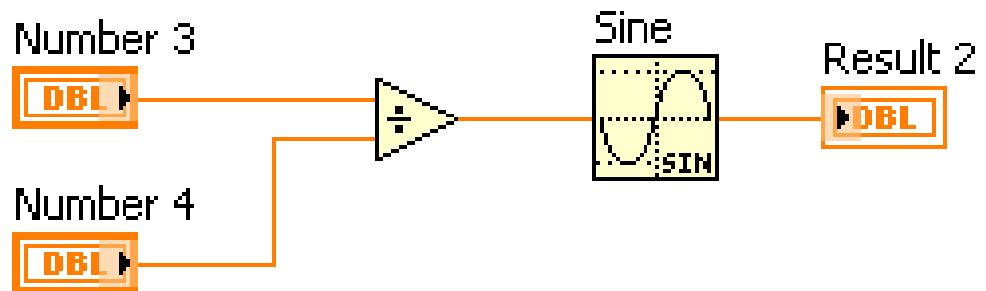
Summary—Quiz Answer

1. Which function executes first:
Add or Subtract?
 - a) Add
 - b) Subtract
 - c) Unknown



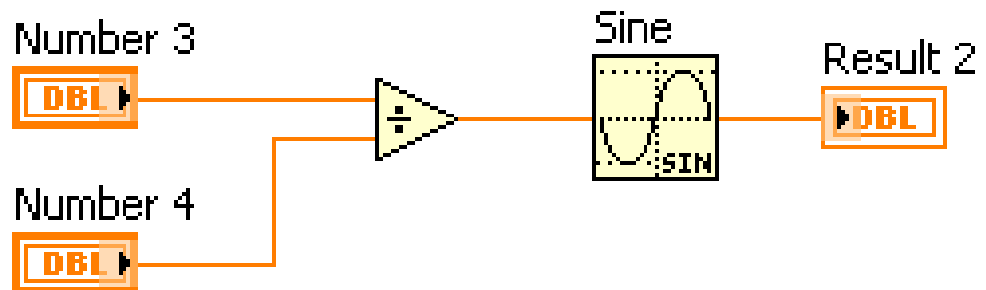
Summary—Quiz

2. Which function executes first:
Sine or Divide?
- a) Sine
 - b) Divide
 - c) Unknown



Summary—Quiz Answer

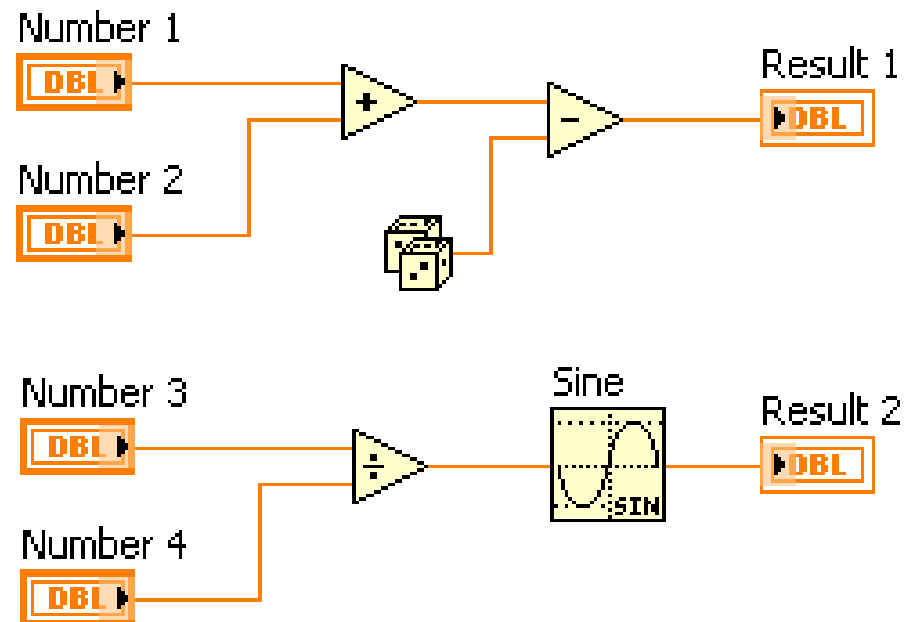
2. Which function executes first:
Sine or Divide?
- a) Sine
 - b) Divide**
 - c) Unknown



Summary—Quiz

3. Which of the following functions executes first: Random Number, Add or Divide?

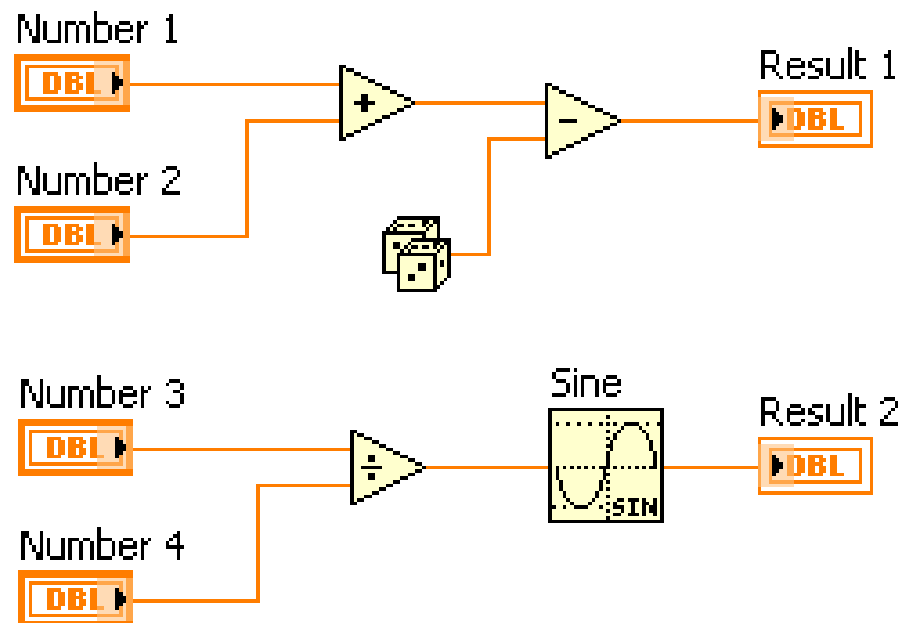
- a) Random Number
- b) Divide
- c) Add
- d) Unknown



Summary—Quiz Answer

3. Which of the following functions executes first: Random Number, Add or Divide?

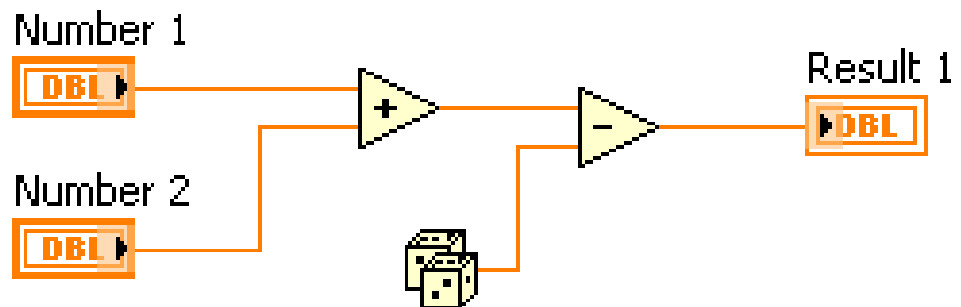
- a) Random Number
- b) Divide
- c) Add
- d) **Unknown**



Summary—Quiz

4. Which of the following functions executes last:
Random Number, Subtract or
Add?

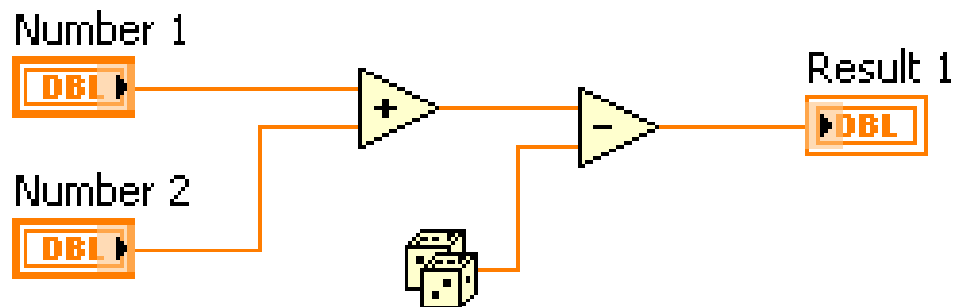
- a) Random Number
- b) Subtract
- c) Add
- d) Unknown



Summary—Quiz Answer

4. Which of the following functions executes last:
Random Number, Subtract or Add?

- a) Random Number
- b) Subtract**
- c) Add
- d) Unknown



Summary—Quiz

5. What are the three parts of a VI?

- a) Front panel
- b) Block diagram
- c) Project
- d) Icon/Connector pane

Summary—Quiz Answer

5. What are the three parts of a VI?

a) Front panel

b) Block diagram

c) Project

d) Icon/Connector pane

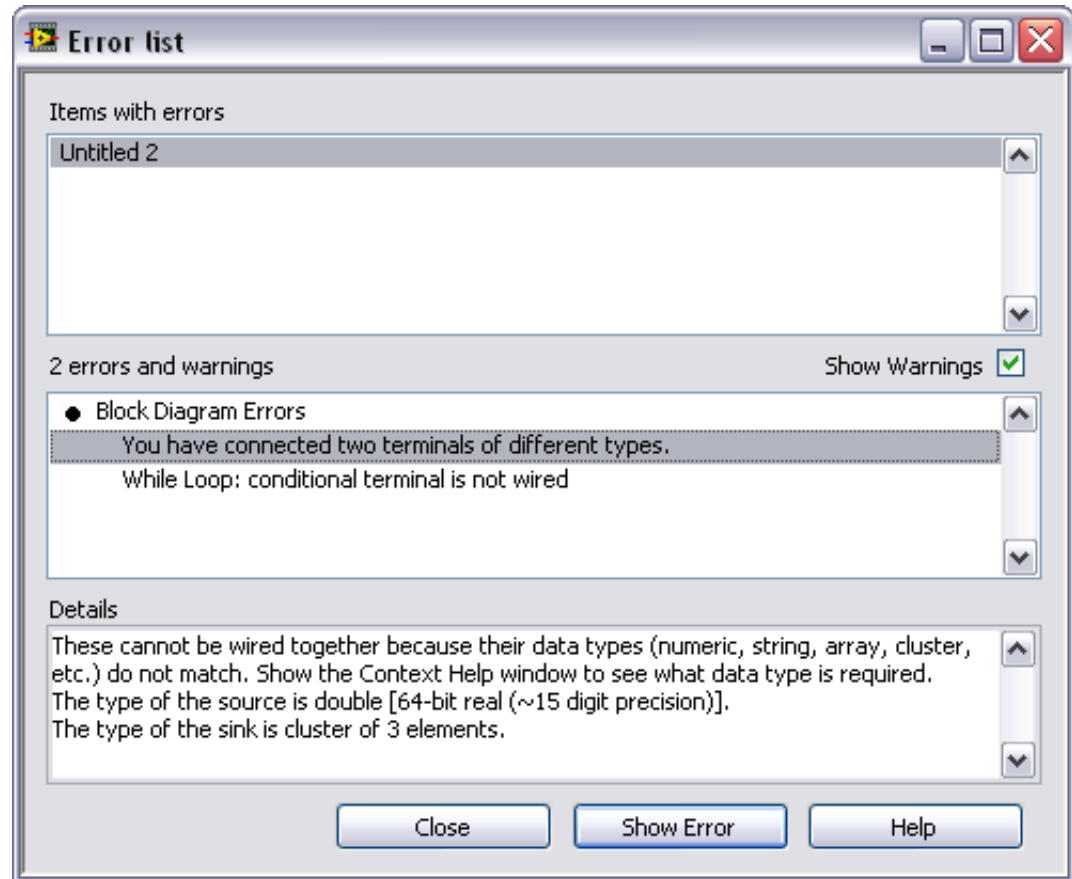
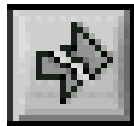
Troubleshooting and Debugging VIs

TOPICS

- A. Correcting Broken VIs
- B. Debugging Techniques
- C. Undefined or Unexpected Data

A. Correcting Broken VIs

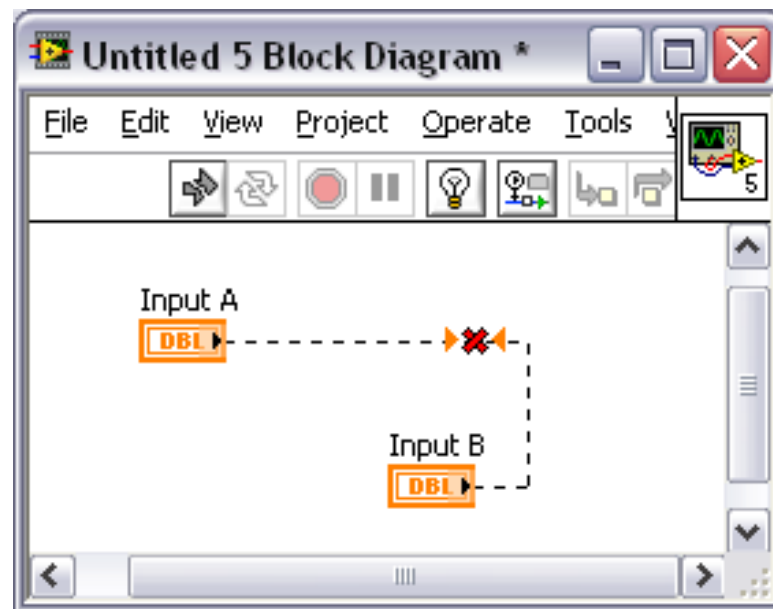
Correcting Broken VIs



Broken Run arrow → VI cannot be compiled → VI cannot be executed

Common Causes of Broken VIs

- Broken wires exist on the block diagram.
 - You wired a Boolean control to a String indicator.
 - You wired a numeric control to a numeric control.
- A required block diagram terminal is unwired.
- A subVI is broken.



B. Debugging Techniques

Execution Highlighting

Single-Stepping

Probes

Breakpoints

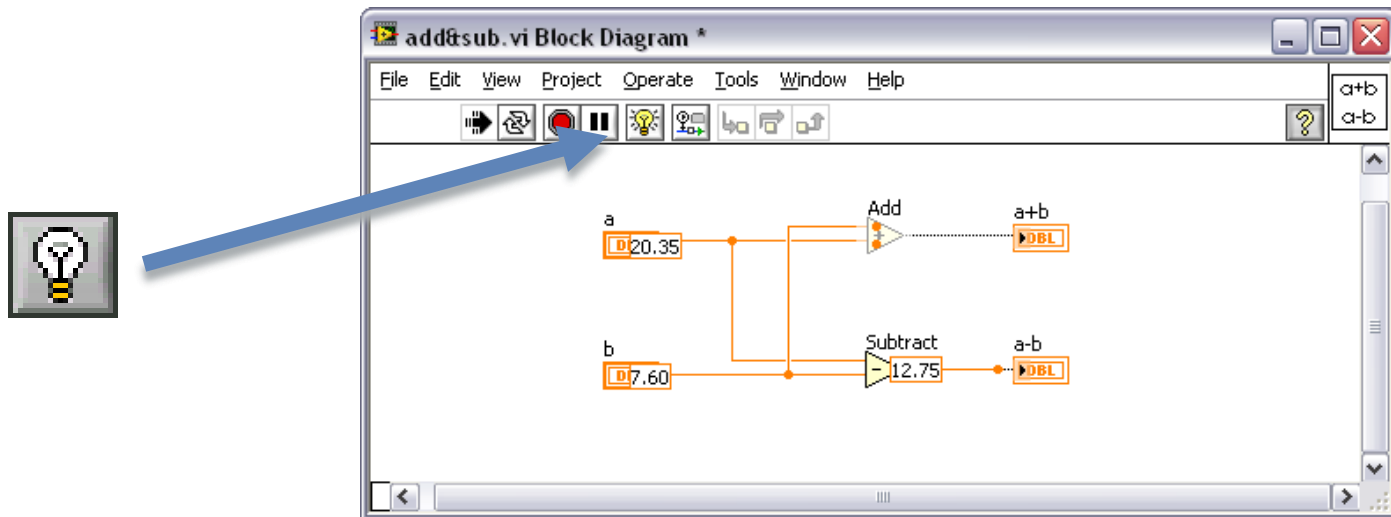
Debugging Techniques

What to look for if a VI produces unexpected data or behavior:

- Are there any unwired or hidden subVIs?
- Is the default data correct?
- Does the VI pass undefined data?
- Are numeric representations correct?
- Are node executed in the correct order?

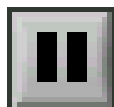
Execution Highlighting

- Use execution highlighting to watch the data flow through the block diagram.
- If the VI runs more slowly than expected, confirm that you turned off execution highlighting in subVIs.



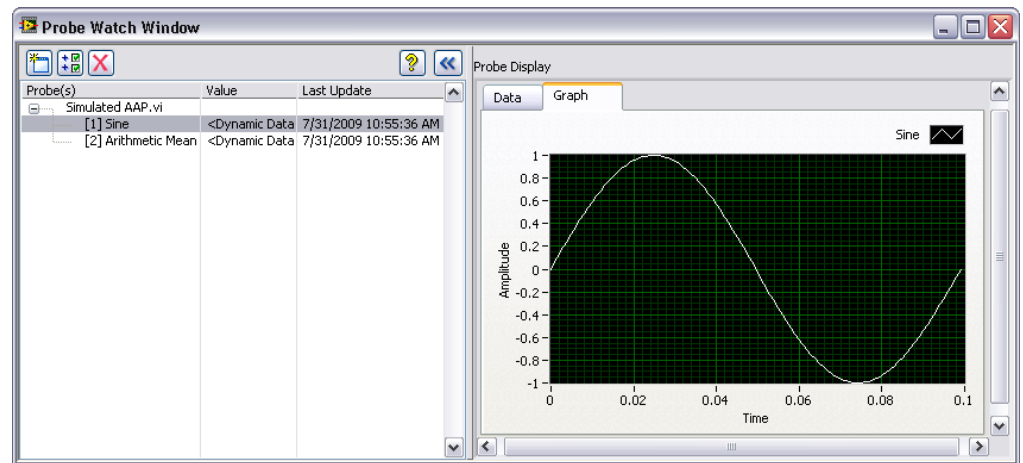
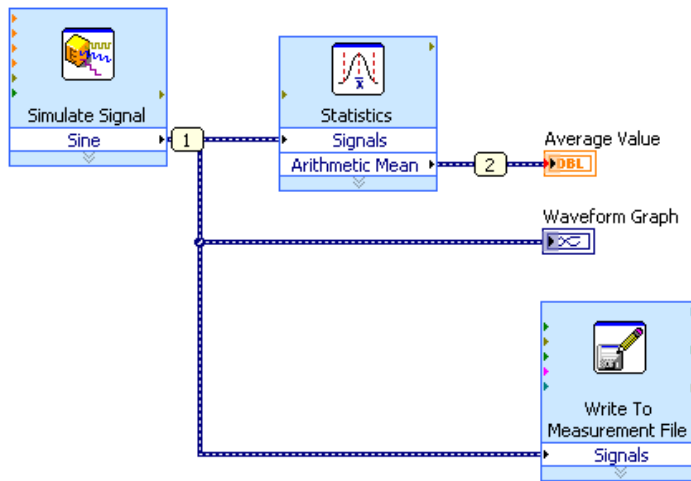
Single-Stepping

- Single-step through the VI to view each action of the VI on the block diagram.
- Suspend the execution of a subVI to edit values of controls and indicators, to control the number of times it runs, or to go back to the beginning of the execution of the subVI.
 - Open subVI and select **Operate»Suspend When Called** from the shortcut menu.



Probes

- Use the Probe tool to observe intermediate data values and check the error output of VIs and functions, especially those performing I/O.
- Specify to retain the values in the wires so that you can probe wires for data after execution.



Breakpoints

- When you reach a breakpoint during execution, the VI pauses and the **Pause** button appears red. 
- You can take the following actions at a breakpoint:
 - Single-step through execution using the single-stepping buttons.
 - Probe wires to check intermediate values.
 - Change values of front panel controls.
 - Click the Pause button to continue running to the next breakpoint or until the VI finishes running.

C. Undefined or Unexpected Data

Undefined or Unexpected Data

Check for unexpected Inf values or NaN values in your mathematical operations:

- ∞ (Inf)
 - Infinity
 - Produced by dividing a number by zero.
- NaN
 - Not a number
 - Produced by invalid operations, such as taking the square root of a negative number.

Exercise

Concept: Debugging

Use the debugging tools built into LabVIEW.

GOAL

Exercise

Concept: Debugging

- If you have a VI with a broken **Run** arrow, what should you do first?
- After you fixed your broken **Run** arrow, your VI results show unexpected data. What can you do next?

DISCUSSION

Summary—Quiz

1. Which of the following will result in a broken run arrow?
 - a) A subVI is broken.
 - b) The diagram includes a divide by zero.
 - c) A required subVI input is unwired.
 - d) A Boolean terminal is wired to a numeric indicator.

Summary—Quiz Answer

1. Which of the following will result in a broken run arrow?
 - a) A subVI is broken.**
 - b) The diagram includes a divide by zero.
 - c) A required subVI input is unwired.
 - d) A Boolean terminal is wired to a numeric indicator.

Implementing a VI

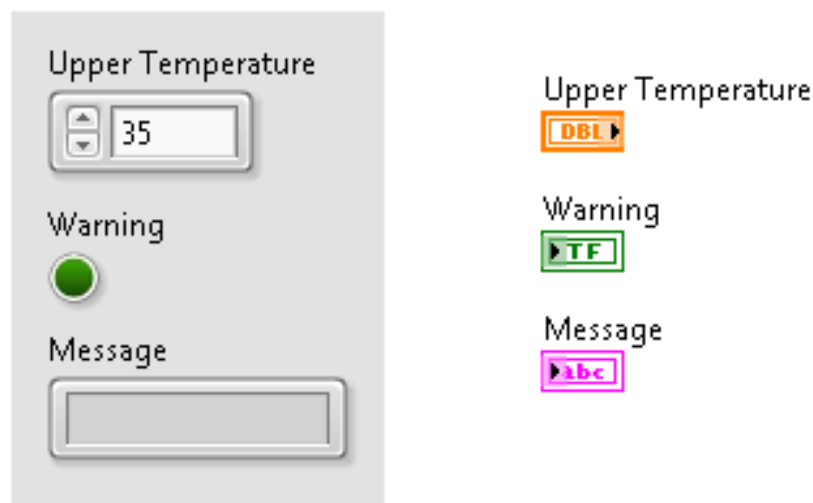
TOPICS

- A. Front Panel Basics
- B. LabVIEW Data Types
- C. Documenting Code
- D. While Loops
- E. For Loops
- F. Timing a VI
- G. Data Feedback in Loops
- H. Plotting Data – Waveform Chart
- I. Case Structures

A. Front Panel Basics

Front Panel Basics

Front panel controls and indicators create terminals on the block diagram.



B. LabVIEW Data Types

Shortcut Menu and Properties Dialog Box

Numeric Types

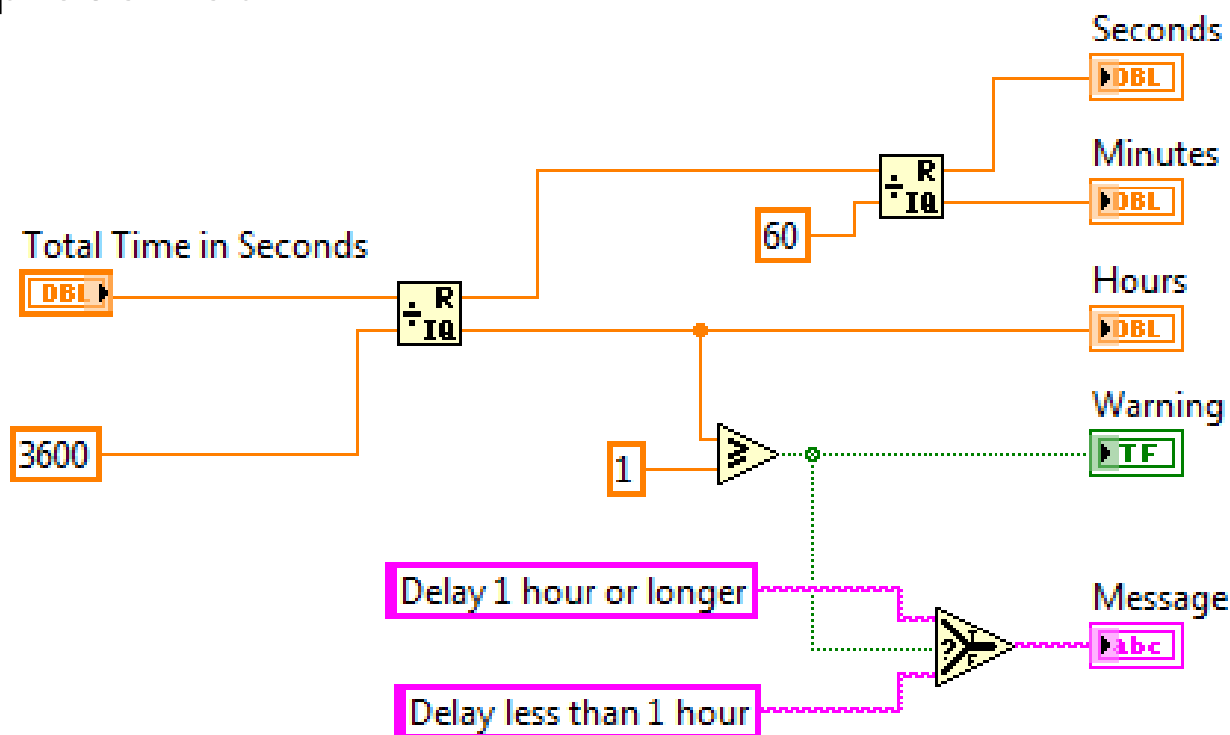
Boolean Types

String Types

Enums and Other Types

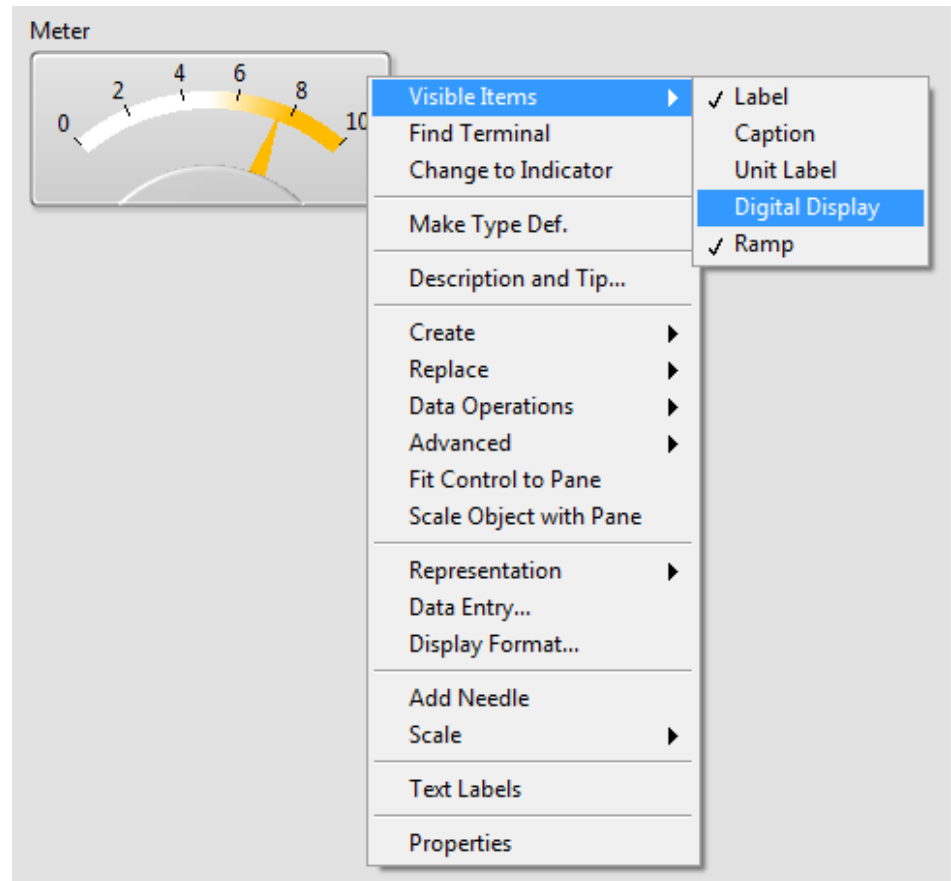
LabVIEW Data Types

Terminals visually communicate information about the data type represented



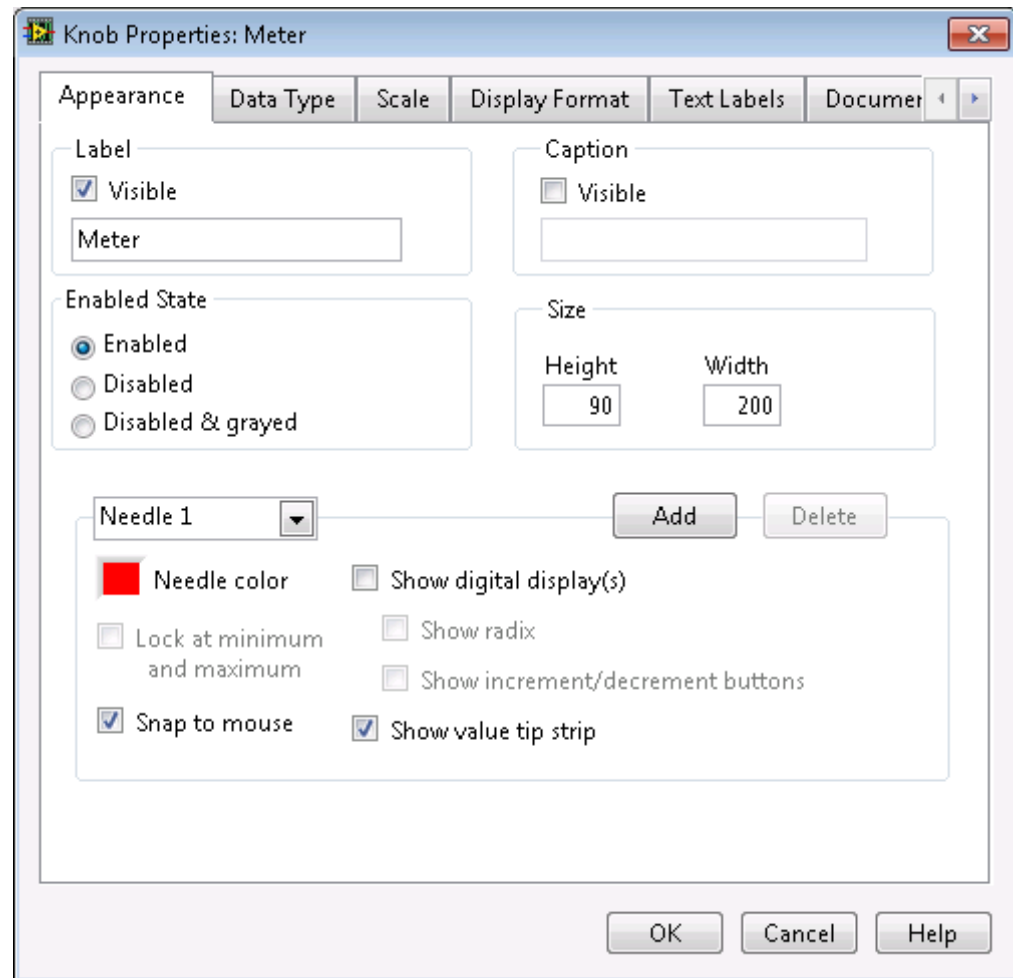
Shortcut Menu

- All LabVIEW objects have associated shortcut menus.
- Use shortcut menu items to change the look or behavior of objects.
- To access the shortcut menu, right-click the object.



Properties Dialog Box

- All LabVIEW objects have properties.
- To access properties, right-click the object and select **Properties**.
- Property options are similar to shortcut menu options.
- Select multiple objects to simultaneously configure shared properties.



Numerics

Various data type representations:

- Floating-point
- Unsigned integers
- Signed integers

DBL



I32



U32



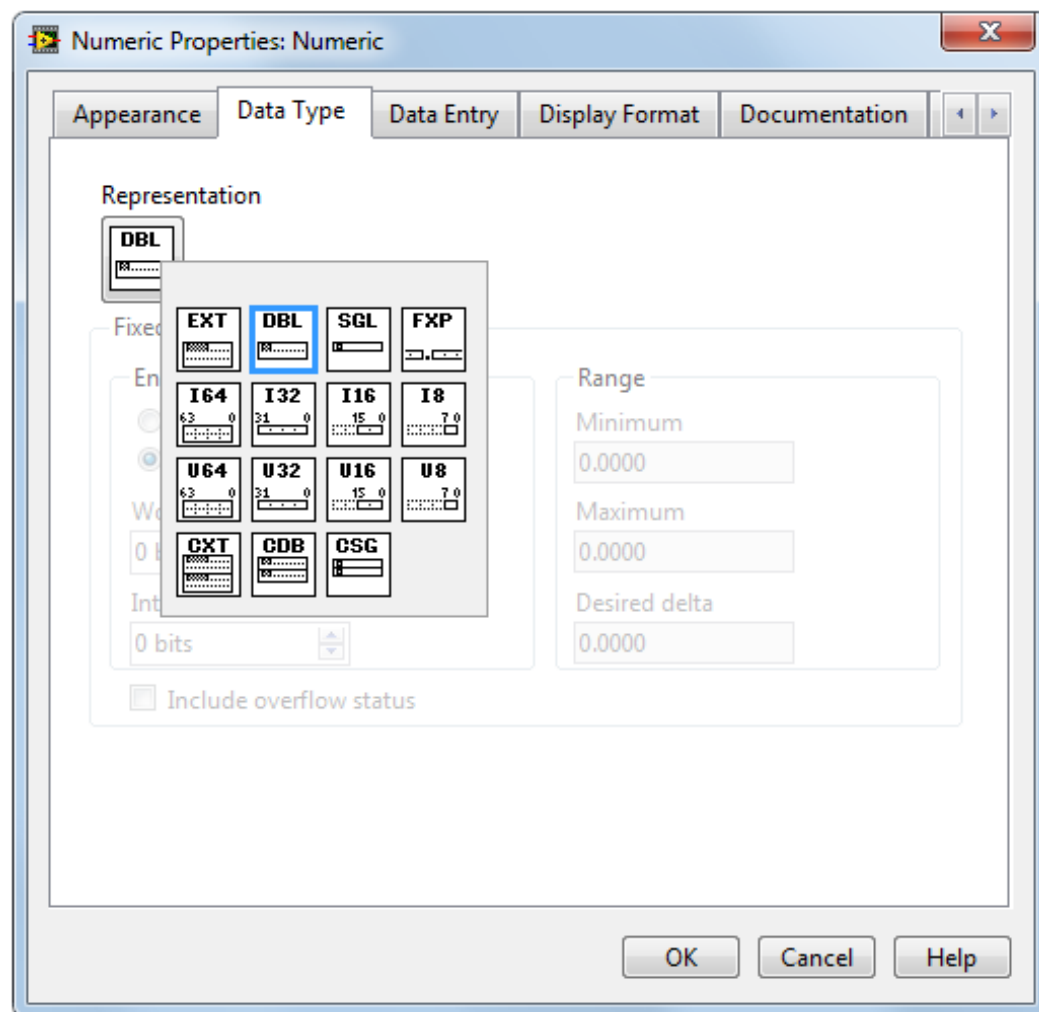
SGL



I8

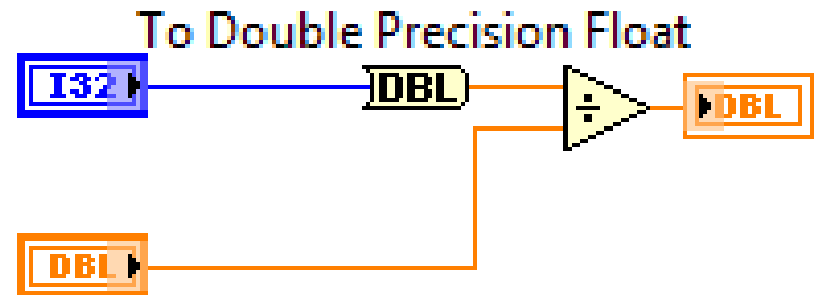
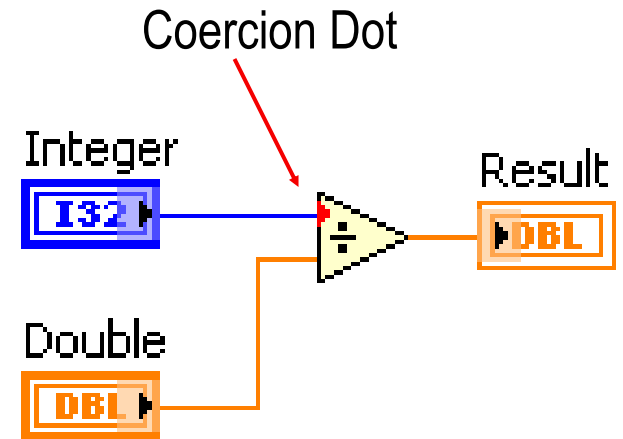


U8



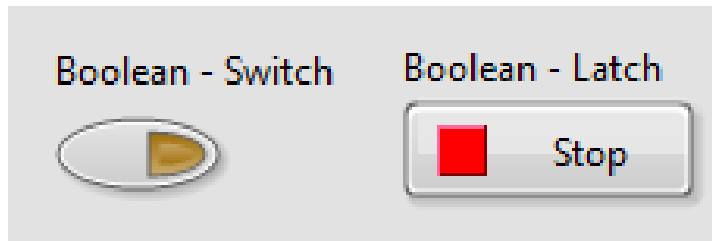
Numeric Conversion

- Coercion dots indicate that LabVIEW converted the value passed into a node to a different representation.
 - Occurs when a node expects an input with a different representation.
- LabVIEW chooses the representation that uses more bits.
- Avoid coercion by programmatically converting to a matching data type.



Booleans

- Behavior of Boolean controls is specified by the mechanical action.
- Boolean have only TRUE/FALSE values.



Boolean - Switch



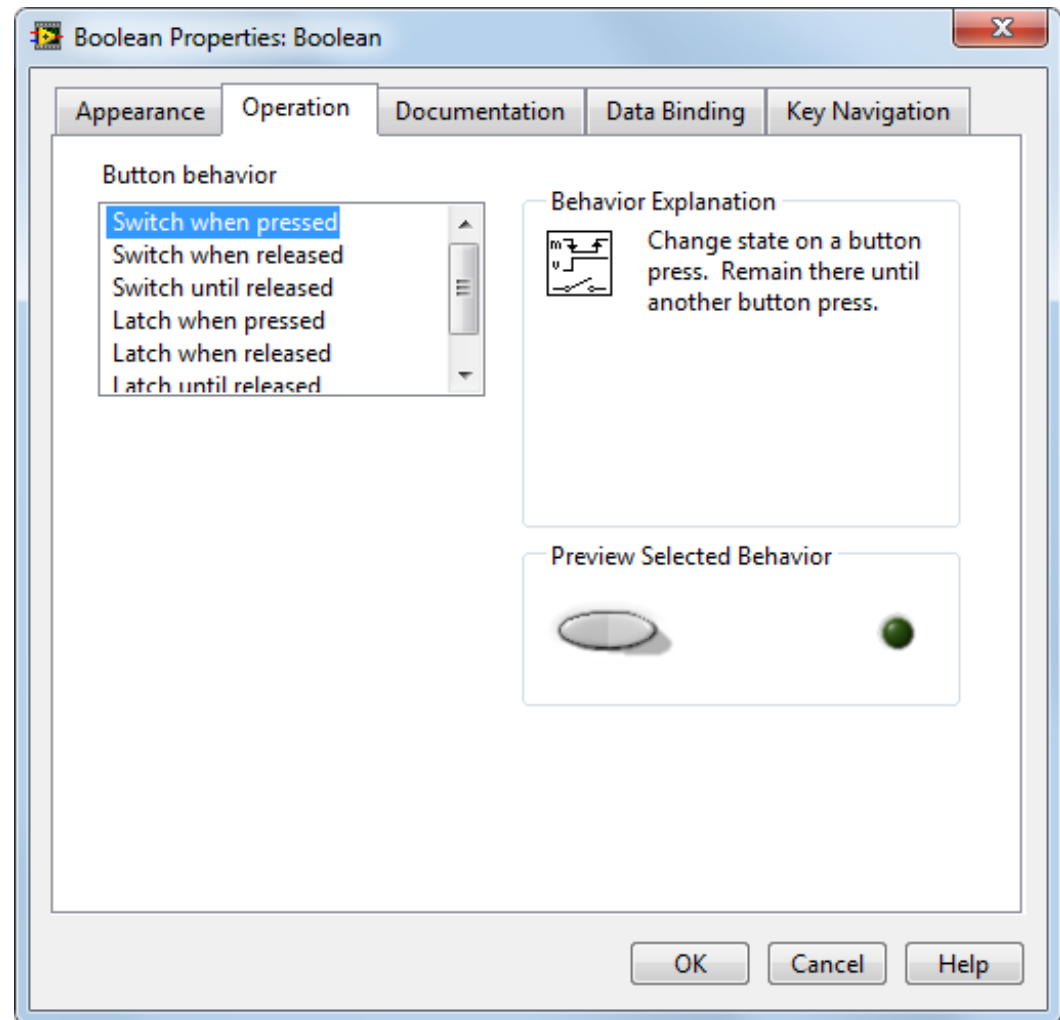
True Constant



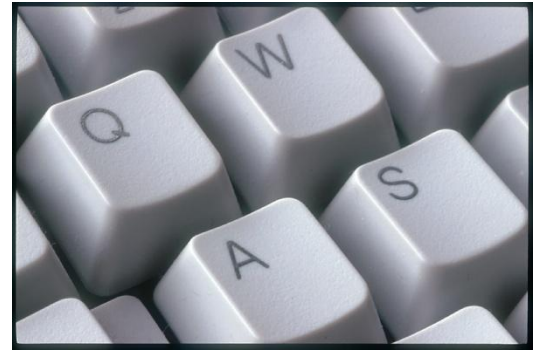
Boolean - Latch



False Constant



Mechanical Action of Booleans



Mechanical Action of Booleans

Use the **Properties»Operations** tab of a Boolean control to learn about the different switch and latch actions.

DEMONSTRATION

Strings

- A string is a sequence of ASCII characters.
- Strings have various display styles.
 - Backslash codes
 - Password
 - Hex



String Control



Empty String Constant



String Indicator



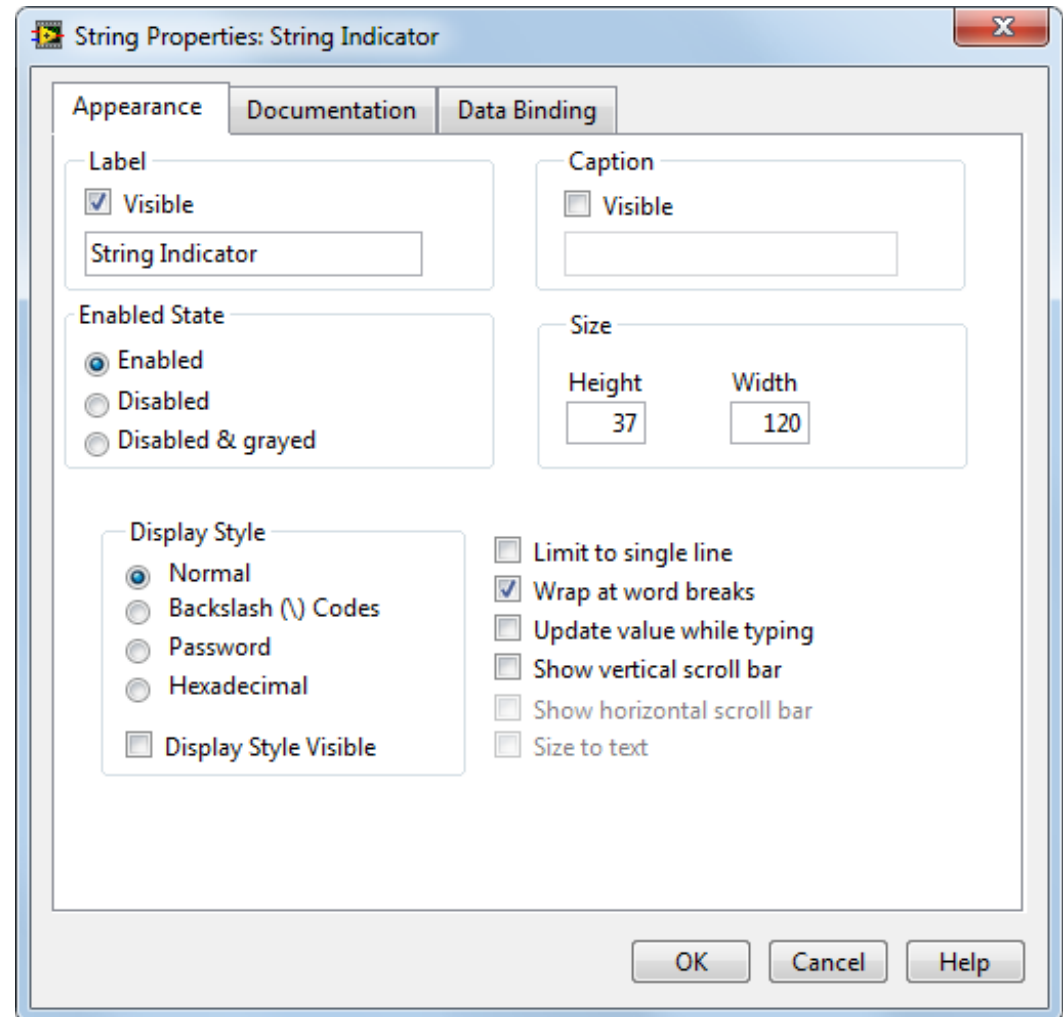
String Constant



End of Line Constant

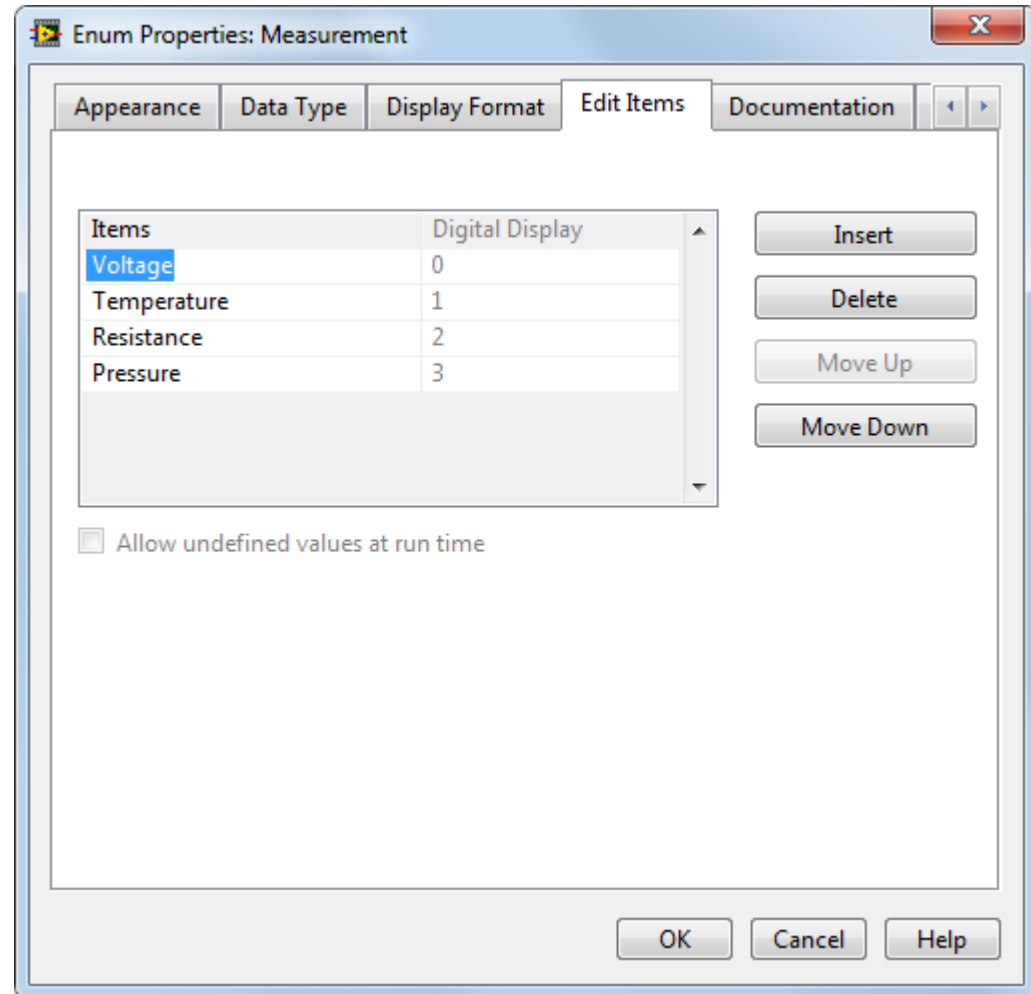
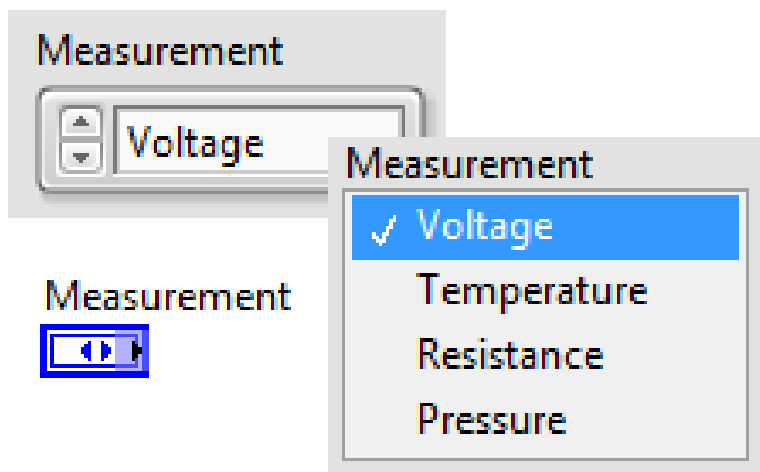


Tab Constant



Enums

- Enums give users a list of items from which to select.
- Each item represents a pair of values.
 - String
 - 16-bit Integer



Other Data Types

Refer to *LabVIEW Help* for complete list of terminal symbols for different types of controls and indicators.

- Dynamic 
 - Stores the information generated or acquired by an Express VI.
- Path 
 - Stores the location of a file or directory using the standard syntax for the platform you are using.
- Waveform 
 - Carries the data, start time, and dt of a waveform.

Searching Help for Data Types

Use *LabVIEW Help* to learn about LabVIEW data types.

DEMONSTRATION

C. Documenting Code

VI Descriptions

Tip Strips

Labels

Free

Owned

Documenting Code

VI

- Name
- Description

Front Panel

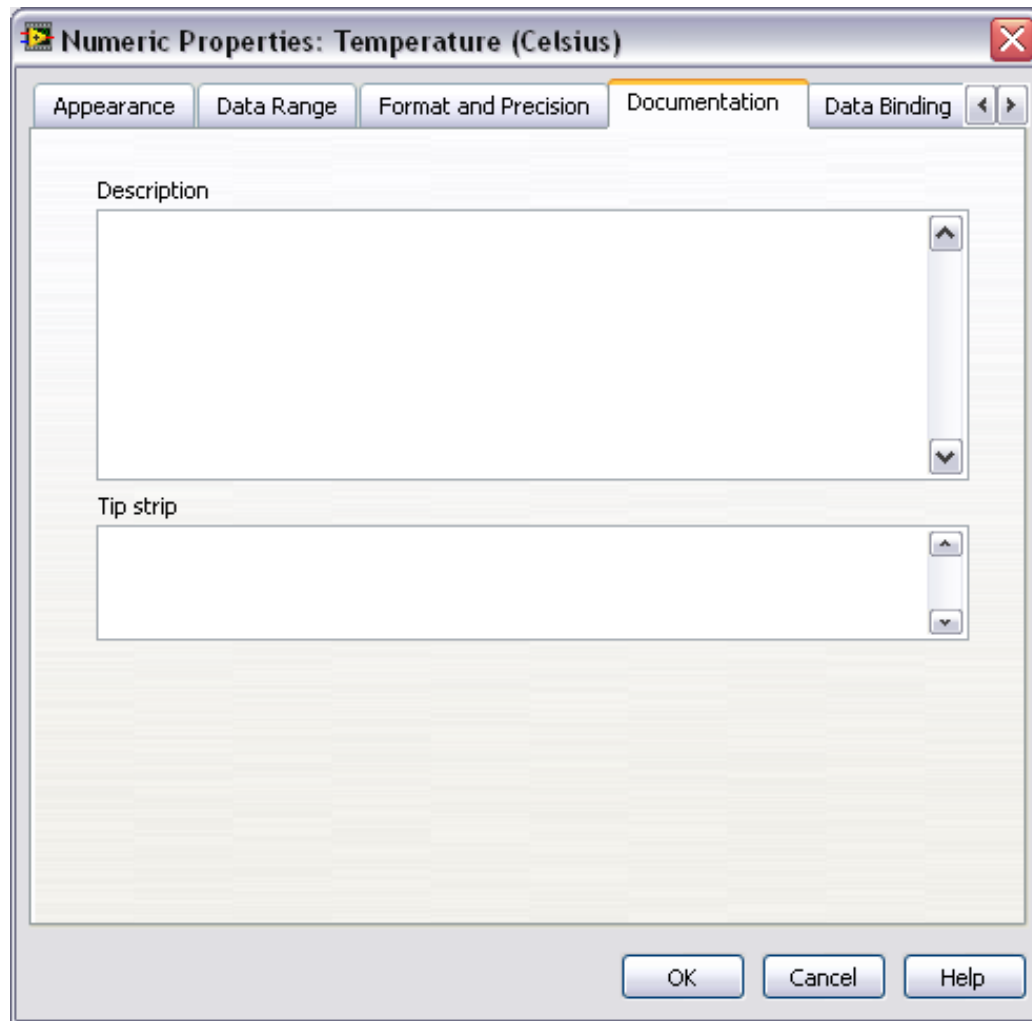
- Label Names
- Tip Strips
- Descriptions
- Free Labels

Block Diagram

- Label Names
- Free Labels
- Owned Labels
- SubVI Descriptions

Creating Descriptions and Tip Strips

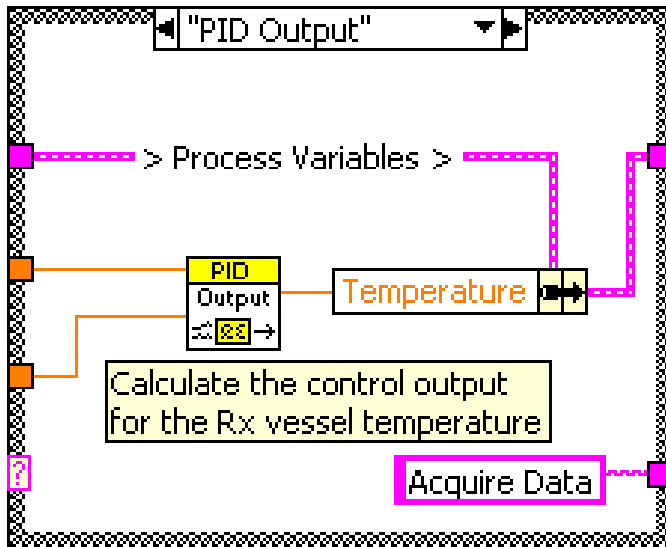
Use the **Properties** dialog box to create documentation for an object.



Documenting Block Diagram Code

Free labels:

- Describe algorithms.
- Have pale yellow backgrounds.
- Double-click in any open space to create.



Owned labels:

- Explain data contents of wires and objects.
- Move with object.
- Have transparent backgrounds.
- Select **Visible Items»Label** from the shortcut menu to create.

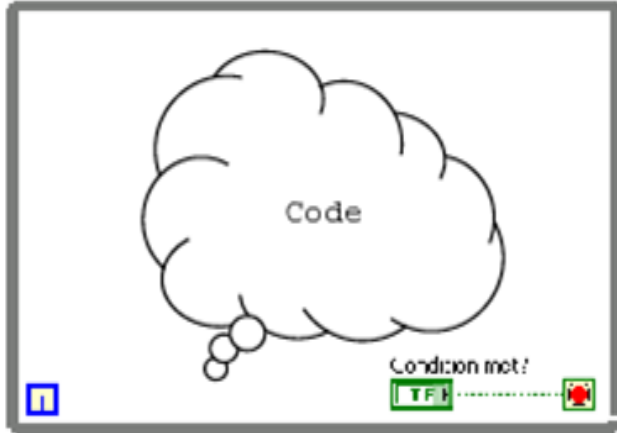
D. While Loops

Iteration and Conditional Terminals

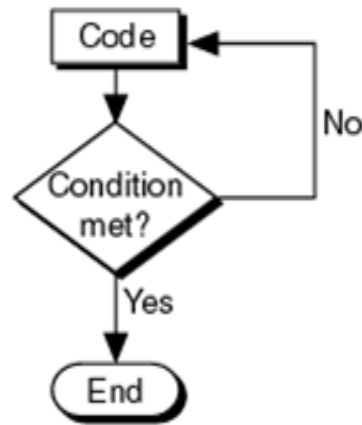
Tunnels

Error Checking

While Loops



LabVIEW While Loop



Flowchart

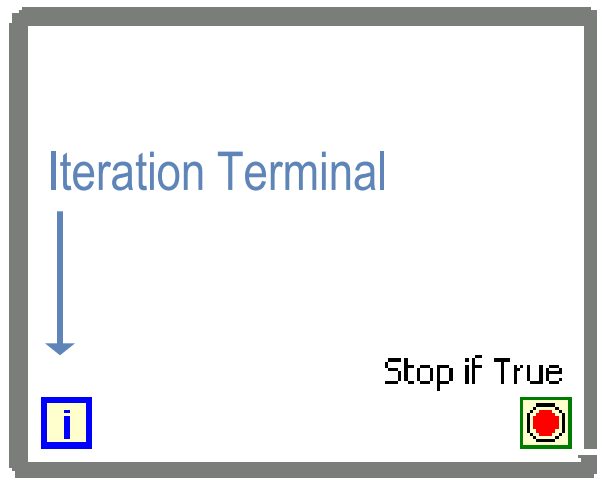
Repeat (code);
Until Condition met;
End;

Pseudo Code

While Loops

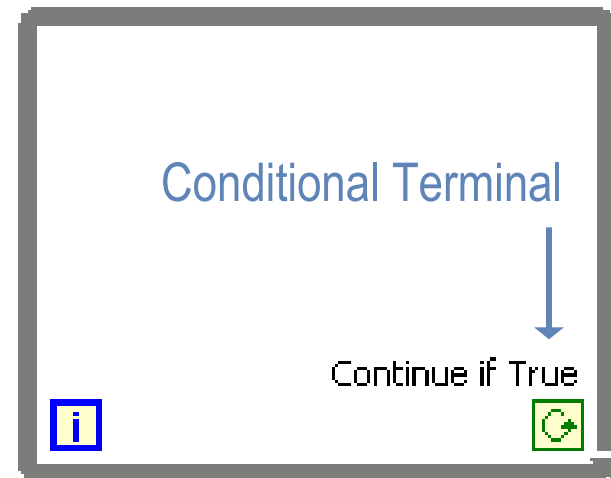
Iteration terminal

- Returns number of times loop has executed.
- Is zero-indexed.



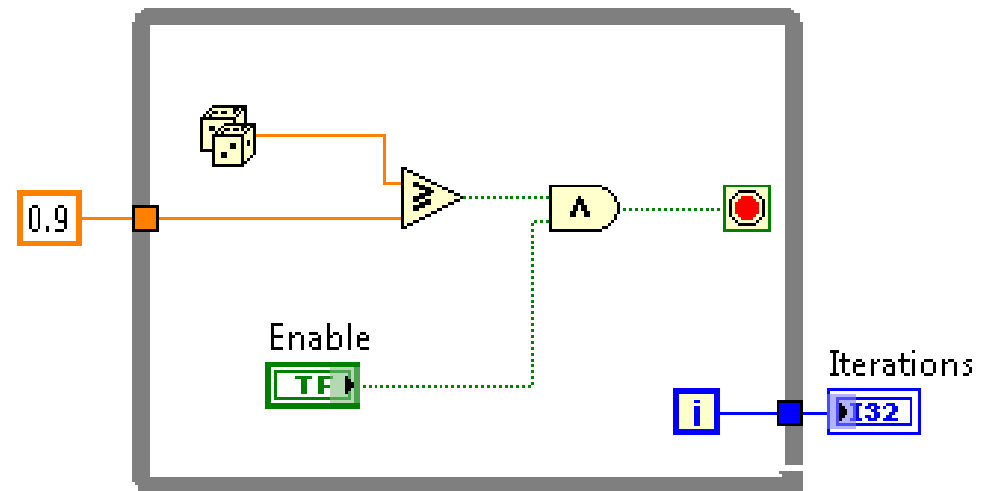
Conditional terminal

- Defines when the loop stops.
- Has two options.
 - Stop if True
 - Continue if True



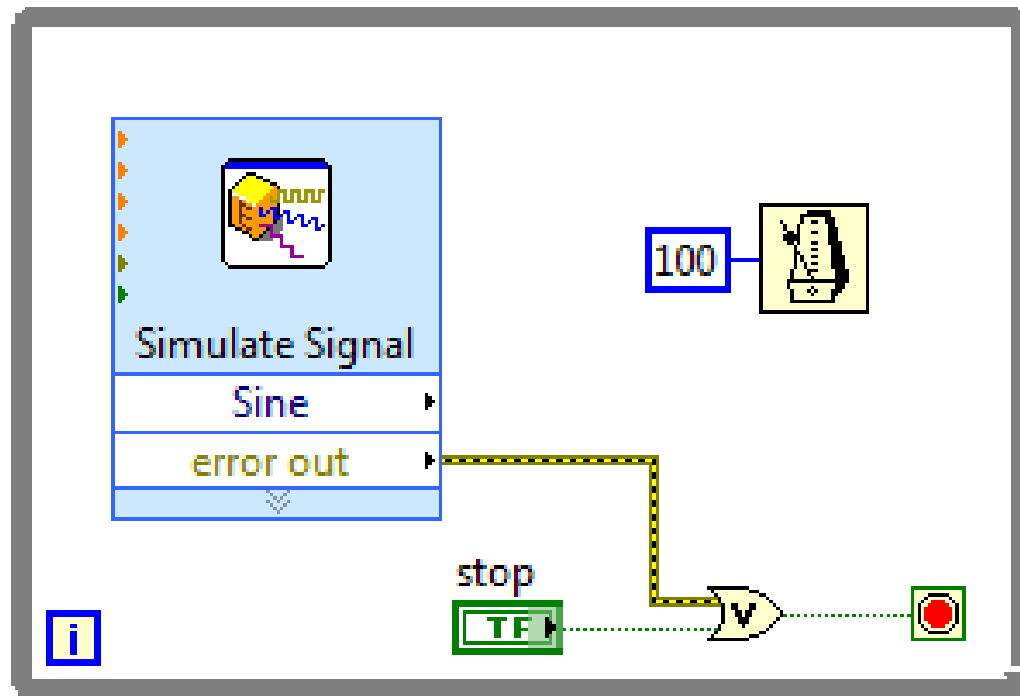
While Loops – Tunnels

- Tunnels transfer data into and out of structures.
- Data pass out of a loop after the loop terminates.
- When a tunnel passes data into a loop, the loop executes only after data arrive at the tunnel.



While Loops – Error Checking and Error Handling

Use an error cluster in a While Loop to stop the While Loop if an error occurs.



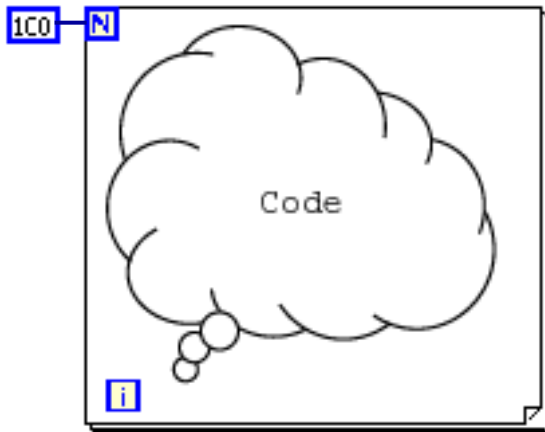
E. For Loops

Conditional Terminal

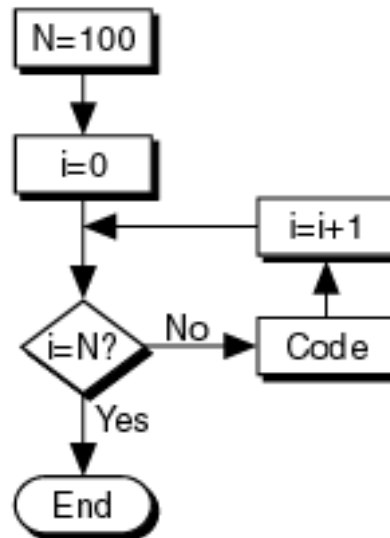
Comparison with While Loops

Numeric Conversion for Count Terminal

For Loops



LabVIEW For Loop



Flowchart

```
N=100;  
i=0;  
Until i=N:  
    Repeat (code;i=i+1);  
End;
```

Pseudo Code

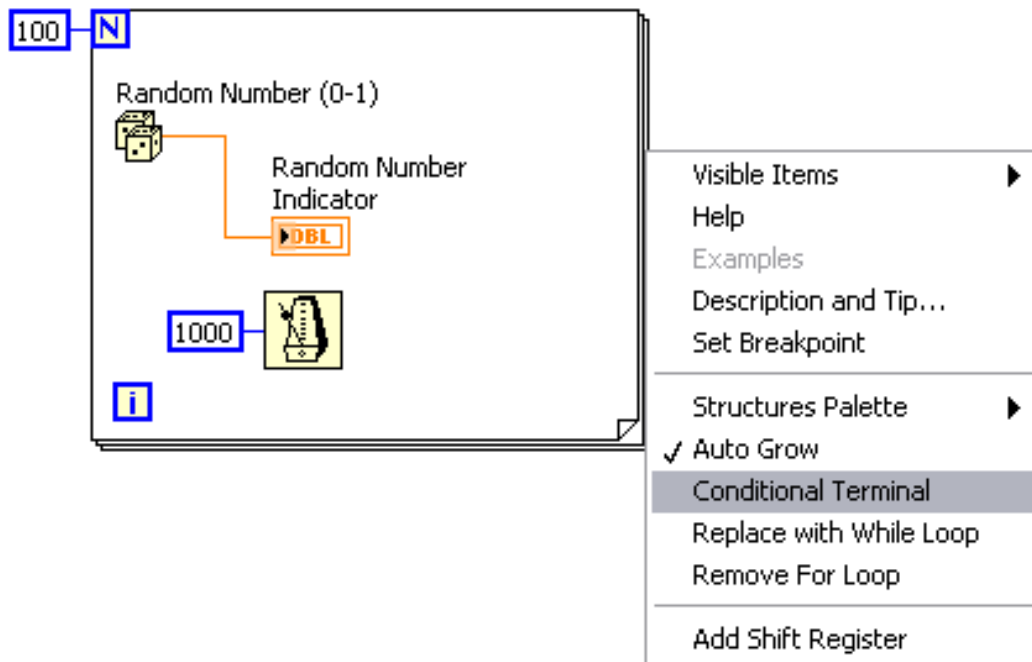
For Loops

- Create a For Loop the same way you create a While Loop.
- You can replace a While Loop with a For Loop by right-clicking the border of the While Loop and selecting **Replace with For Loop** from the shortcut menu.
- The value in the count terminal (an input terminal) indicates how many times to repeat the subdiagram in the For Loop.



For Loops – Conditional Terminal

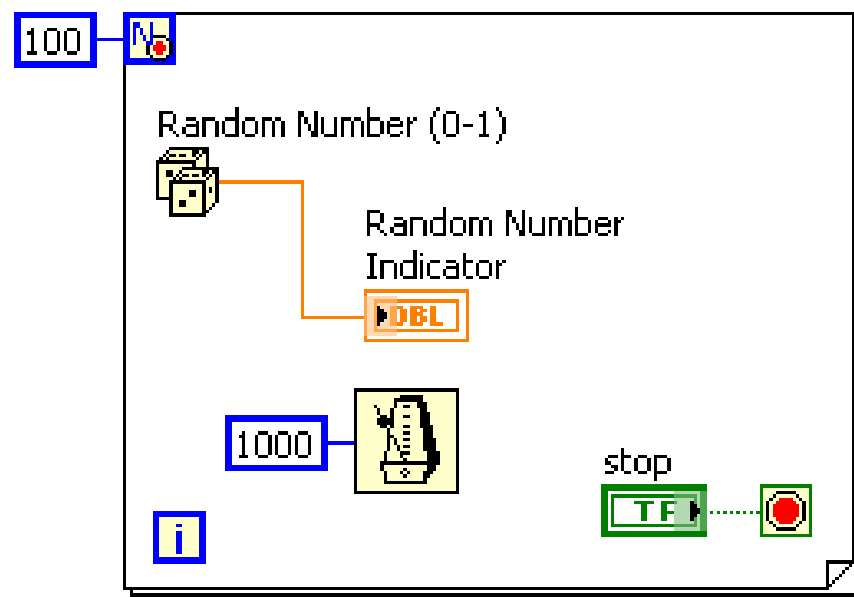
You can add a conditional terminal to configure a For Loop to stop when a Boolean condition is true or an error occurs.



For Loops – Conditional Terminal

For Loops configured with a conditional terminal have:

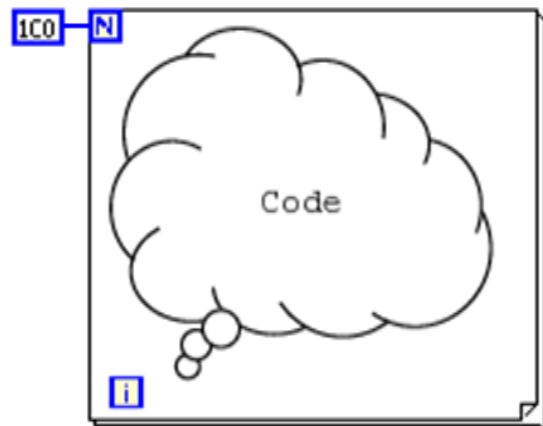
- A red glyph next to the count terminal.
- A conditional terminal in the lower right corner



For Loop/While Loop Comparison

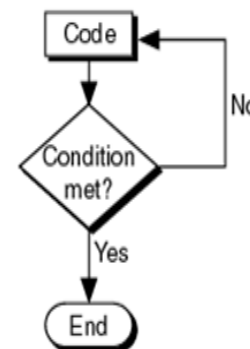
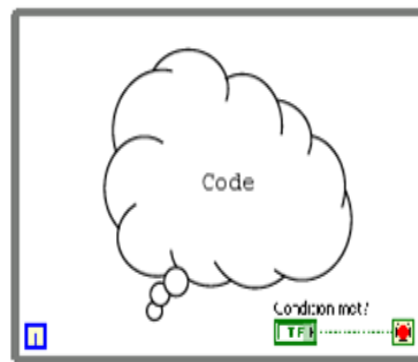
For Loop

- Executes a set number of times unless a conditional terminal is added.
- Can execute zero times.
- Tunnels automatically output an array of data.



While Loop

- Stops executing only if the value at the conditional terminal meets the condition.
- Must execute at least once.
- Tunnels automatically output the last value.



For Loops – Numeric Conversion

- The number of iterations a For Loop executes must be specified in non-negative integers.
- If you wire a double-precision, floating-point numeric value to the count terminal, LabVIEW converts the numeric value to a 32-bit signed integer.

Double-Precision
Floating Point



32-Bit Signed Integer



Group Exercise

Concept: While Loops versus For Loops

Understand when to use a While Loop and when to use a For Loop.

GOAL

F. Timing a VI

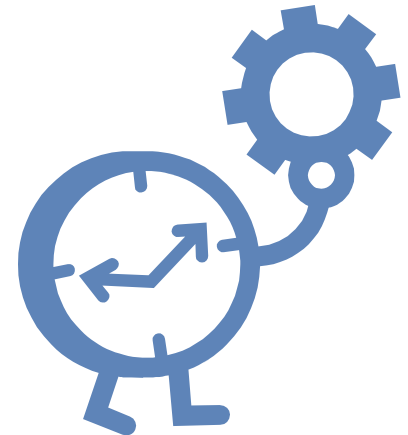
Reasons To Use Timing

Wait Functions and Express VIs

Timing a VI

Why do you need timing in a VI?

- To control the frequency at which a loop executes.
- To provide the processor with time to complete other tasks, such as processing the user interface.

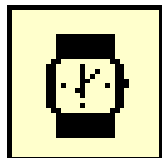


Wait Functions

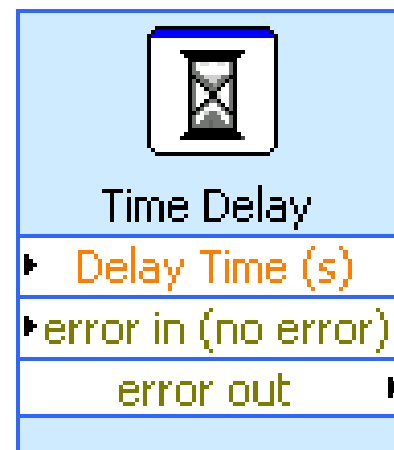
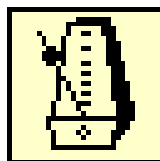
A wait function inside a loop:

- Allows the VI to sleep for a set amount of time.
- Allows the processor to address other tasks during the wait time.
- Uses the operating system millisecond clock.

Wait (ms)



Wait Until
Next ms Multiple



Elapsed Time Express VI

- Determines how much time elapses after some point in your VI.
- Keeps track of time while the VI continues to execute.
- Does not provide the processor with time to complete other tasks.



G. Data Feedback in Loops

Shift Registers

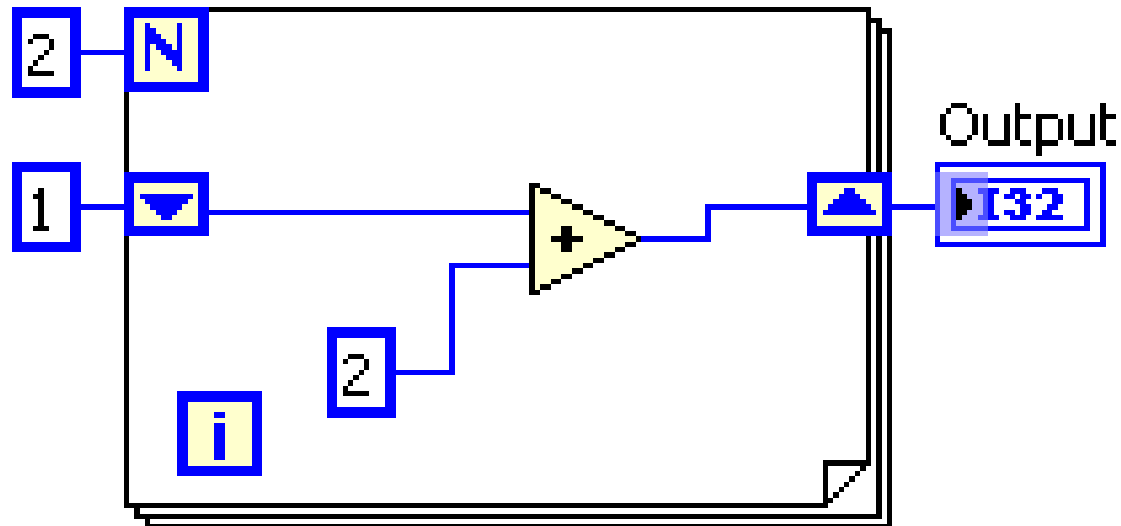
Initializing Shift Registers

Default for Unwired Values

Compound Shift Registers

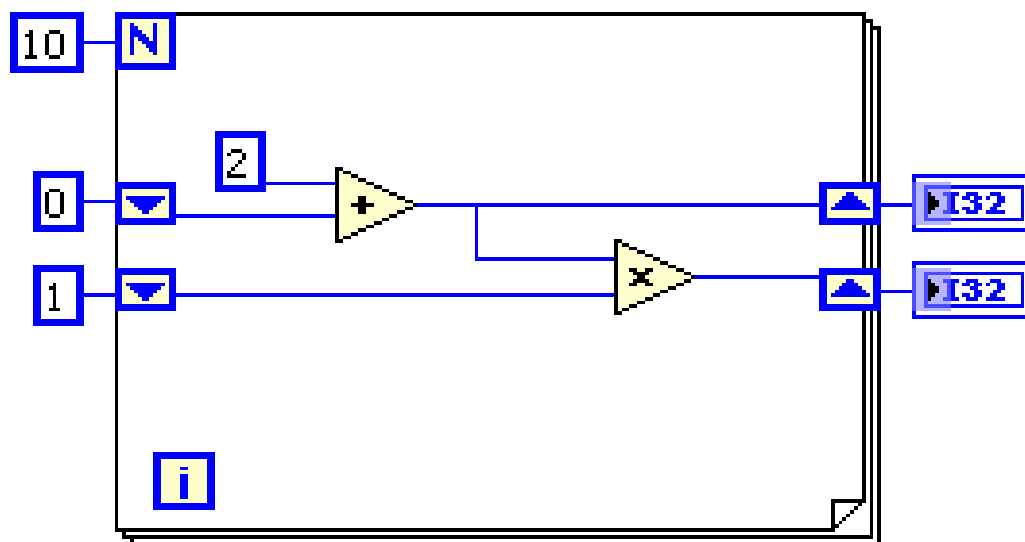
Data Feedback in Loops

- When programming with loops, you often need to know the values of data from previous iterations of the loop.
- Shift registers transfer values from one loop iteration to the next.



Shift Registers

- Right-click the border and select **Add Shift Register** from the shortcut menu.
- Right shift register stores data on completion of an iteration.
- Left shift register provides stored data at beginning of the next iteration.

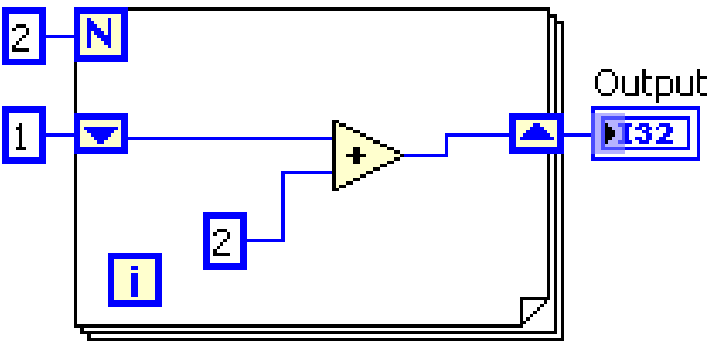
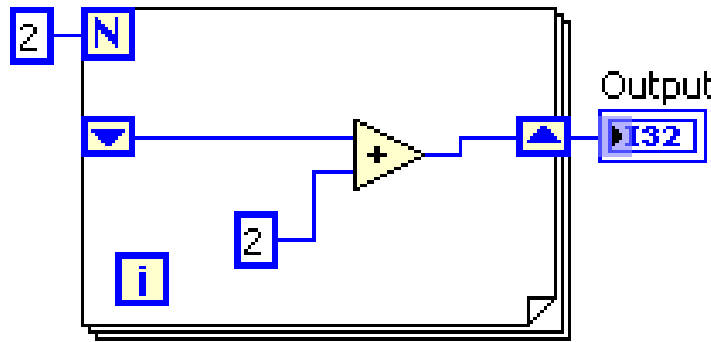


Initializing Shift Registers

Run once

VI finishes

Run again

Block Diagram	1st run	2nd run
Initialized Shift Register 	Output = 5	Output = 5
Not Initialized Shift Register 	Output = 4	Output = 8

Use Default if Unwired

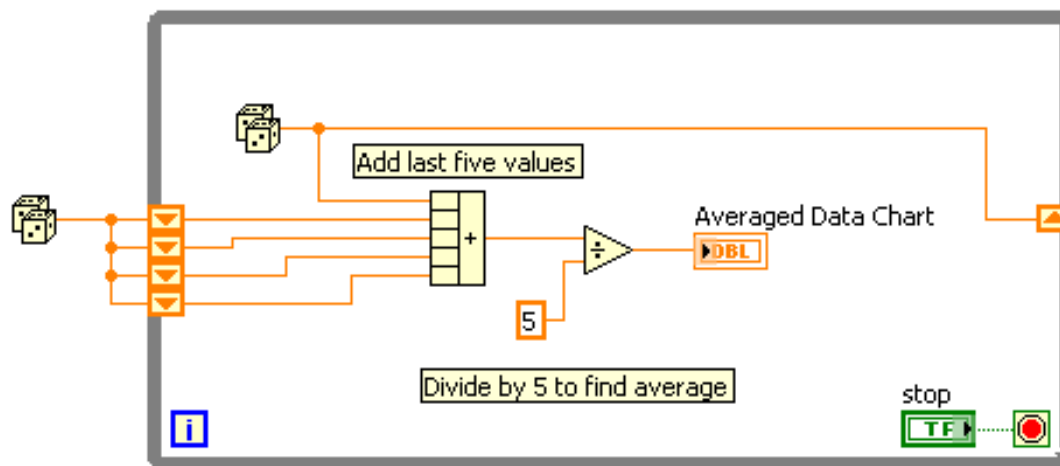
Default values vary by data type:

Data Type	Default Value
Numeric	0
Boolean	FALSE
String	Empty

Uninitialized shift registers use default values for first run.

Multiple Previous Iterations

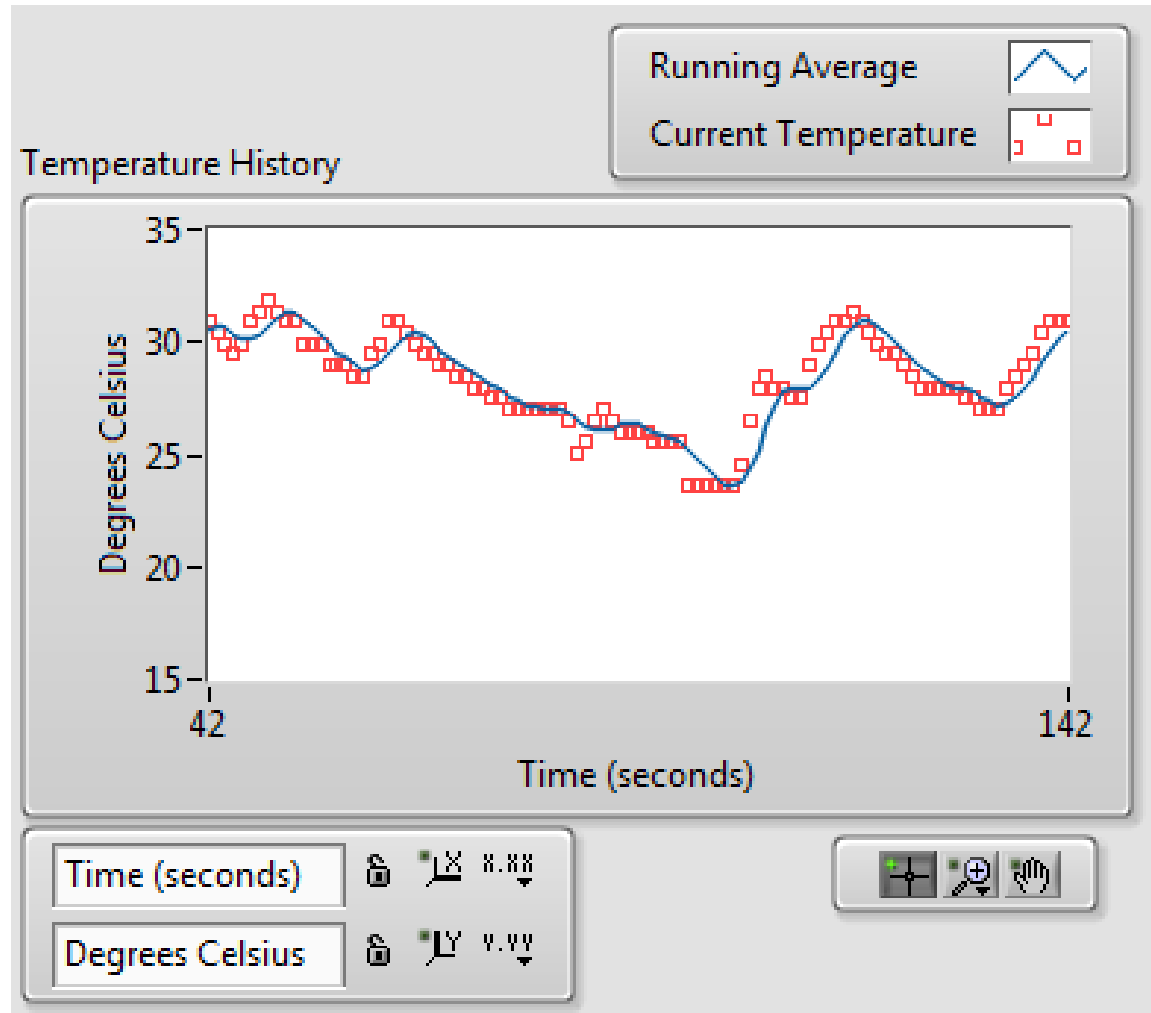
- Stacked shift registers remember values from multiple previous iterations and carry those values to the next iterations.
- Right-click the left shift register and select **Add Element** from the shortcut menu to stack a shift register.



H. Plotting Data – Waveform Chart

Plotting Data – Waveform Chart

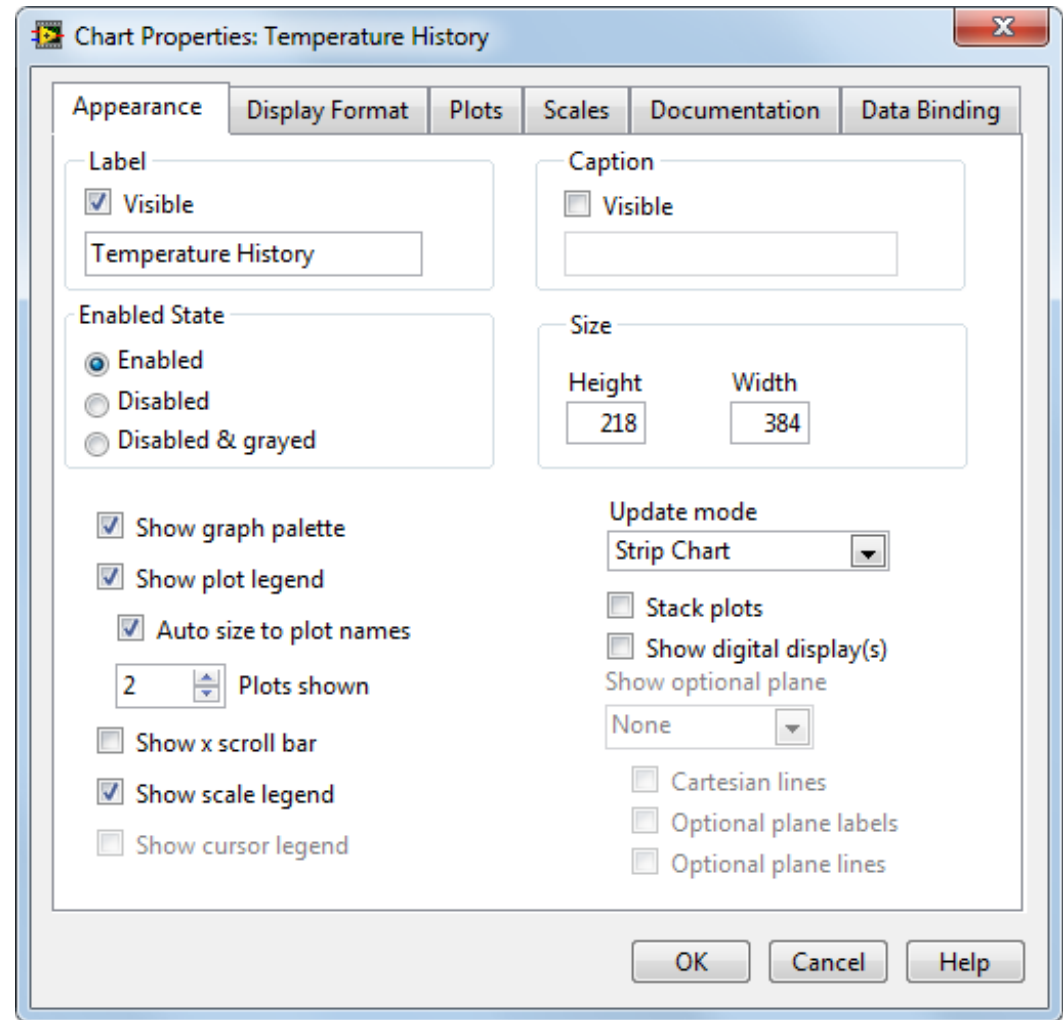
- Waveform chart is a special type of numeric indicator.
- Waveform charts display single or multiple plots.



Waveform Chart Properties

Extensive plot customization lets you:

- Show or hide legends.
- Change color and line styles.
- Change interpolation styles.



I. Case Structures

Parts of a Case Structure

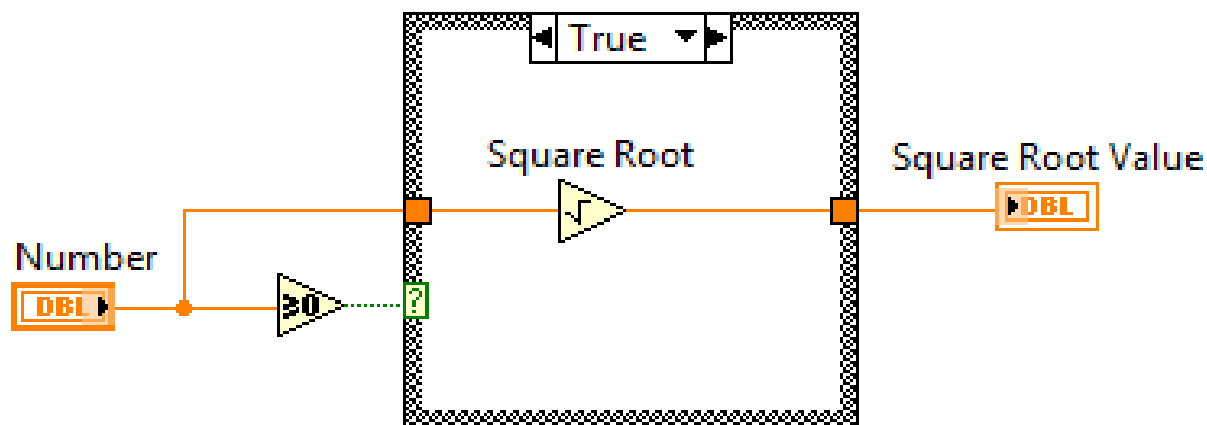
Enum Case Structures

Error Case Structures

Input and Output Tunnels

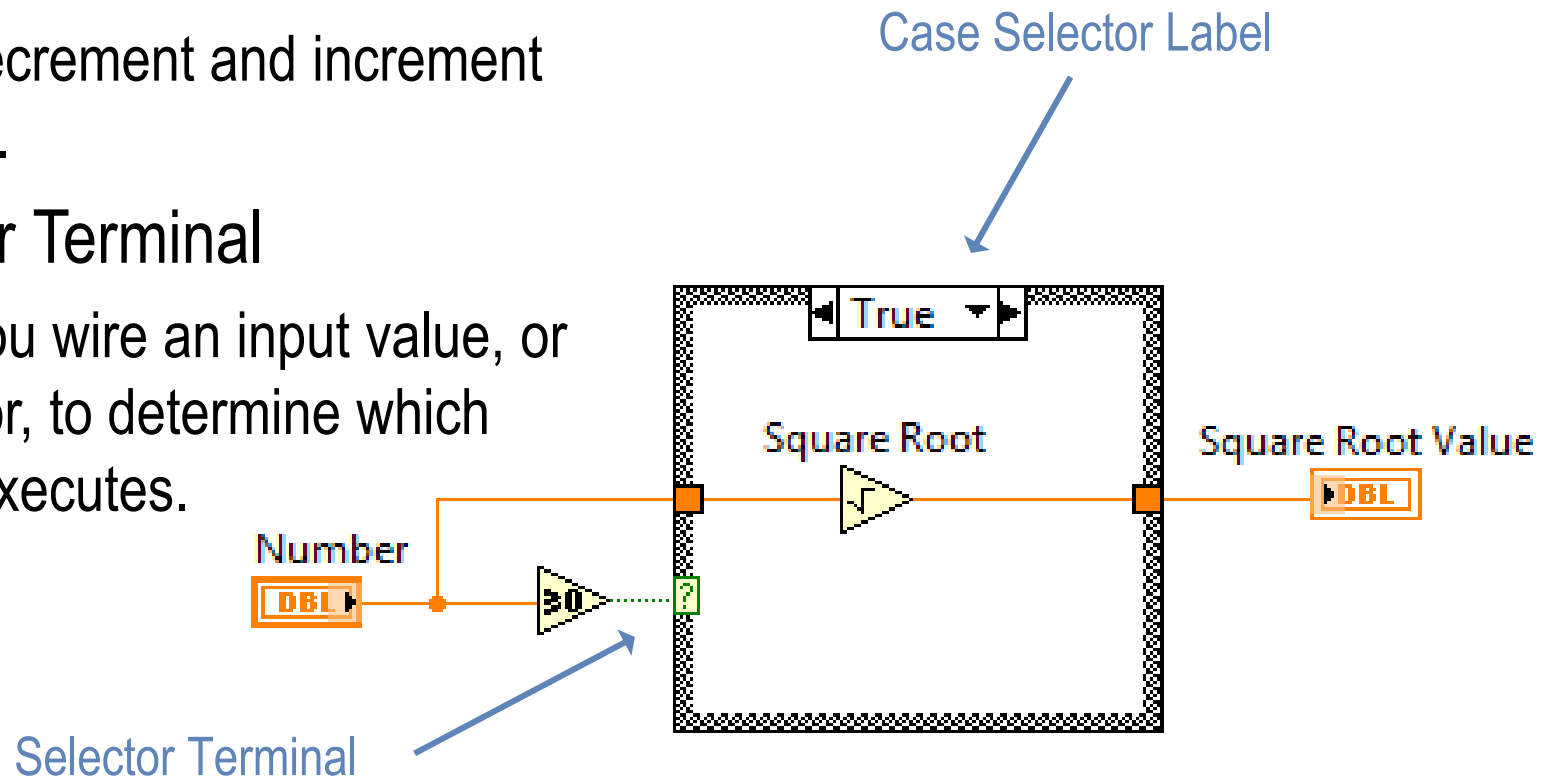
Case Structures

- Have two or more subdiagrams or cases.
- Use an input value to determine which case to execute.
- Execute and display only one case at a time.
- Are similar to **case** statements or **if...then...else** statements in text-based programming languages.



Case Structures

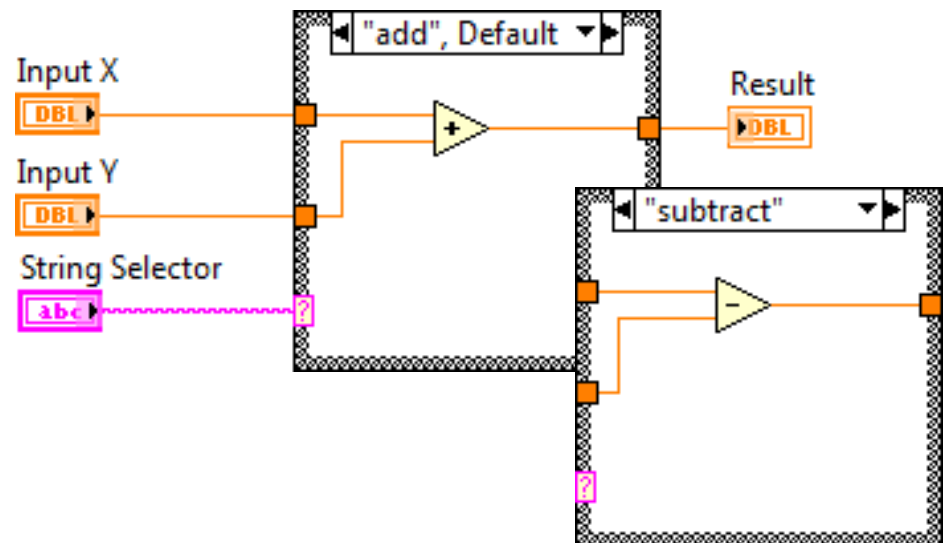
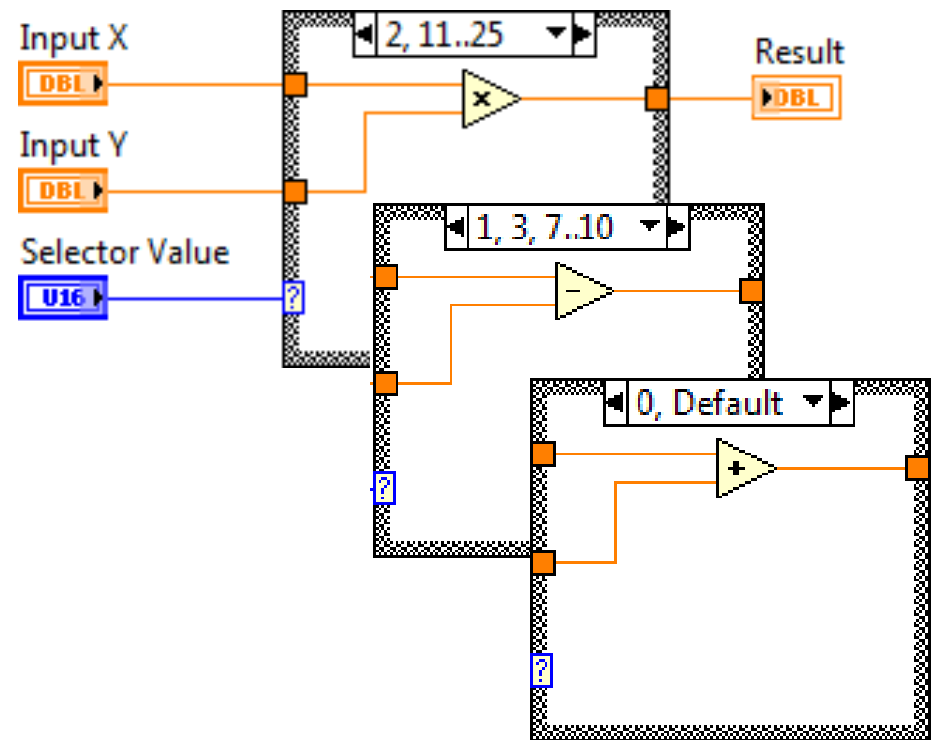
- Case Selector Label
 - Contains the name of the current case.
 - Has decrement and increment arrows.
- Selector Terminal
 - Lets you wire an input value, or selector, to determine which case executes.



Case Structures

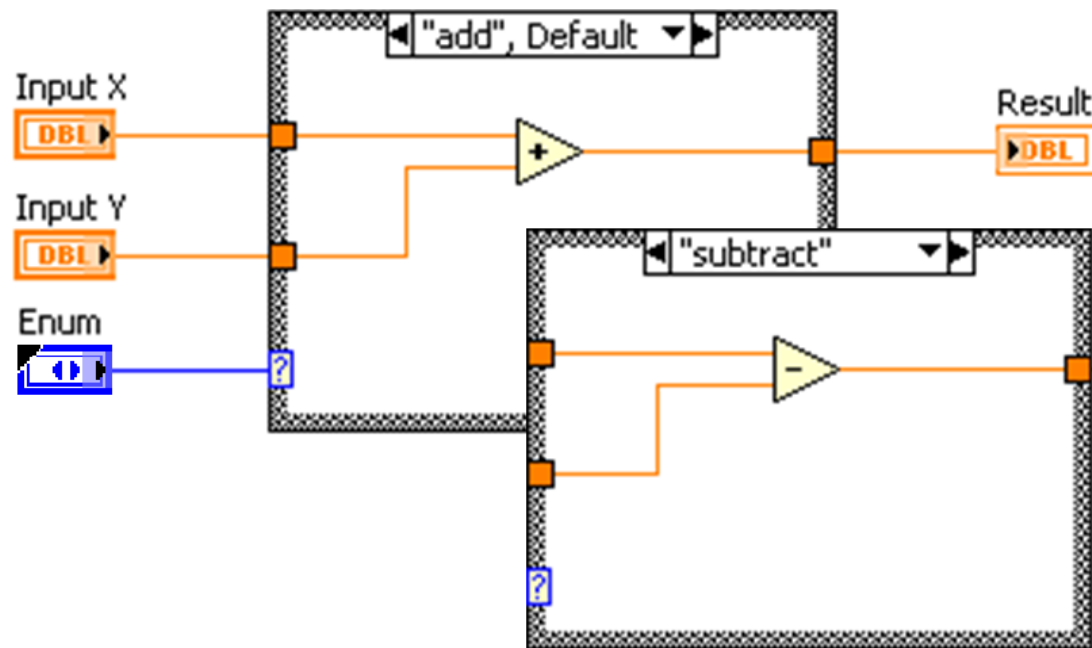
Selector terminal data types:

- Boolean
 - True case and False Case
- Error Cluster
 - Error Case and No Error Case
- Integer, string, or enum
 - Structure can have any number of cases.
 - Include a Default diagram to avoid listing every possible input value.



Enum Case Structure

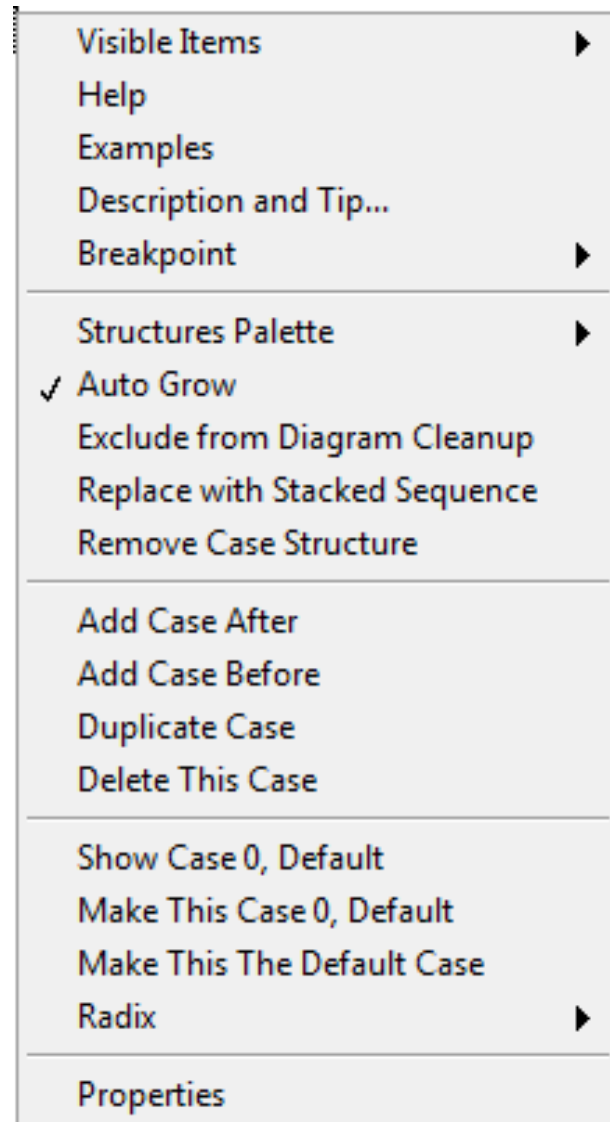
- Gives users a list of items from which to select
- The case selector displays a case for each item in the enumerated type control



Shortcut Menu

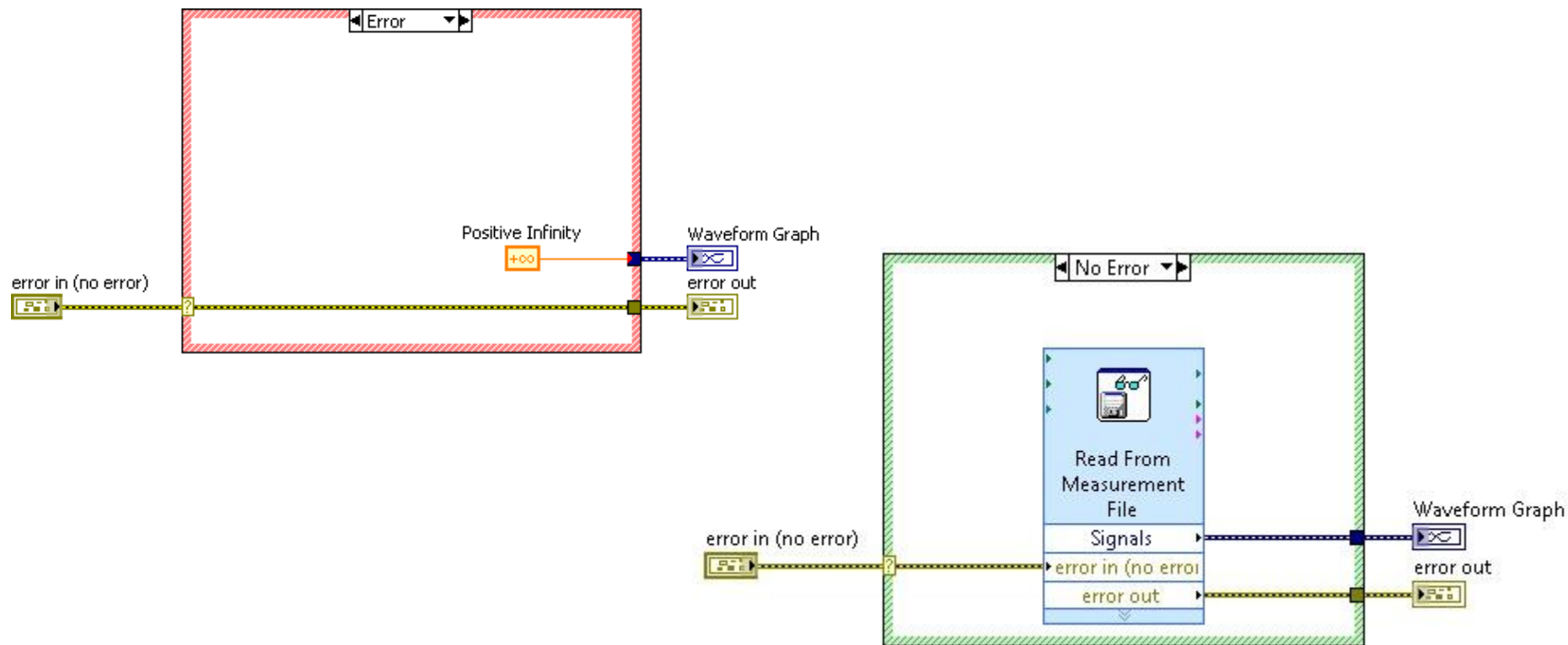
Use the shortcut menu of a Case structure to:

- Customize the structure and diagrams.
- Remove or replace the structure.
- Add, duplicate, remove, or rearrange cases.
- Specify the Default case.
- Switch cases.



Error Case Structure

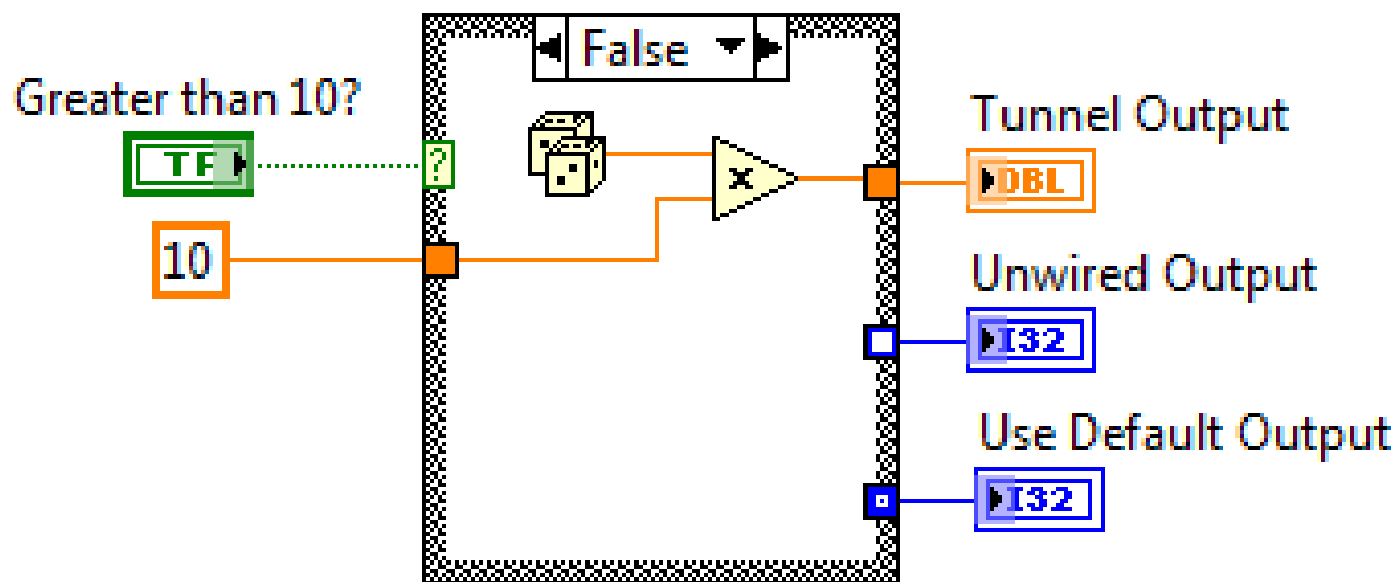
Use Case structures inside VIs to execute the code if there is no error and skip the code if there is an error.



Input and Output Tunnels

You can create multiple input and output tunnels.

- Inputs tunnels are available to all cases if needed.
- You must define each output tunnel for each case.

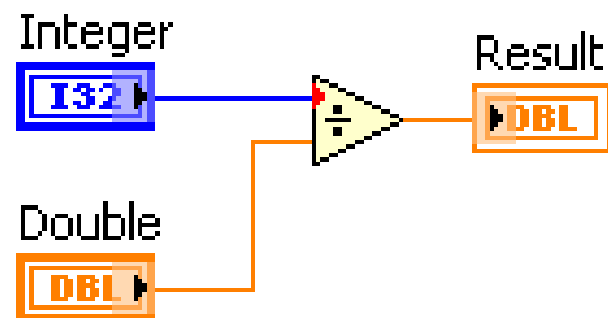


Summary—Quiz

1. If an input to a function is marked with a red dot (known as a coercion dot), what does the dot indicate?
 - a) Data was transferred into a structure.
 - b) A For Loop was configured with a conditional terminal.
 - c) A For Loop iteration terminal is unwired.
 - d) The value passed into a node was converted to a different representation.

Summary—Quiz Answer

1. If an input to a function is marked with a red dot (known as a coercion dot), what does the dot indicate?
 - a) Data was transferred into a structure.
 - b) A For Loop was configured with a conditional terminal.
 - c) A For Loop iteration terminal is unwired.
 - d) **The value passed into a node was converted to a different representation.**



Summary—Quiz

2. Which structure must run at least one time?
- a) While Loop
 - b) For Loop

Summary—Quiz Answer

2. Which structure must run at least one time?
- a) **While Loop**
 - b) For Loop

Summary—Quiz

3. Which is only available on the block diagram?
- a) Control
 - b) Constant
 - c) Indicator
 - d) Connector Pane

Summary—Quiz Answer

3. Which is only available on the block diagram?
- a) Control
 - b) Constant**
 - c) Indicator
 - d) Connector Pane

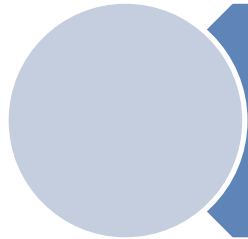
Developing Modular Applications

TOPICS

- A. Understanding Modularity
- B. Icon
- C. Connector Pane
- D. Using SubVIs

A. Understanding Modularity

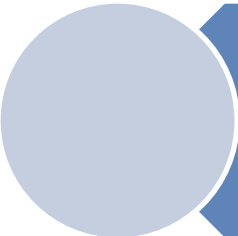
Understanding Modularity



Modularity — The degree to which a program is composed of discrete modules such that a change to one module has minimal impact on other modules

LabVIEW uses subVIs to create modularity.

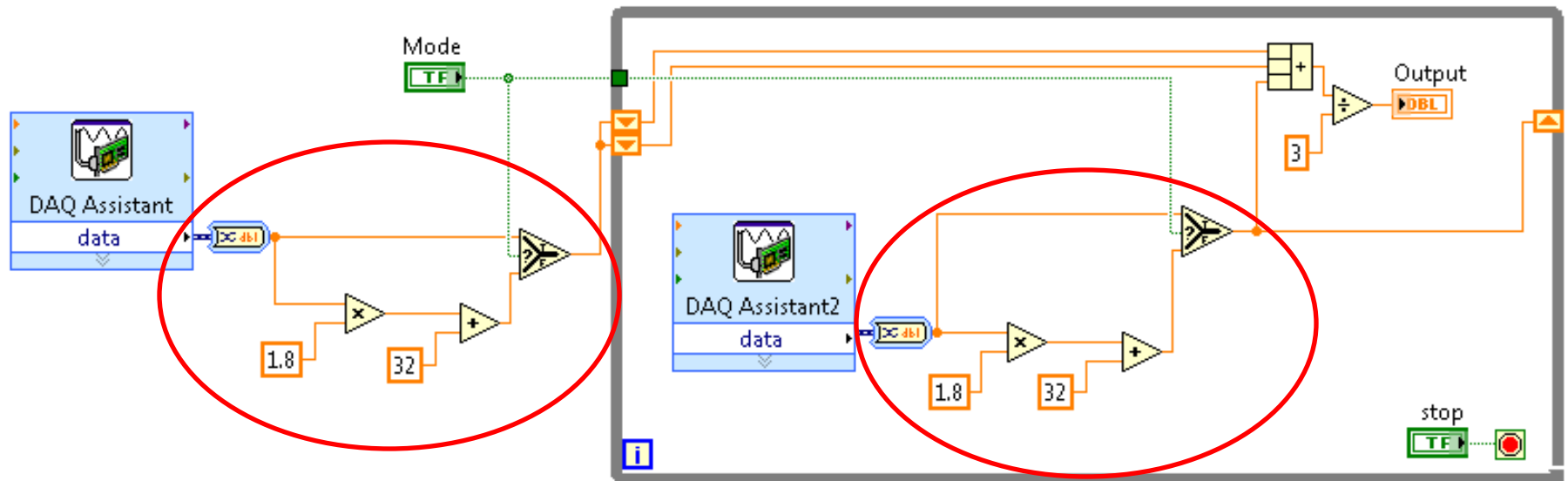
Understanding Modularity – SubVIs



SubVI — A VI within another VI

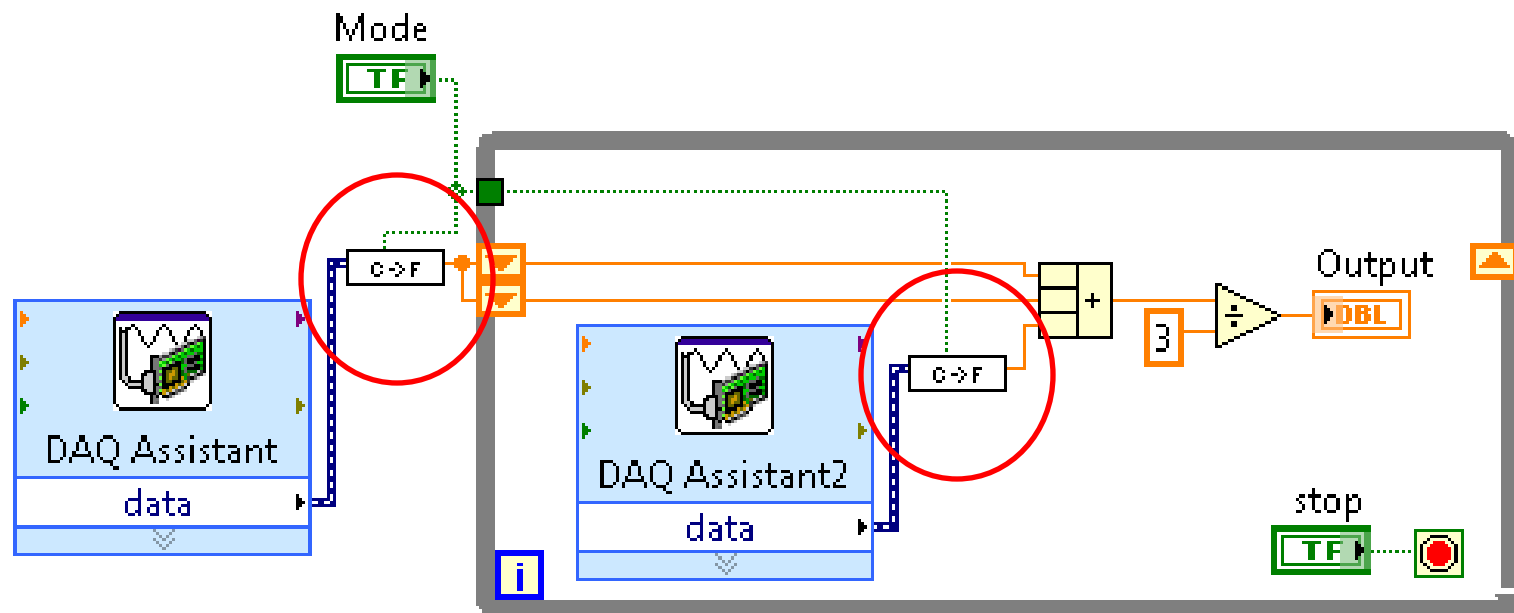
- SubVIs correspond to subroutines in text-based programming languages.
- The upper-right corner of the front panel and block diagram displays the icon for the VI.
- This icon identifies the VI when you place the VI on a block diagram.

Understanding Modularity – SubVIs

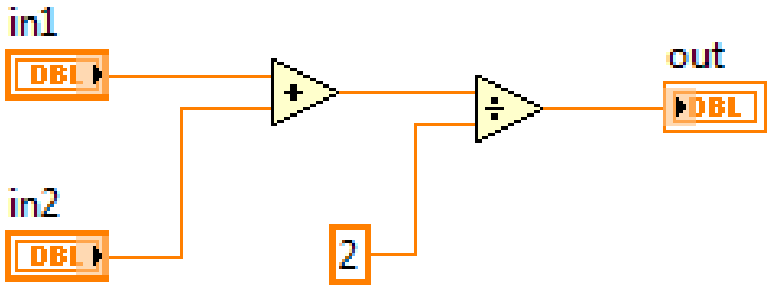
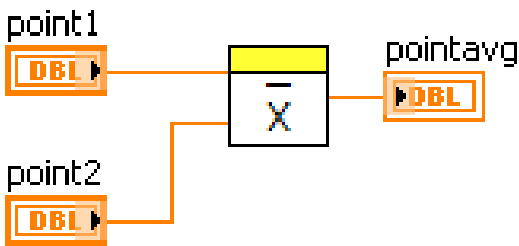


Repeated code can become subVIs.

Understanding Modularity – SubVIs



Understanding Modularity – SubVIs

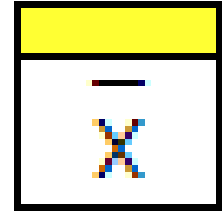
Function Code	Calling Program Code
<pre>function average (in1, in2, out) { out = (in1 + in2)/2.0; }</pre>	<pre>main { average (point1, point2, pointavg) }</pre>
 SubVI Block Diagram	Calling VI Block Diagram
	

B. Icon

Characteristics of a Good Icon

Using the Icon Editor

Icon

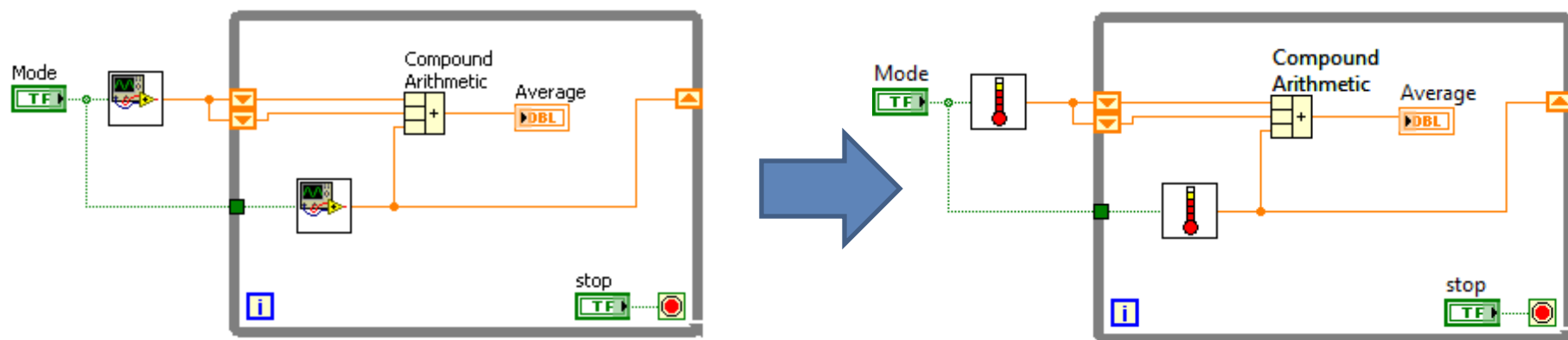


- An icon is a graphical representation of a VI.
- If you use a VI as a subVI, the icon identifies the subVI on the block diagram of the VI.

Characteristics of a Good Icon

Good icons convey the functionality of the VI using:

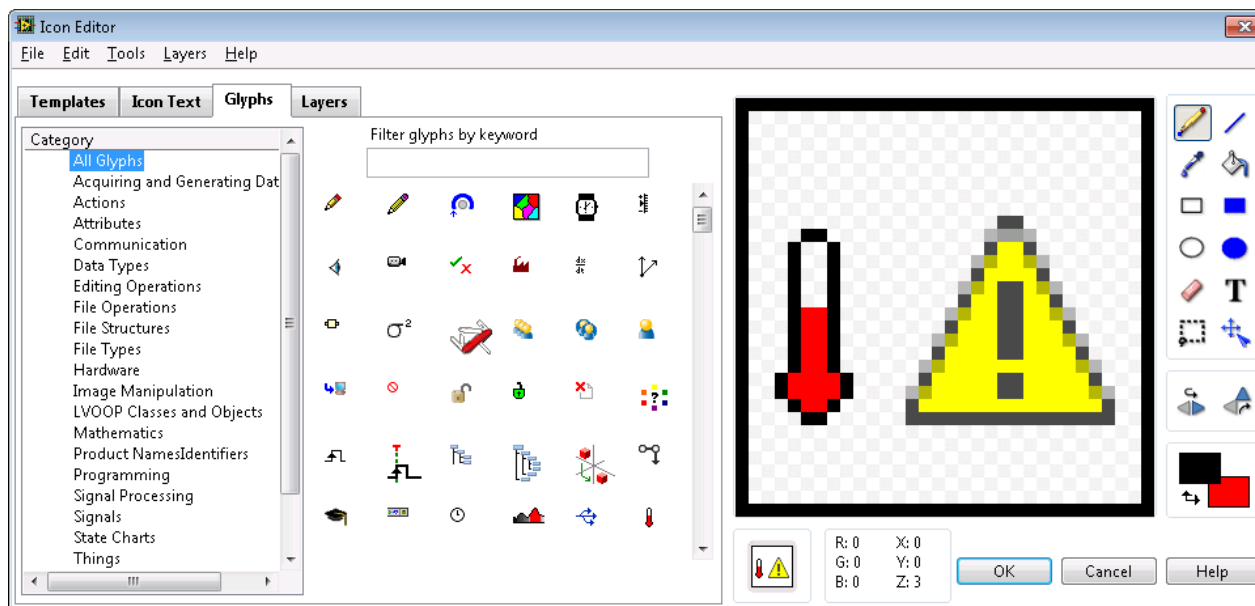
- Relevant graphics
- Descriptive text, if necessary



Creating Icons - Icon Editor

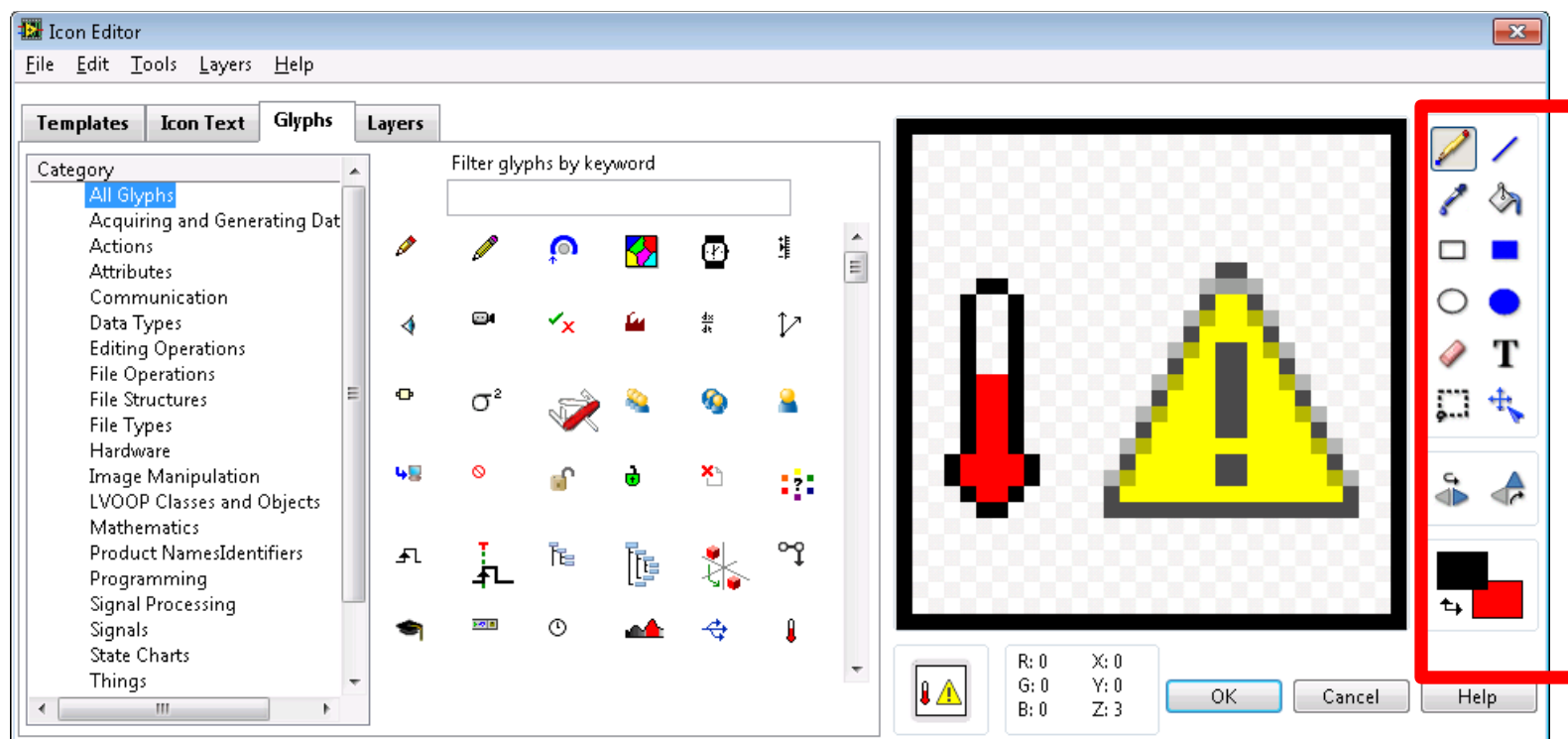
Open the Icon Editor using one of these methods:

- Right-click the icon in the upper-right corner of the front panel or block diagram and select **Edit Icon**.
- Double-click the icon.



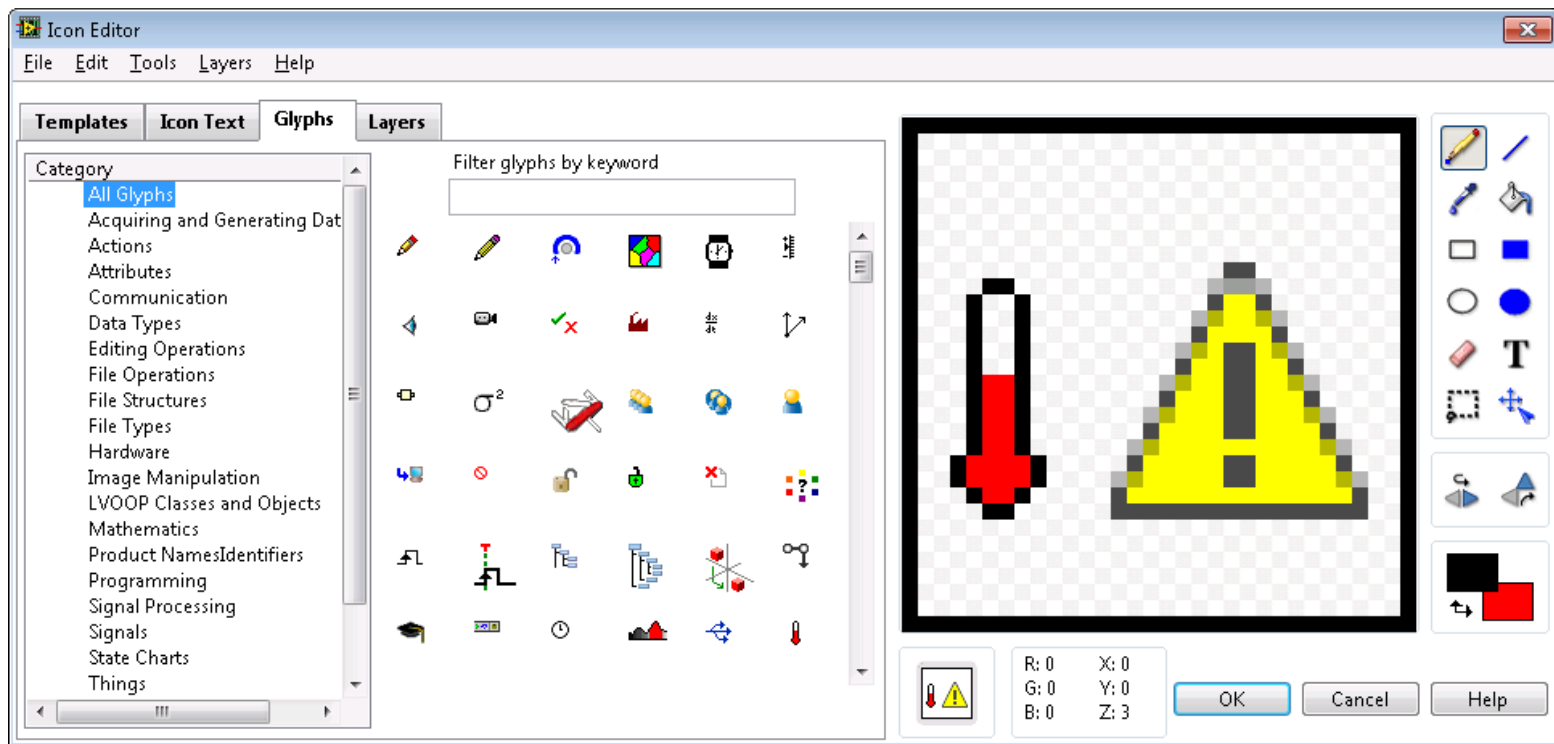
Icon Editor

Use the editing tools to modify an icon manually.



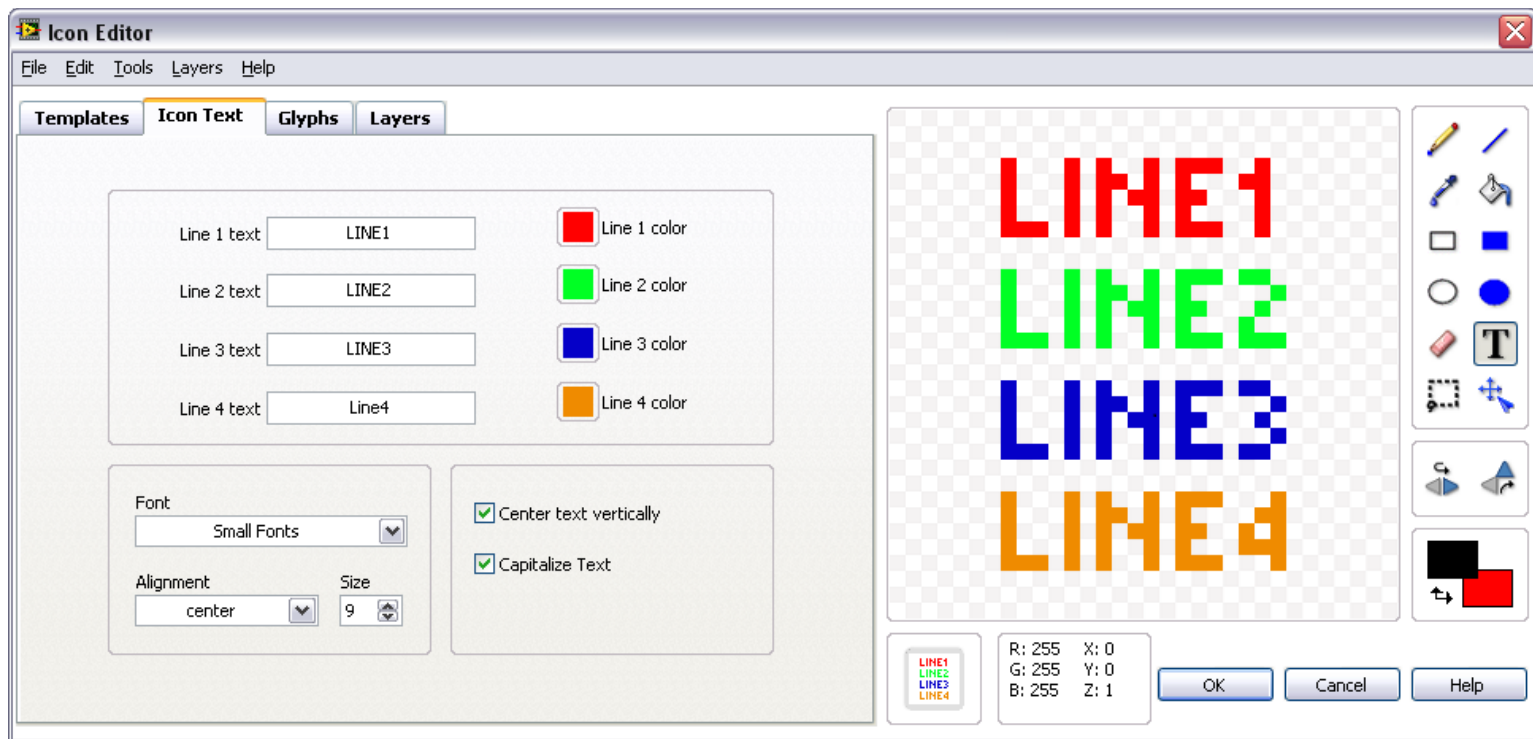
Icon Editor

Use the **Glyphs** tab to display glyphs you can include in the icon.



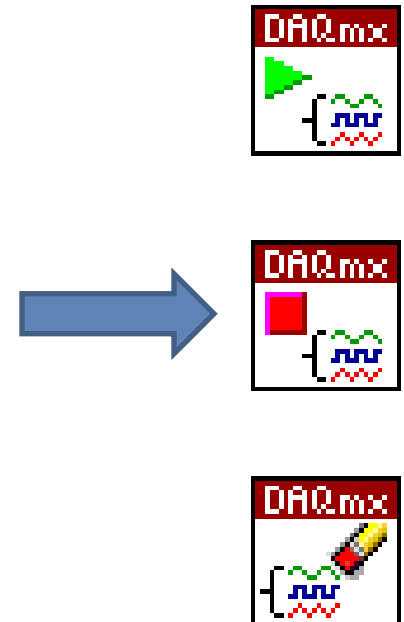
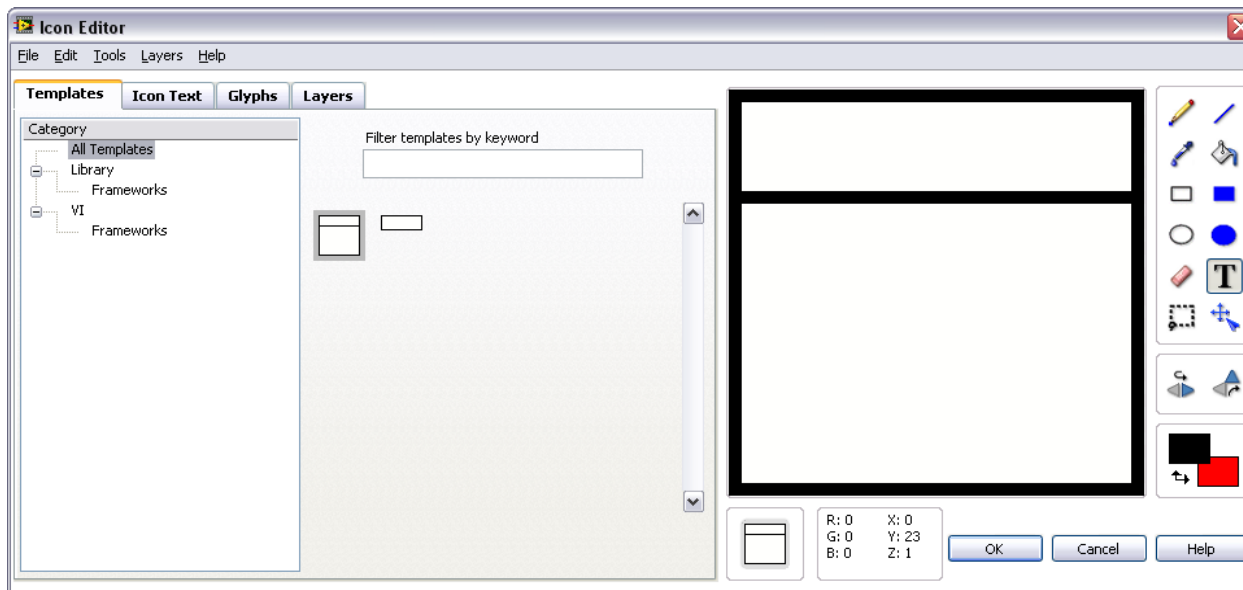
Icon Editor

Use the **Icon Text** tab to specify the text to display in the icon.



Icon Editor

Use the **Templates** tab to display icon templates you can use as a background for the icon.



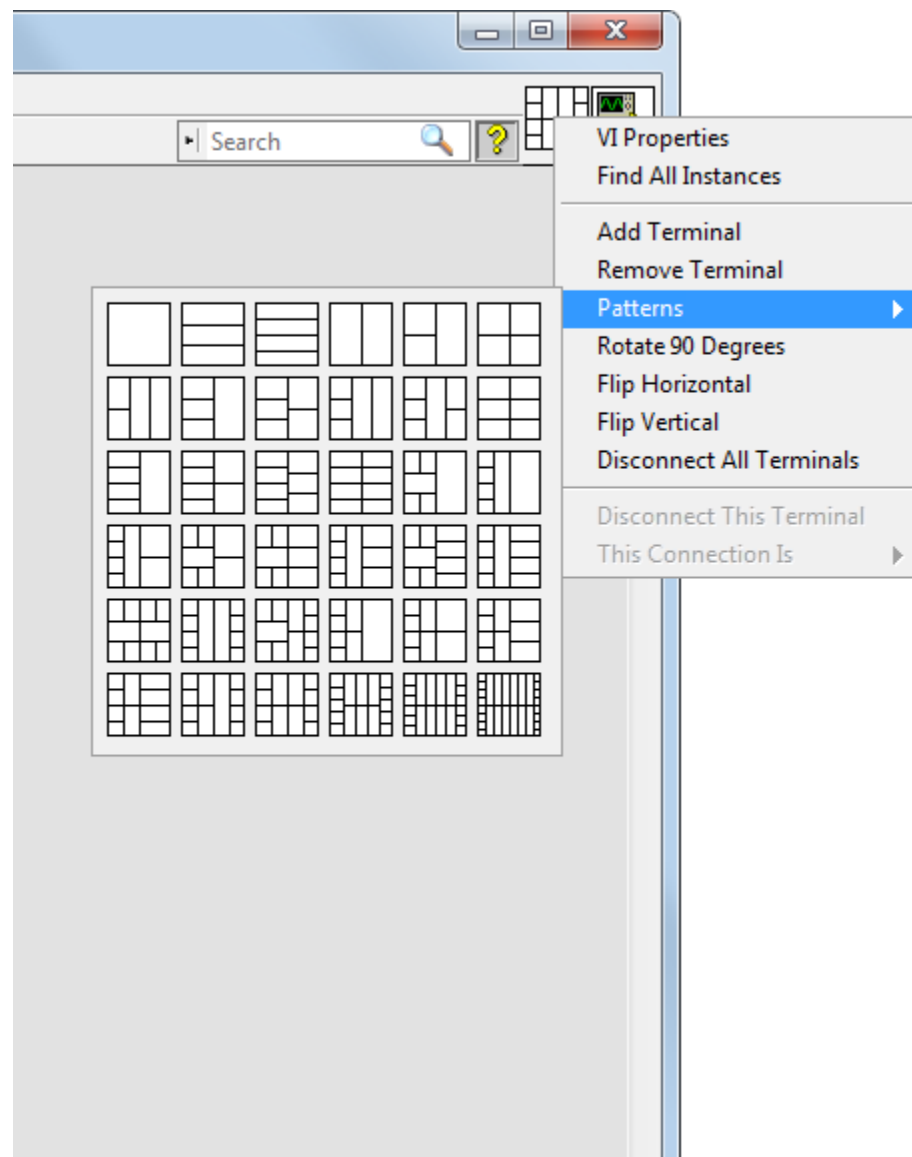
C. Connector Pane

Patterns

Standards

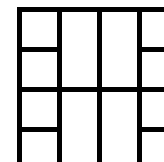
Connector Pane

- The connector pane is displayed next to the icon in the upper right corner of the front panel.
 - Each rectangle on the connector pane represents a terminal.
 - Use the terminals to assign inputs and outputs.
- Select a different pattern by right-clicking the connector pane and selecting **Patterns** from the shortcut menu.

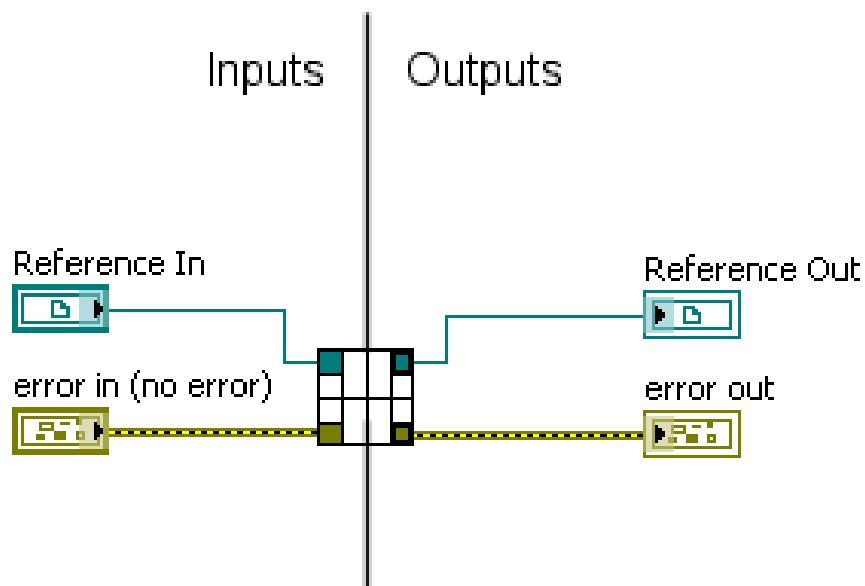


Connector Pane – Standards

- Use this connector pane layout as a standard.



- Top terminals are usually reserved for references, such as a file reference.
- Bottom terminals are usually reserved for error clusters.



D. Using SubVIs

Using on Block Diagram

Terminal Settings

Handling Errors

Creating from a Section of Block Diagram

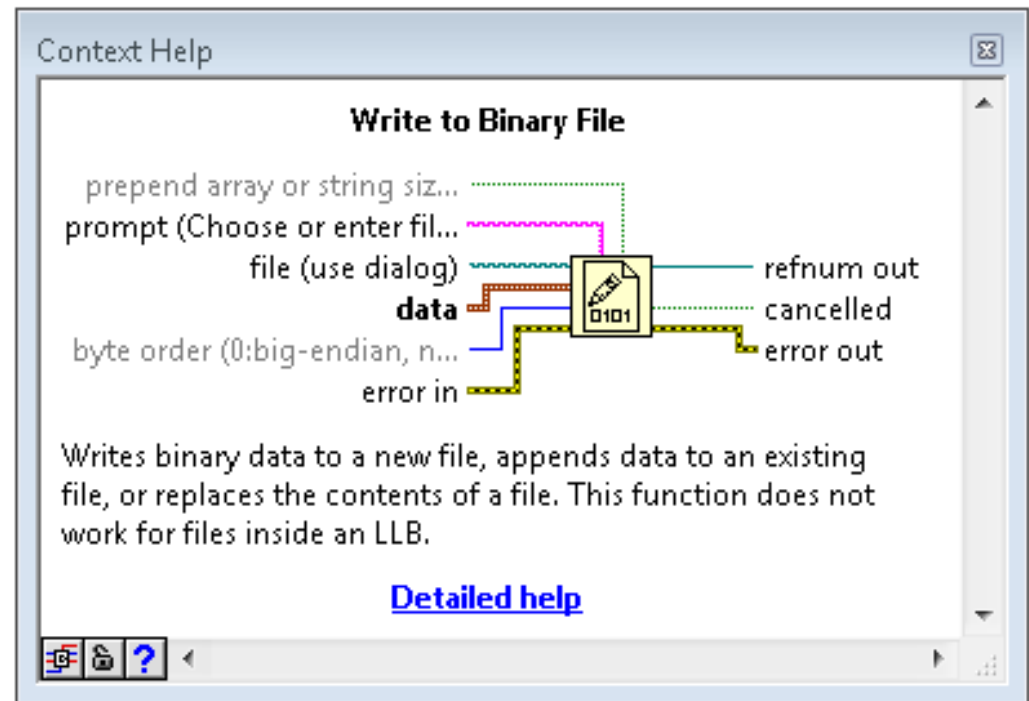
Using SubVIs

Options to place a subVI on the block diagram:

- Drag the VI from the Project Explorer to the block diagram.
- Click **Select a VI** on the **Functions** palette and then navigate to the VI.
- Drag the icon from an open VI to the block diagram of another VI.

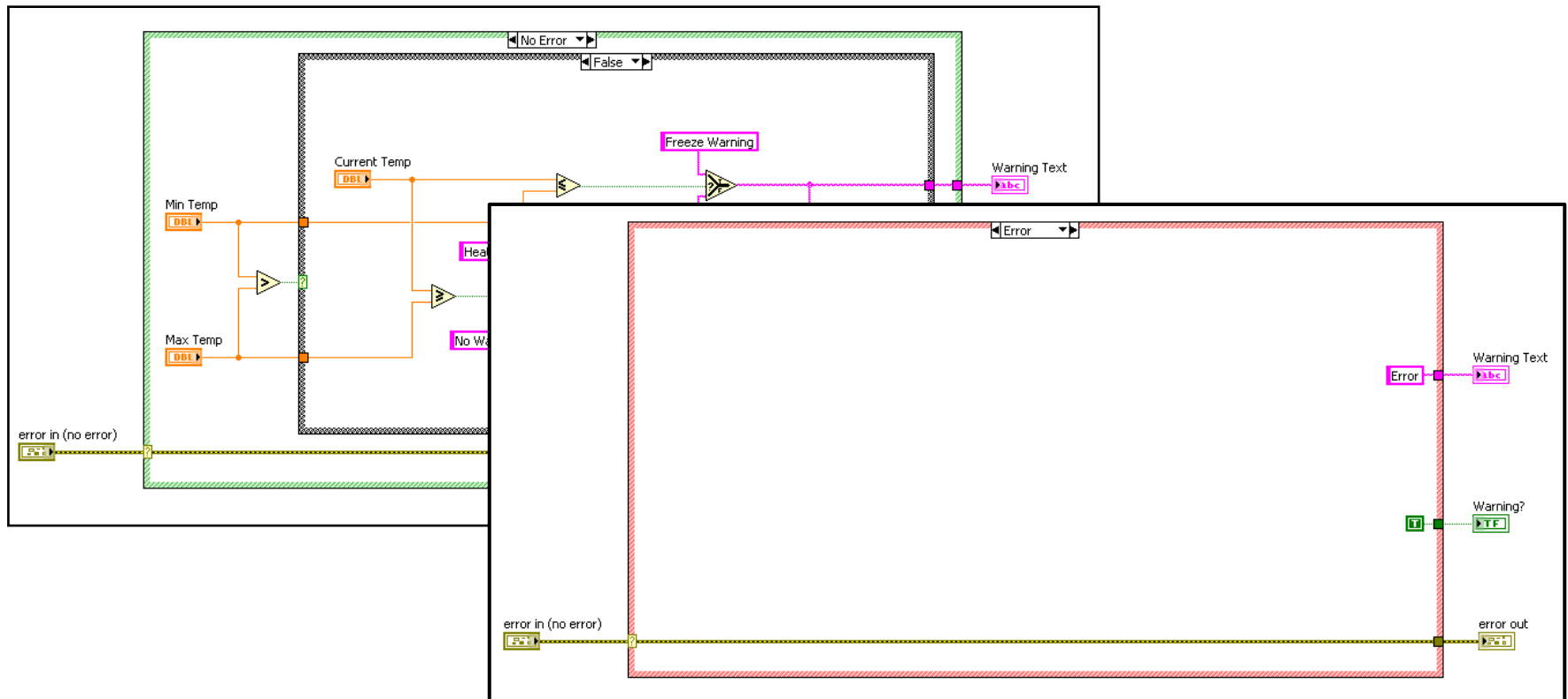
Terminal Settings

- **Bold**
 - Required terminal
- Plain
 - Recommended terminal
- Dimmed
 - Optional terminal



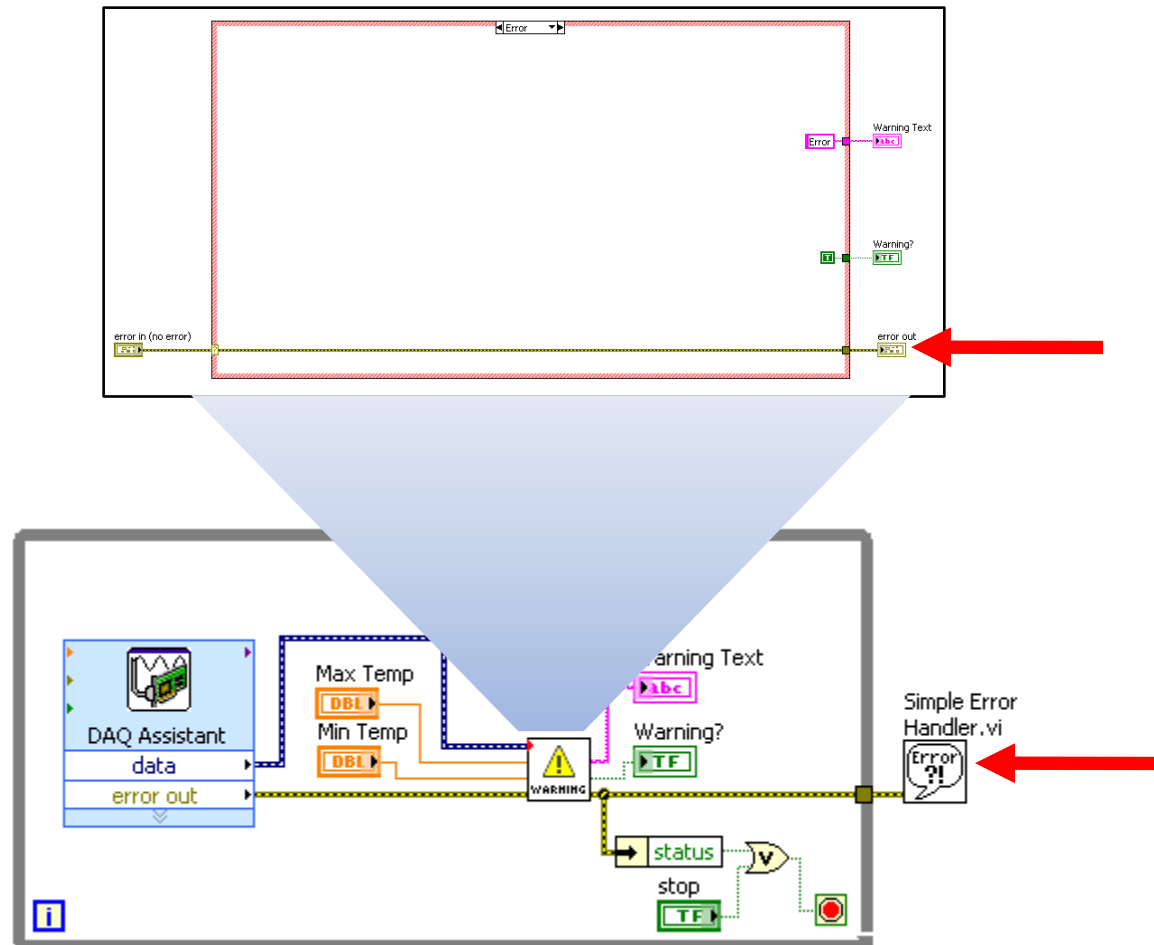
Handling Errors

Use a Case structure to handle errors passed into the subVI.



Handling Errors

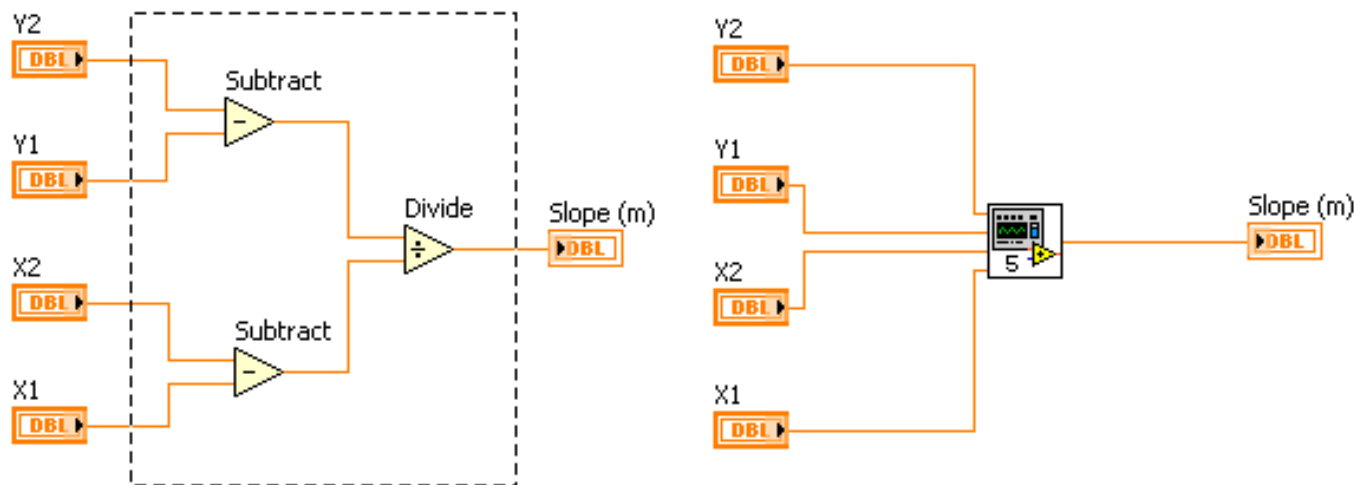
Avoid using LabVIEW error handler VIs inside subVIs.



Convert a Section of a VI to SubVI

To convert a section of a VI into a subVI:

1. Use the Positioning tool to select the section of the block diagram you want to reuse.
2. Select **Edit»Create SubVI**.



Summary—Quiz

1. On a subVI, which terminal setting causes a broken VI if the terminal is not wired?
 - a) Required
 - b) Recommended
 - c) Optional

Summary—Quiz Answer

1. On a subVI, which terminal setting causes a broken VI if the terminal is not wired?
 - a) **Required**
 - b) Recommended
 - c) Optional

Creating and Leveraging Data Structures

TOPICS

- A. Arrays
- B. Common Array Functions
- C. Polymorphism
- D. Auto-Indexing
- E. Clusters

A. Arrays

1D and 2D Arrays

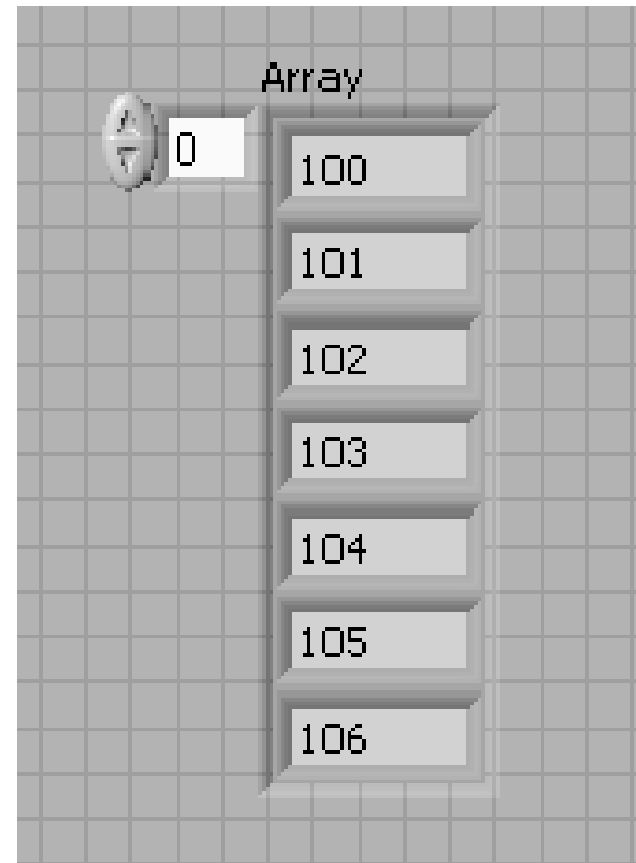
Creating an Array Control and Constant
Initializing Arrays

Arrays

An array:

- Is a collection of data elements that are of same type.
- Has one or more dimensions.
- Contains up to $(2^{31})-1$ elements per dimension, memory permitting.
- Accesses elements by its index.

Note: The first element is index 0.



Arrays – 1D and 2D Examples

1D array

One row of 10-elements

0	1	2	3	4	5	6	7	8	9

2D array

Five-row by seven-column table of 35 elements

	0	1	2	3	4	5	6
0							
1							
2							
3							
4							

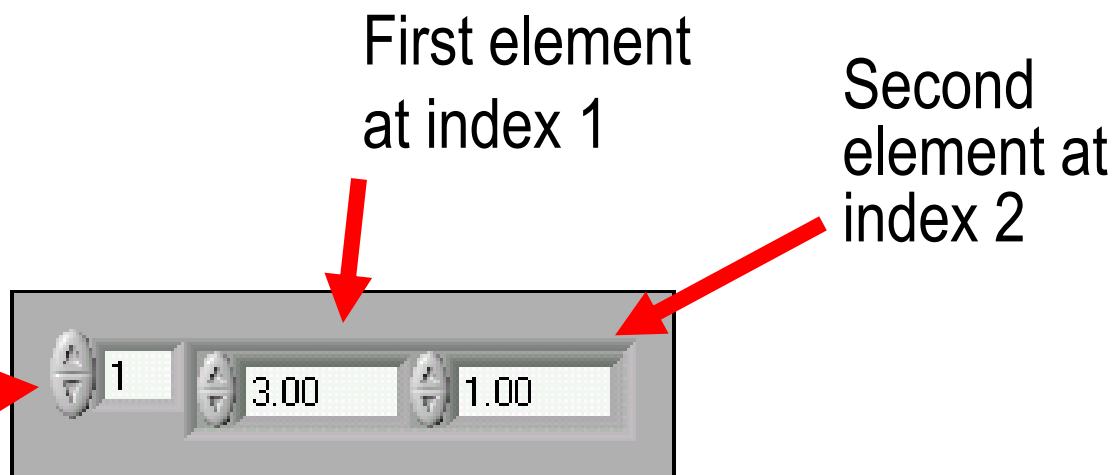
Index
numbers

Why Use Arrays?

Use arrays when you work with a collection of similar data and when you perform repetitive computations.

Viewing Arrays on the Front Panel

The elements at index 0 are not shown because element 1 is selected in the index display.

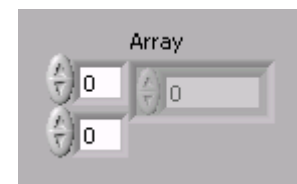
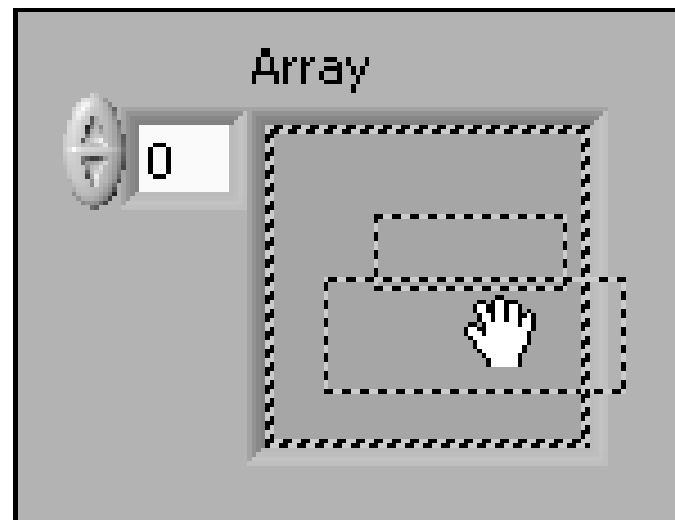


The element selected in the index display always refers to the element shown in the upper-left corner of the element display.

Creating an Array Control

For a new array:

1. Select an Array control from the **Controls** palette on the front panel.
2. Place a data object, such as a numeric control, into the array shell.
3. Add a second dimension, if necessary, by resizing the index.



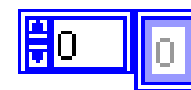
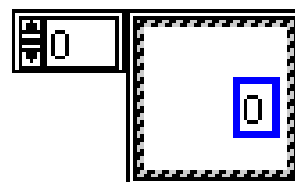
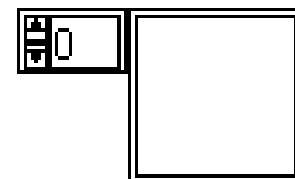
From a block diagram terminal or wire:

1. Right-click the object and select **Create»Control** or **Create»Indicator**.

Creating an Array Constant

For a new array:

1. Select Array Constant from the **Functions** palette on the block diagram.
2. Place a constant, such as a numeric, into the array shell.
3. Add a second dimension, if necessary, by resizing the index.

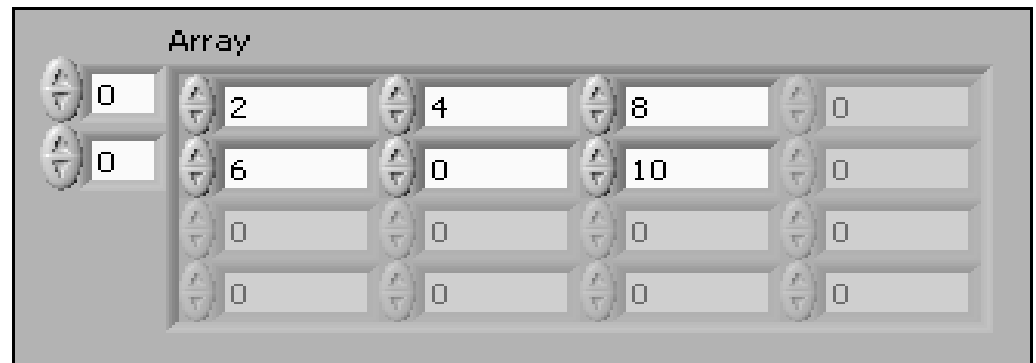
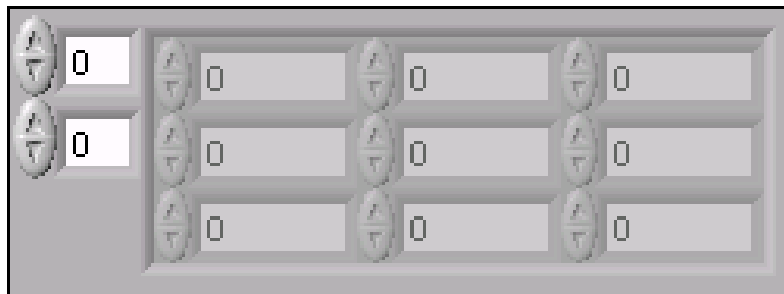


From a block diagram terminal or wire:

1. Right-click and select **Create»Constant**.

Initializing Arrays

- You can initialize an array or leave it uninitialized.
- For initialized arrays, you define the number of elements in each dimension and the contents of each element.
- Uninitialized arrays have dimension but no elements.



2D Arrays

- 2D arrays:
 - Store elements in a grid.
 - Require a column index and a row index to locate an element, both of which are zero-based.
- Create a multidimensional array on the front panel by right-clicking the index display and selecting **Add Dimension** from the shortcut menu.
- Resize the index display until you have as many dimensions as you want.



B. Common Array Functions

Array Size

Initialize Array

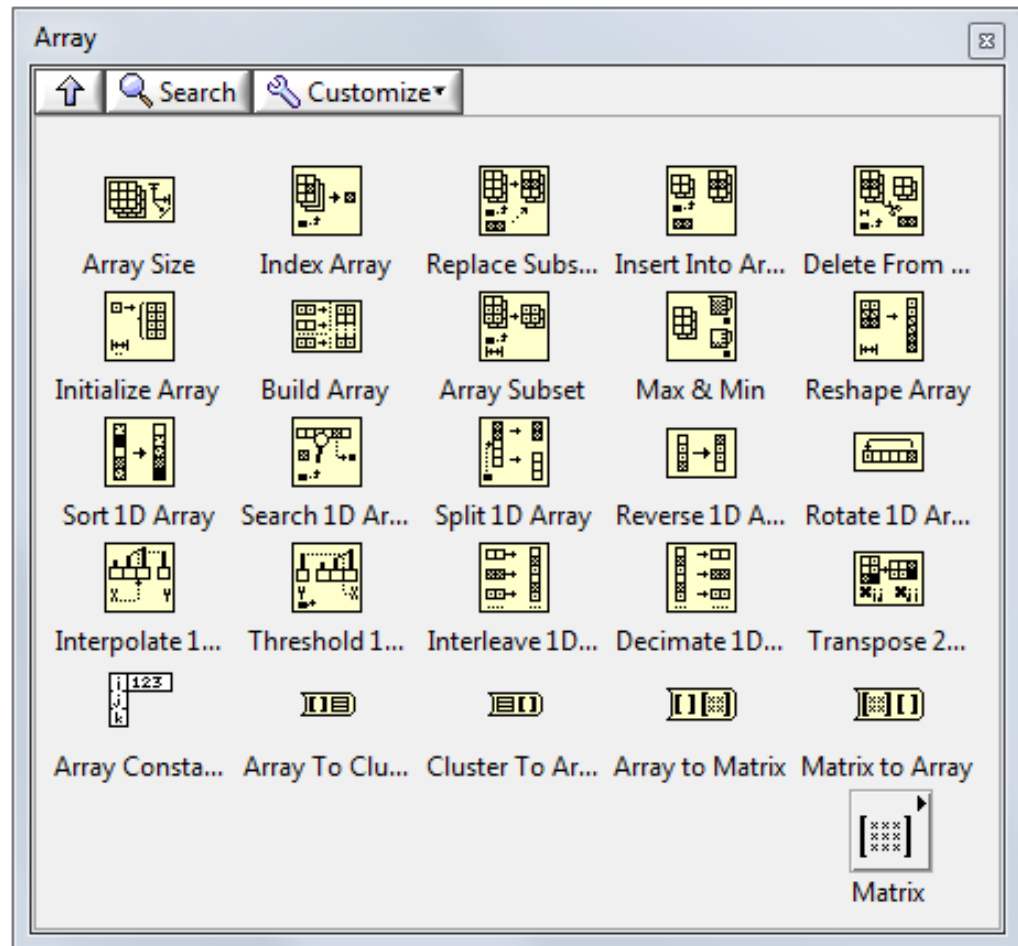
Array Subset

Build Array

Index Array

Common Array Functions

- Array Size
- Initialize Array
- Array Subset
- Build Array
- Index Array



Array Size

1-D Array



Array Size



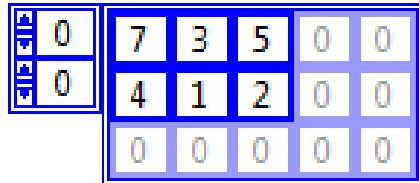
1-D Array Size

132

1-D Array Size

4

2-D Array



Array Size



2-D Array Size

132

2-D Array Size

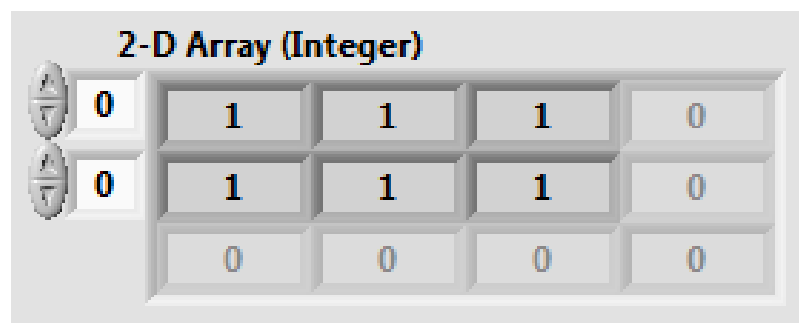
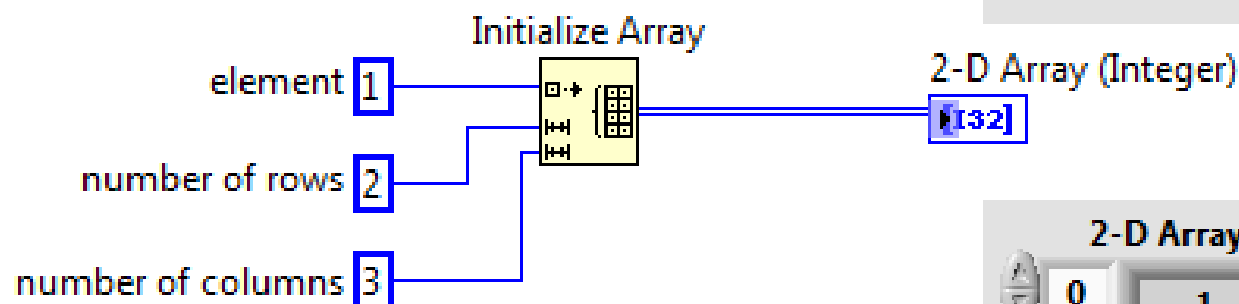
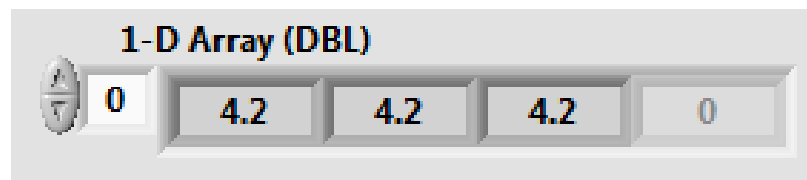
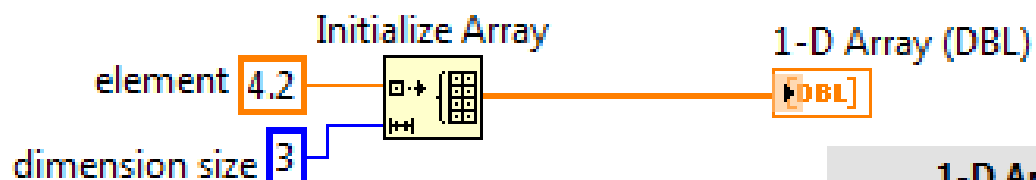
0

2

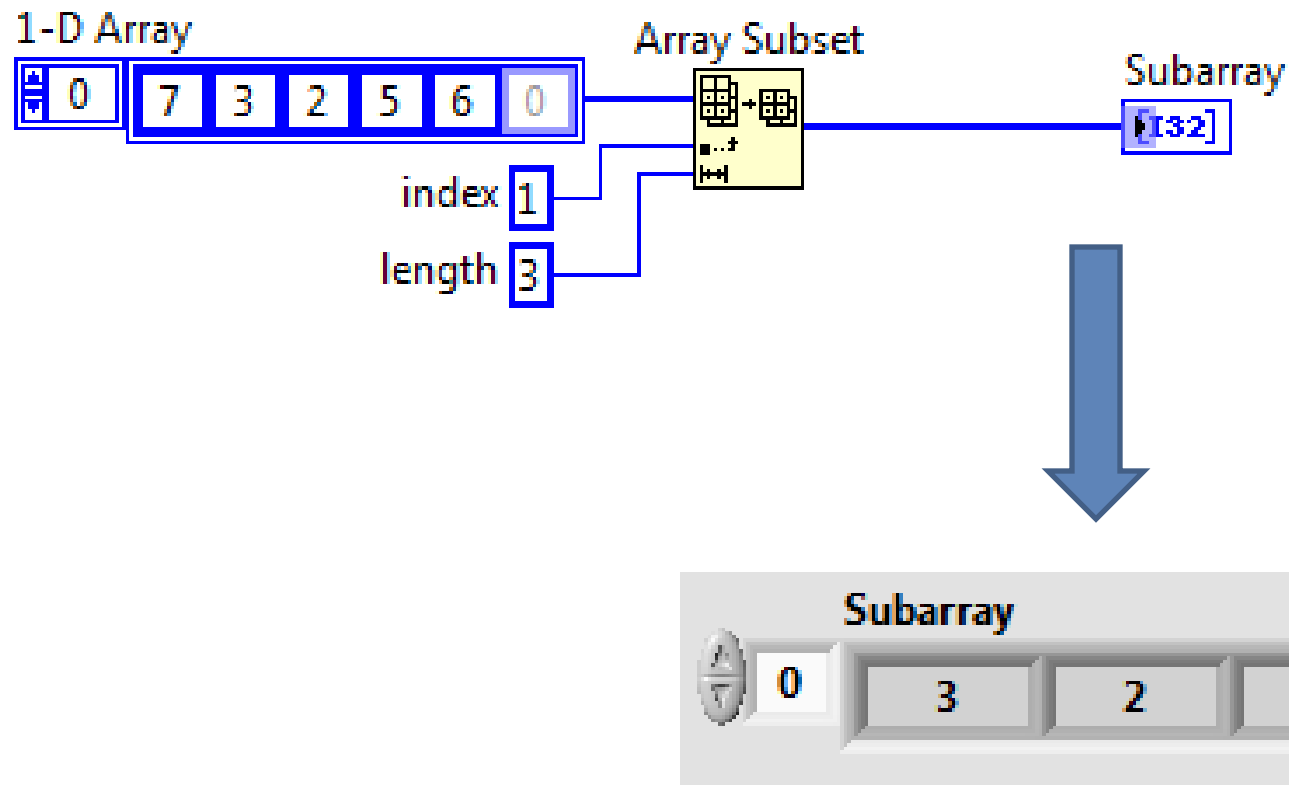
3

0

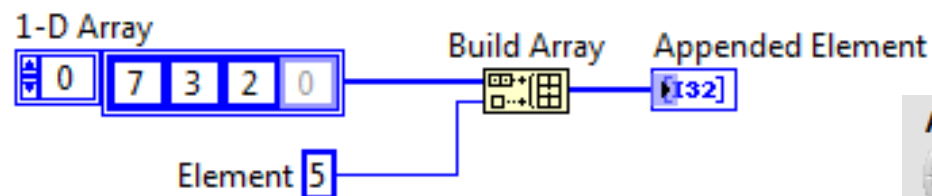
Initialize Array



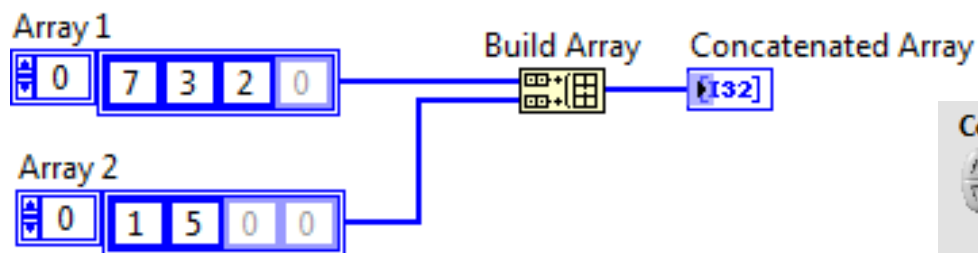
Array Subset



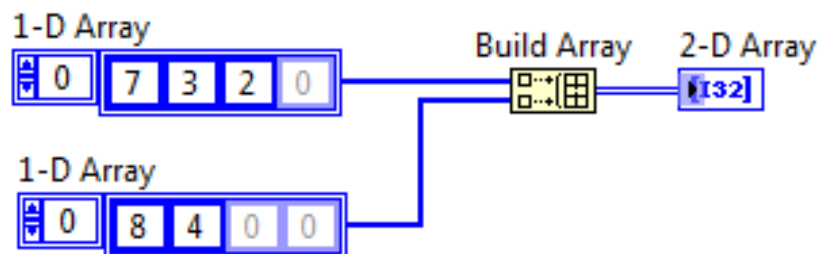
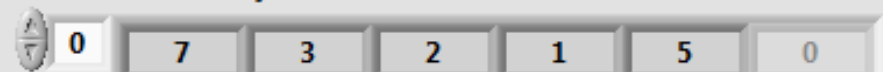
Build Array



Appended Element



Concatenated Array

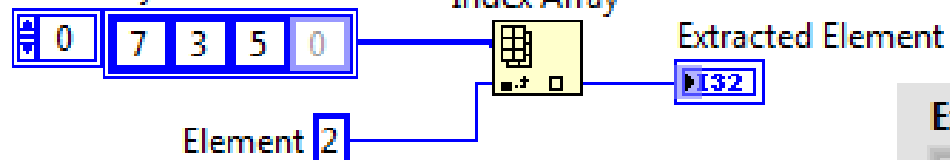


2-D Array



Index Array

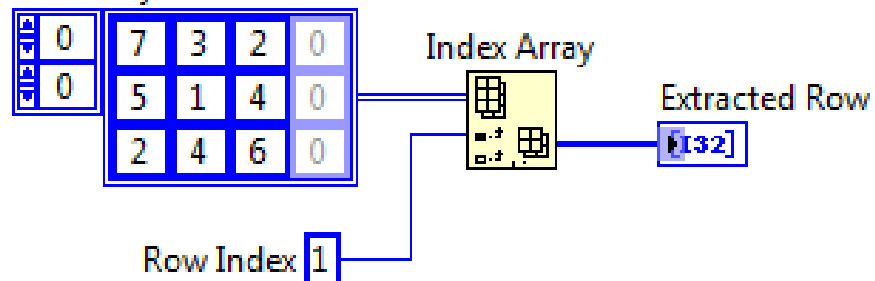
1-D Array



Extracted Element

5

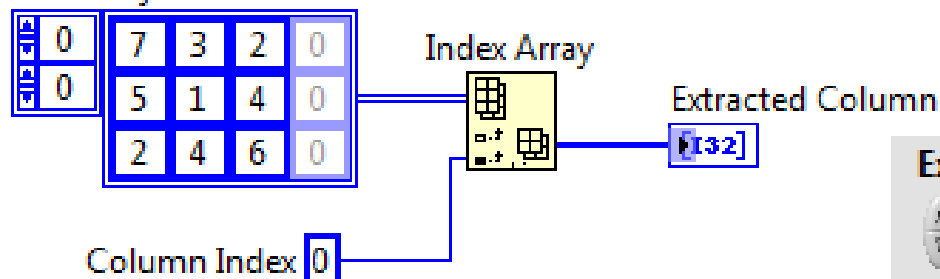
2-D Array



Extracted Row

0 5 1 4 0

2-D Array

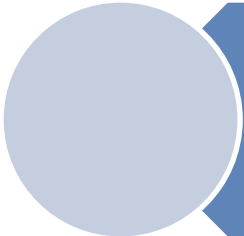


Extracted Column

0 7 5 2 0

C. Polymorphism

Polymorphism



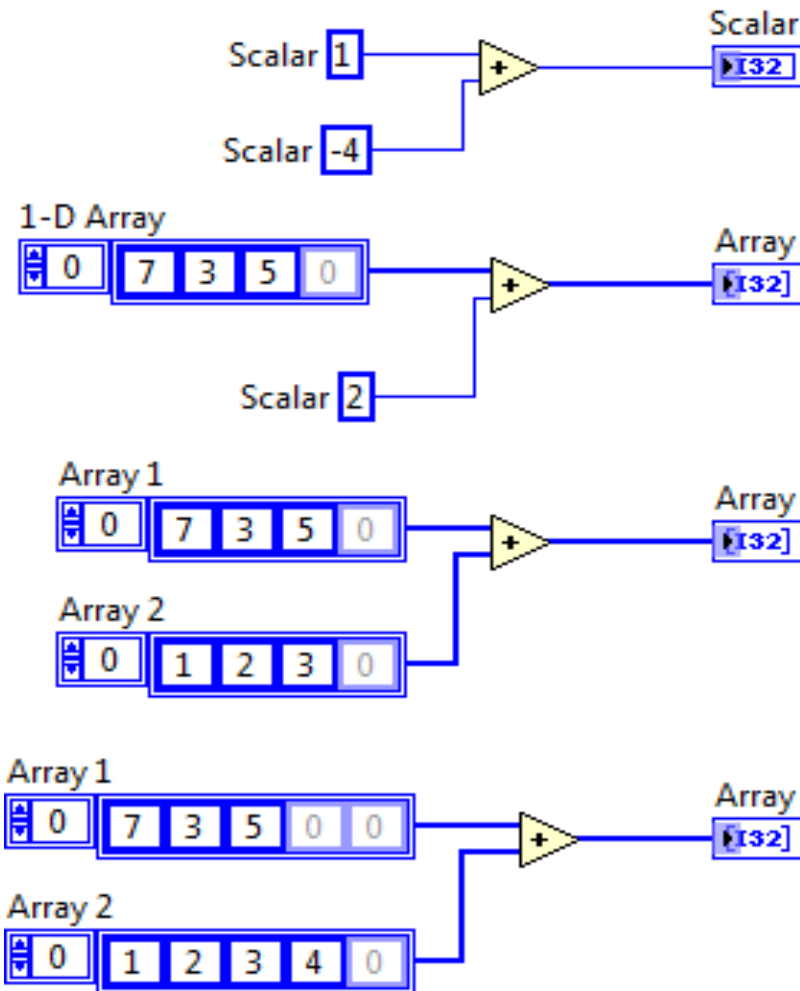
Polymorphism - The ability of VIs and functions to automatically adapt to accept input data of different data types

Functions are polymorphic to varying degrees:

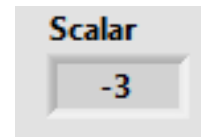
- None, some, or all of their inputs can be polymorphic.
- Some accept numeric or Boolean values.
- Some accept numeric or strings.
- Some accept scalars, numeric arrays, or clusters of numerics.

Arithmetic Functions Are Polymorphic

Combination



Result



D. Auto-Indexing

Use in For Loops and While Loops

Waveform Graphs

Auto-Indexing with a Conditional Terminal

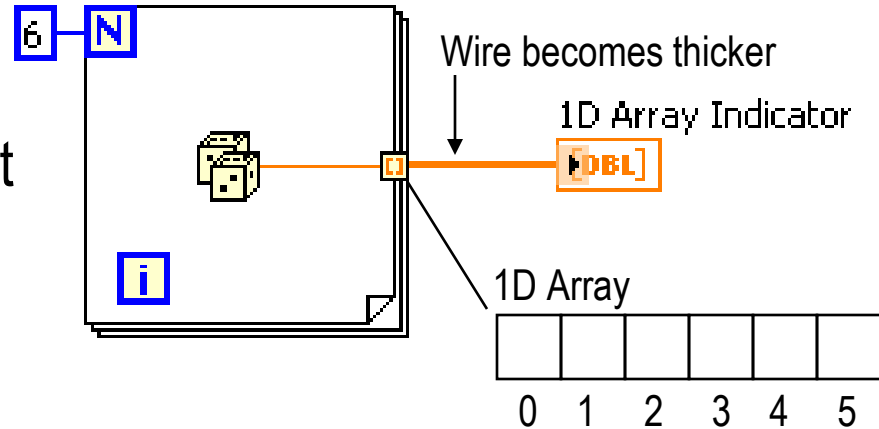
Creating 2D Arrays

Auto-Indexing Input to a Loop

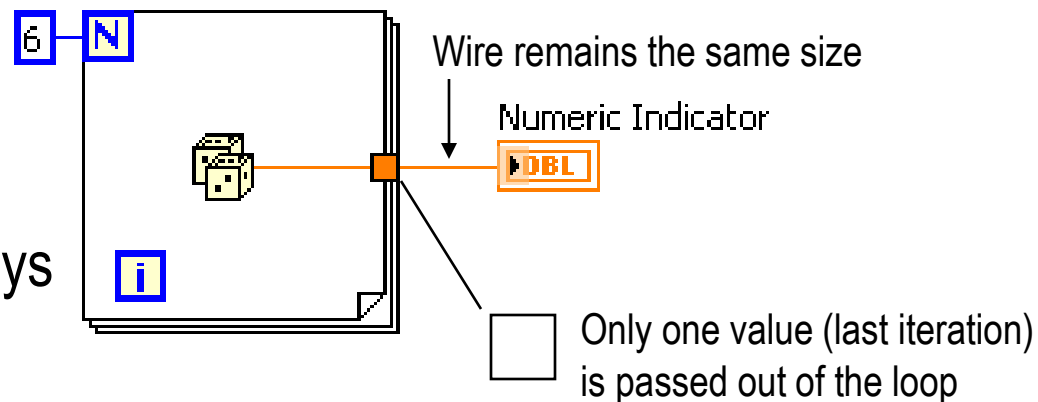
Auto-Indexing

- Allows For Loops and While Loops to accumulate arrays at their boundaries.
- Is the default behavior for For Loops.
- Is disabled by default for While Loops.
- Is enabled/disabled by right-clicking on a tunnel.
- Produces arrays that are always equal in size to the number of iterations of the loop.

Auto-Indexing Enabled

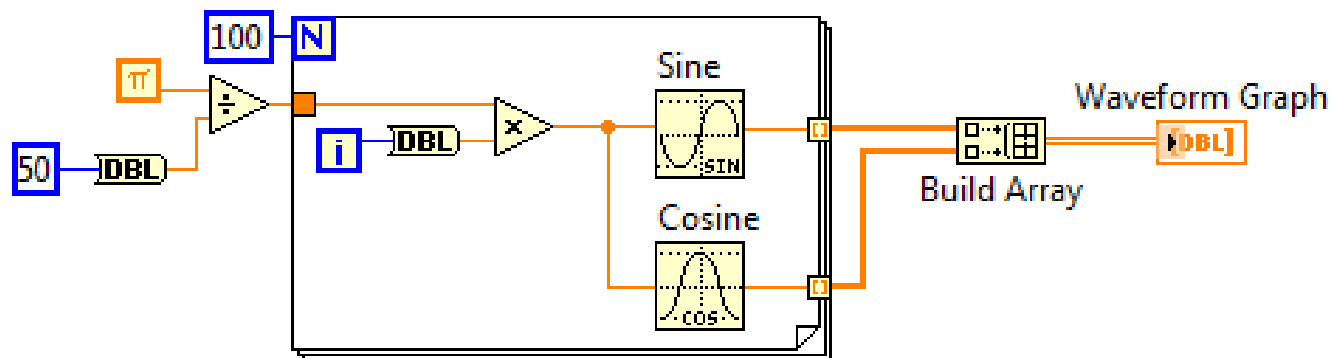
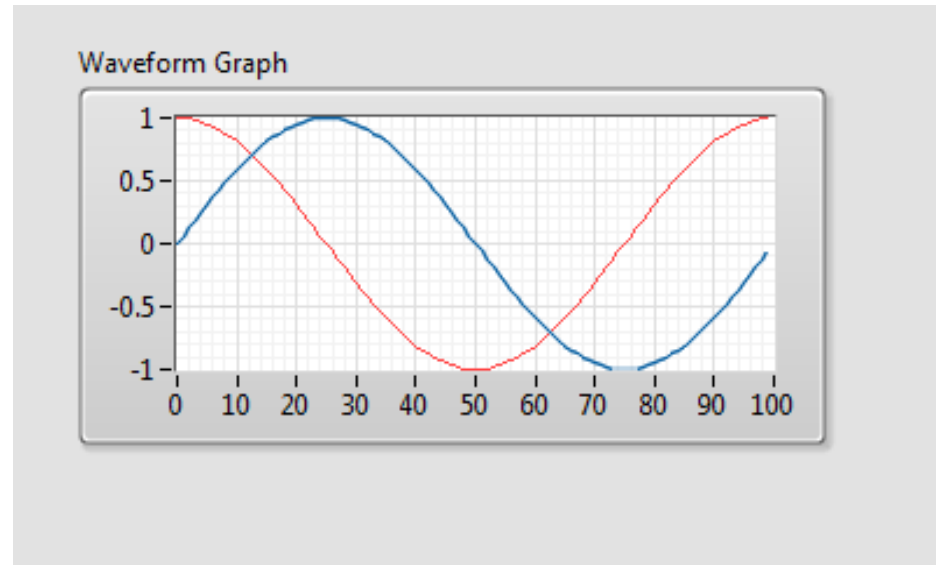


Auto-Indexing Disabled

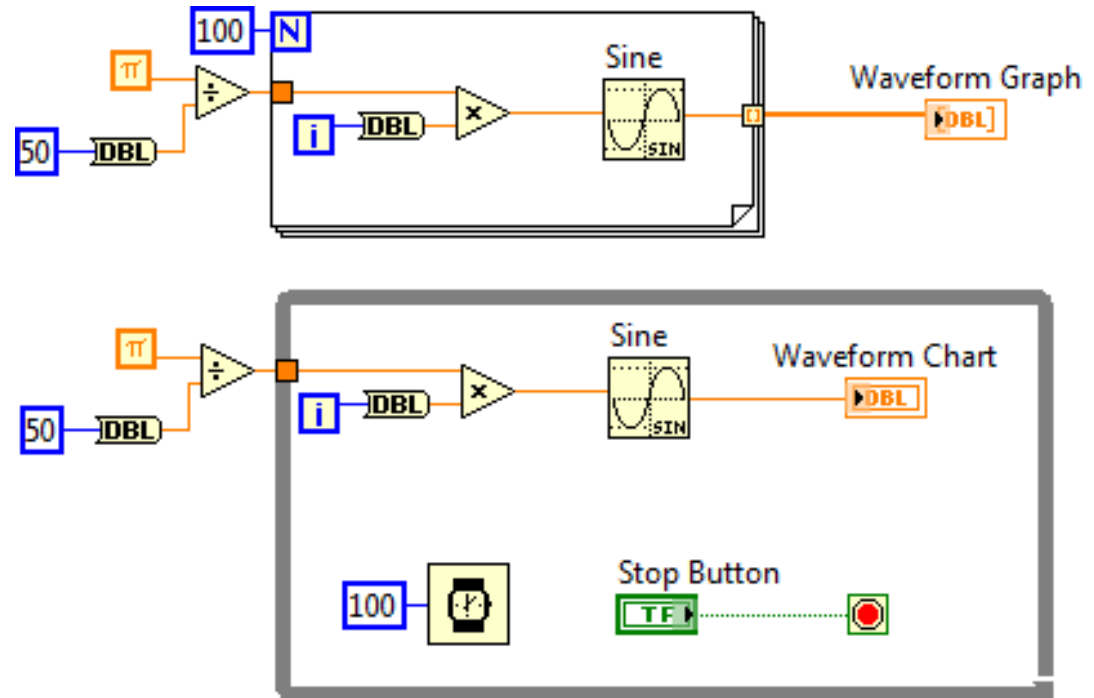
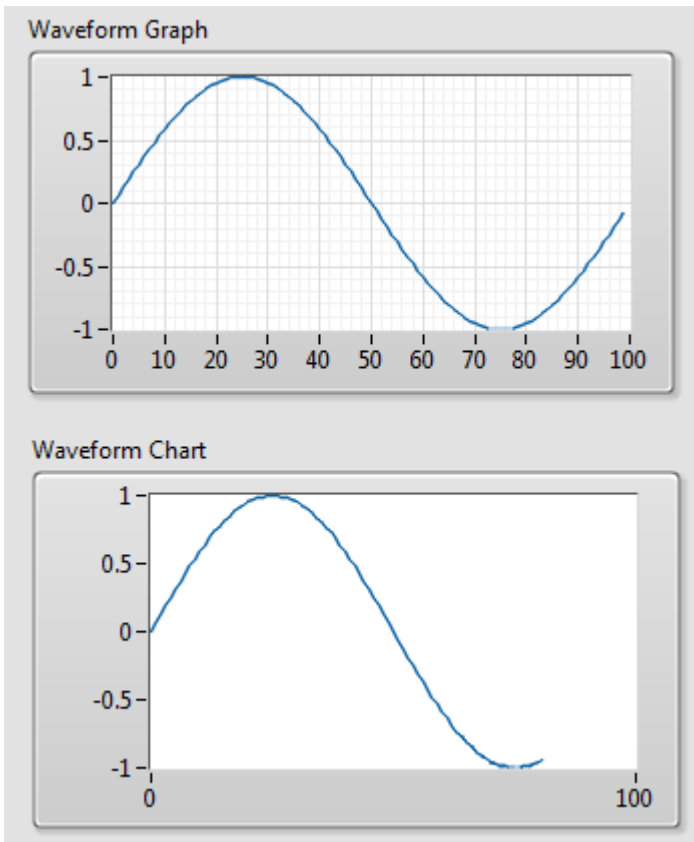


Waveform Graph

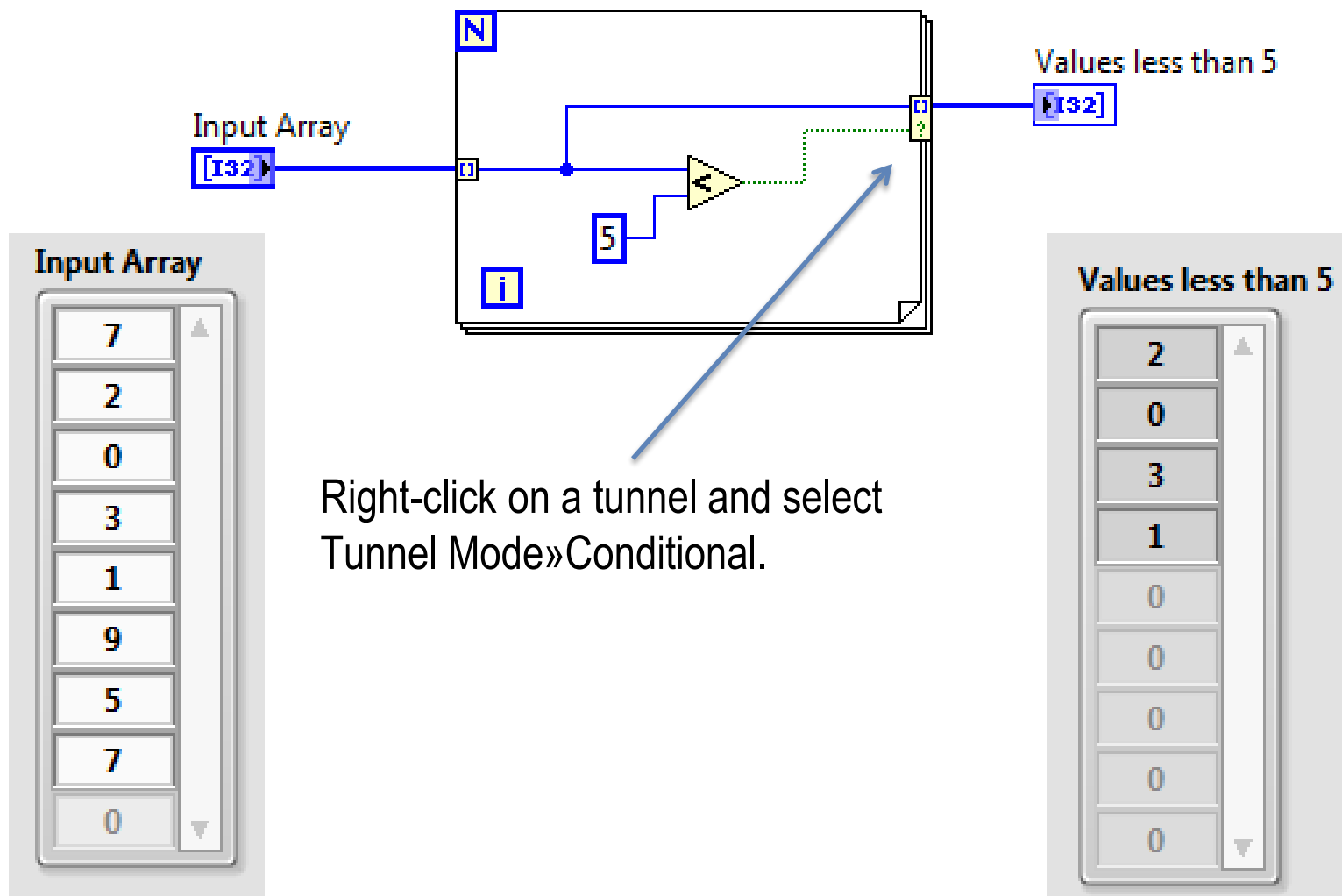
- Is a graphical display of data.
- Displays one or more plots of evenly sampled measurements.
- Is used to plot pre-generated arrays of data.
- Can display plots with any number of data points.



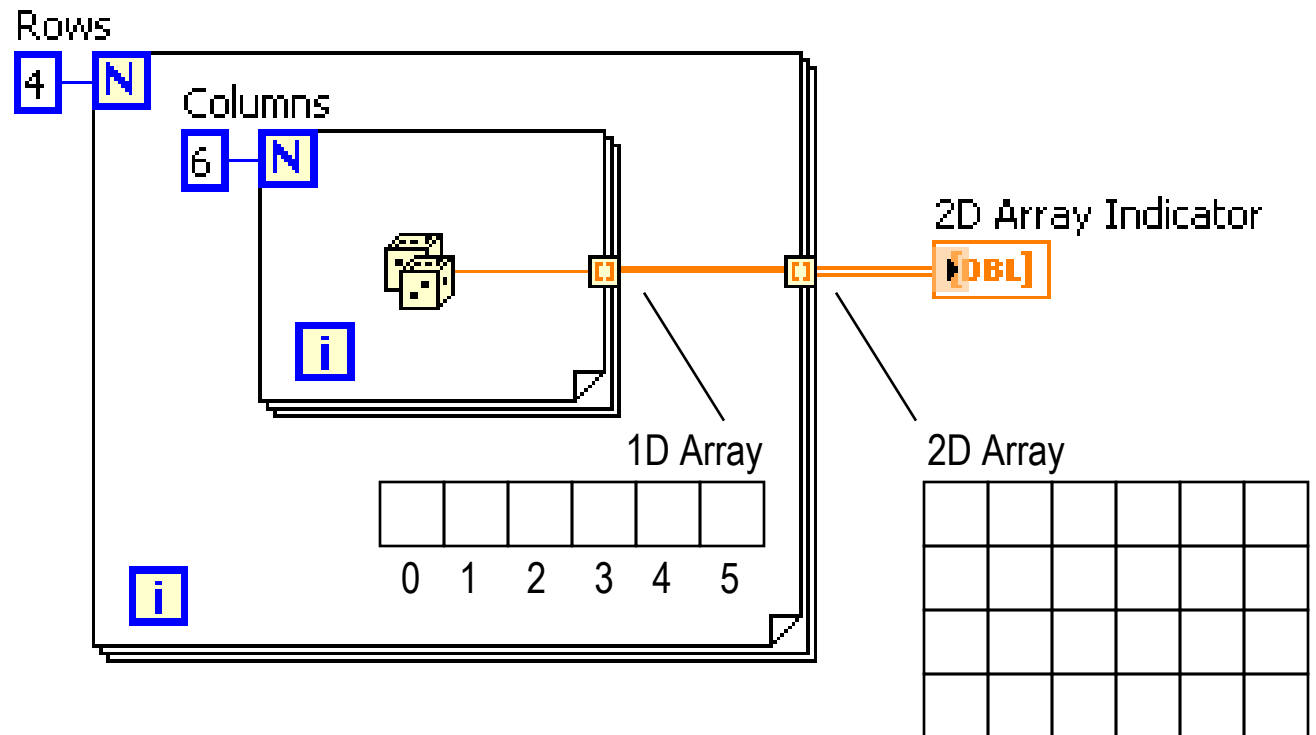
Charts vs. Graphs – Single-Plot



Auto-Indexing with a Conditional Tunnel



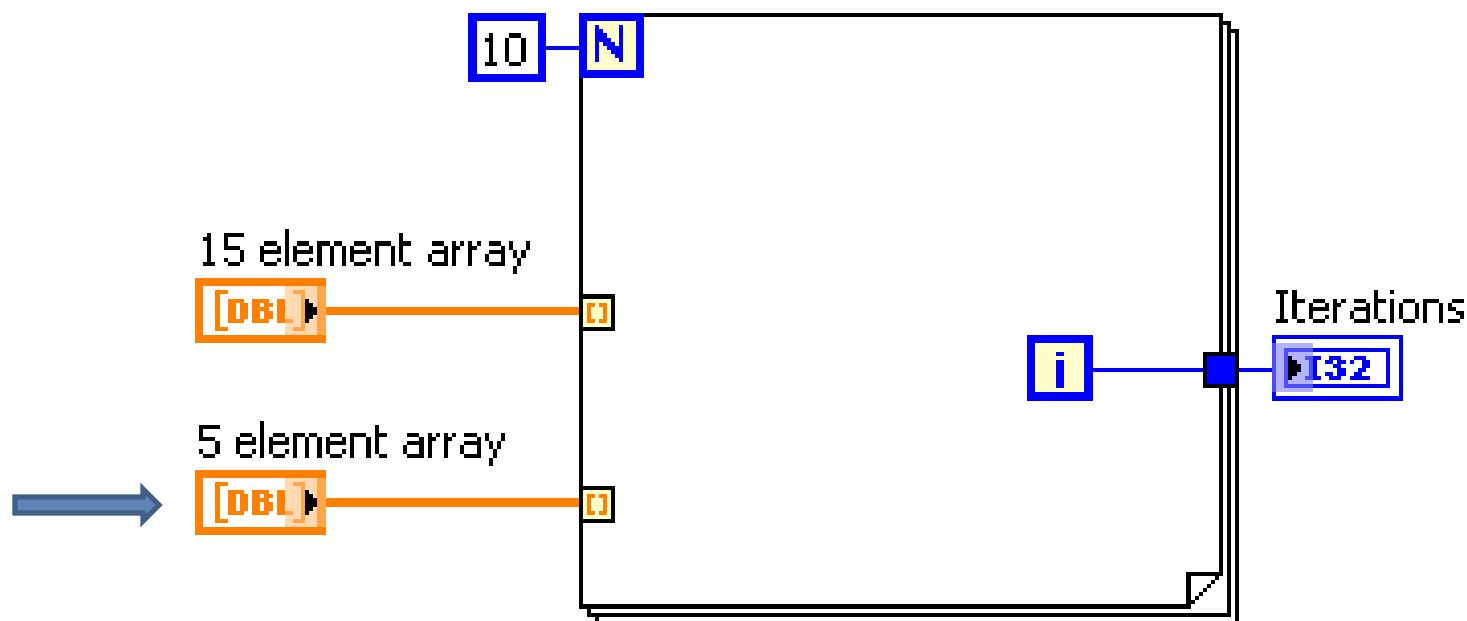
Creating 2D Arrays



- Inner loop creates column elements.
- Outer loop stacks column elements into rows.

Auto-Indexing Input

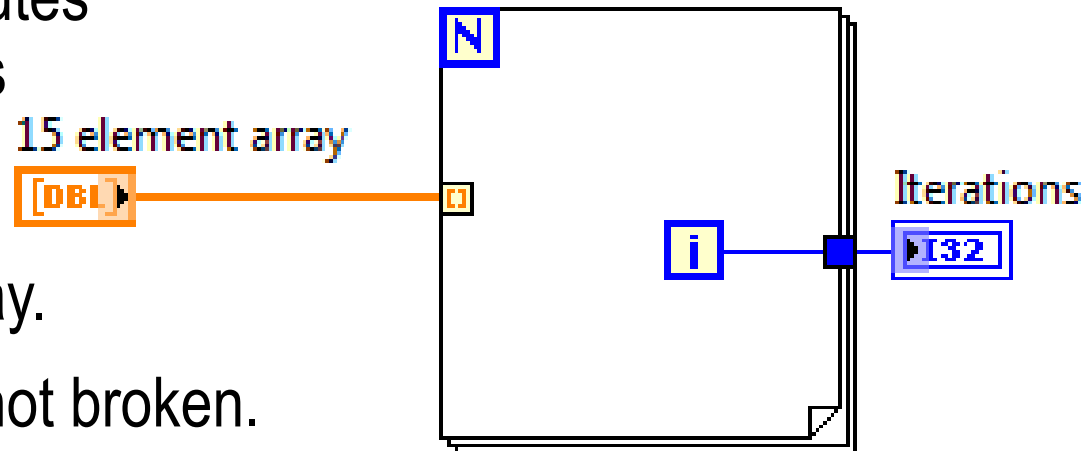
If the iteration count terminal is wired and arrays of different sizes are wired to auto-indexed tunnels, the actual number of iterations becomes the smallest of the choices.



Auto-Indexing Input

Use an auto-indexing input array to perform calculations on each element in an array.

- Wire an array to an auto-indexing tunnel on a For Loop.
- You do not need to wire the count (N) terminal.
 - The For Loop executes the number of times equal to the number of elements in the array.
 - The **Run** button is not broken.



E. Clusters

Reasons To Use Clusters

Cluster vs. Arrays

Creating a Cluster Control and Constant

Ordering Items and Resizing Clusters

Disassembling and Modifying Clusters

Plotting Data

Clusters

- Clusters group data elements of mixed types.
- Clusters are similar to a record or a `struct` in text-based programming languages.

Weather Data In

Current Temperature

0

Max Temperature

0

Min Temperature

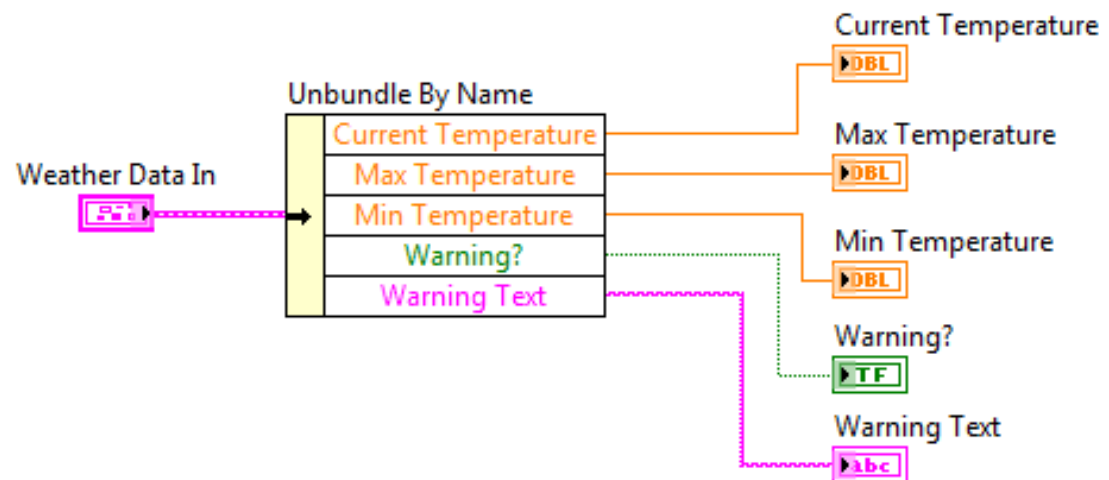
0

Warning?

☒

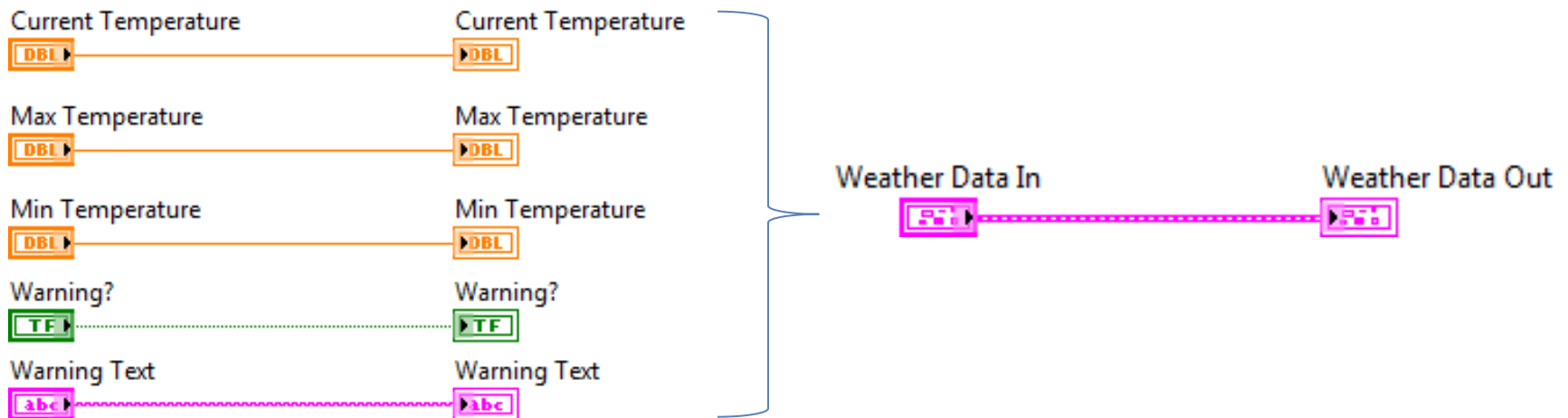
Warning Text

No Warning



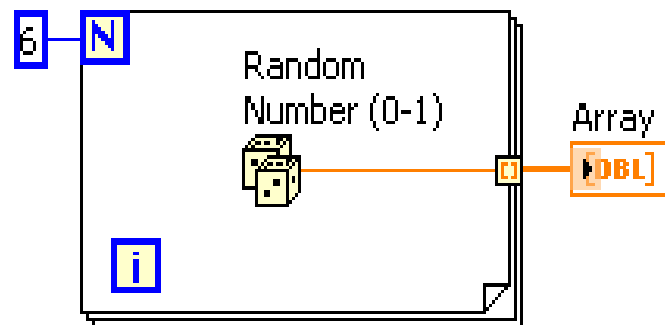
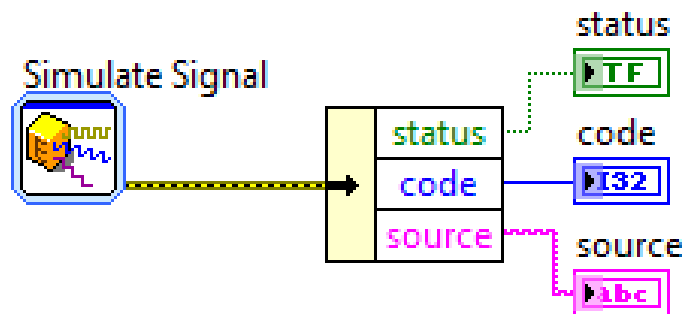
Why Use Clusters?

- Keep data organized.
 - Logically group related data values together.
 - Improve diagram readability by eliminating wire clutter.
- Reduce the number of connector pane terminals.



Clusters vs. Arrays

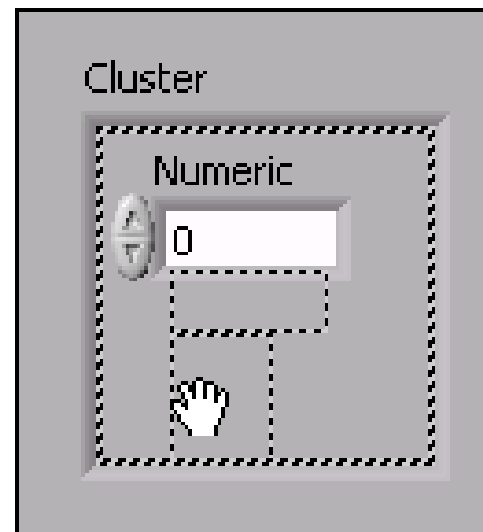
- Clusters are a fixed size.
 - Clusters can contain mixed data types.
 - Clusters can be a control, an indicator, or a constant.
 - All elements have to be controls, indicators, or constants.
- Arrays vary in size.
 - Arrays contain only one data type.
 - Arrays can be a control, an indicator, or a constant.



Create a Cluster Control

For a new cluster:

1. On the front panel, select Cluster from the **Controls** palette.
2. Place a data object into the cluster shell.
3. Place additional data objects, if necessary, into the shell.



From block diagram terminal or wire:

1. Right-click and select **Create»Control** or **Create»Indicator**.

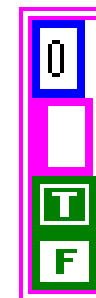
Create a Cluster Constant

For a new cluster:

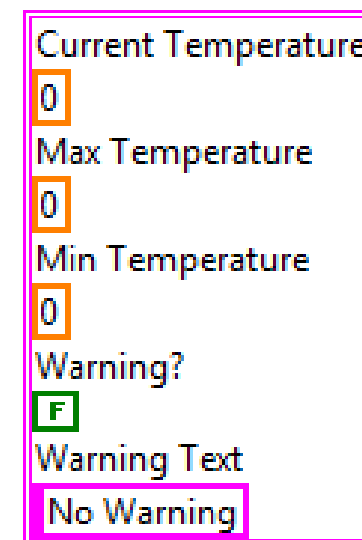
1. On the block diagram, select Cluster Constant from the **Functions** palette.
2. Place a constant into the cluster shell.
3. Place additional data objects, if necessary, into the cluster shell.

From block diagram terminal or wire:

1. Right-click and select **Create»Constant**.

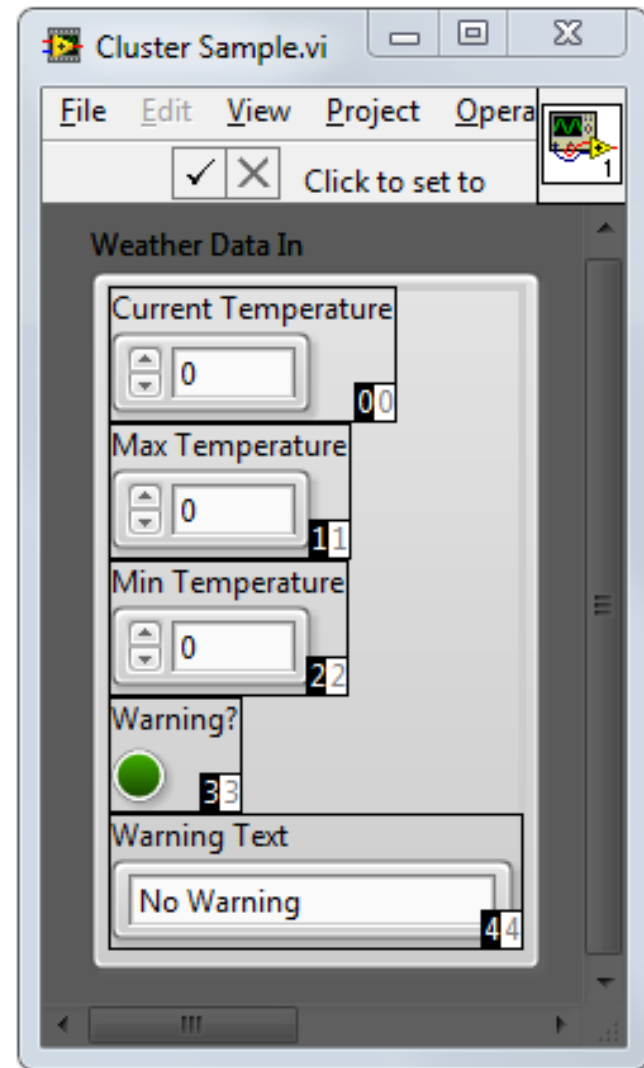


Weather Data



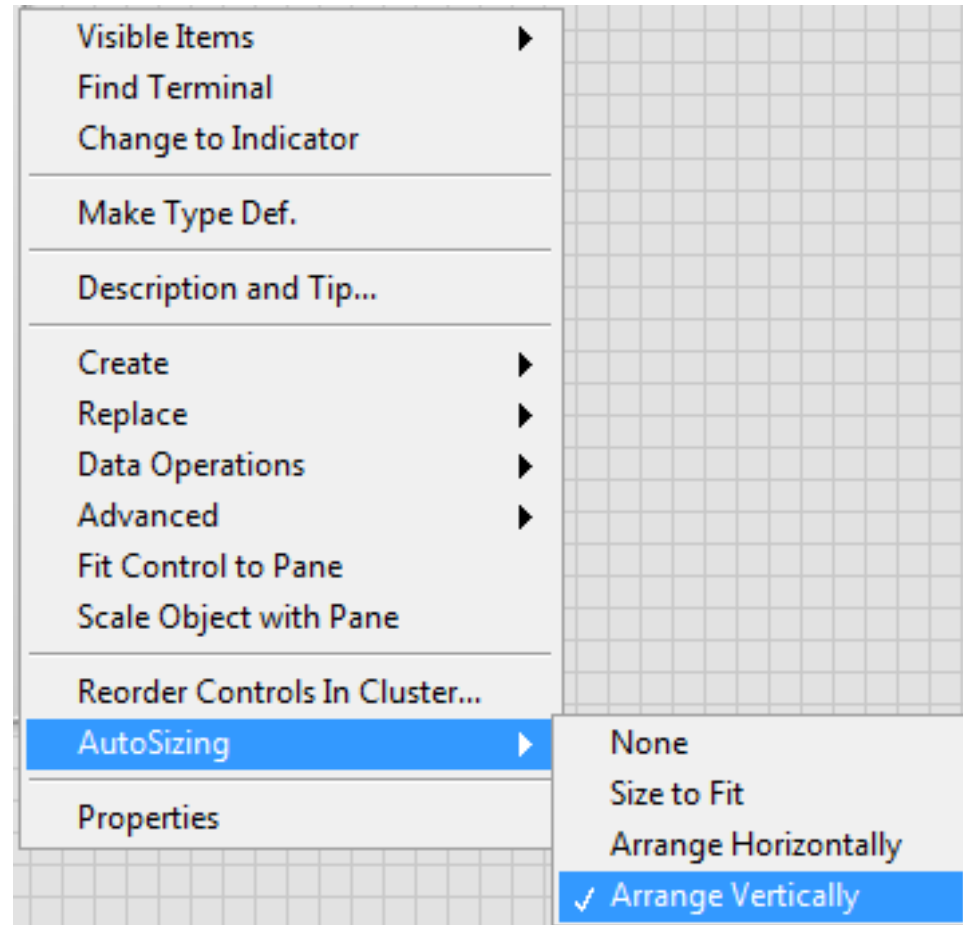
Cluster Order

- Cluster elements have a logical order unrelated to their position in the shell.
- You can view and modify the cluster order by right-clicking the cluster border and selecting **Reorder Controls In Cluster**.



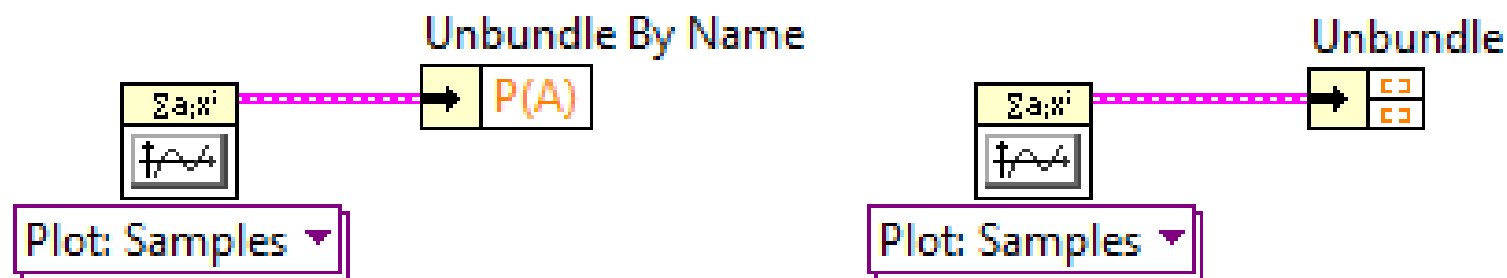
Autosizing Clusters

- Autosizing helps you arrange elements in clusters.
- NI recommends the following:
 - Arrange cluster elements vertically.
 - Arrange elements compactly.
 - Arranges elements in their preferred order.



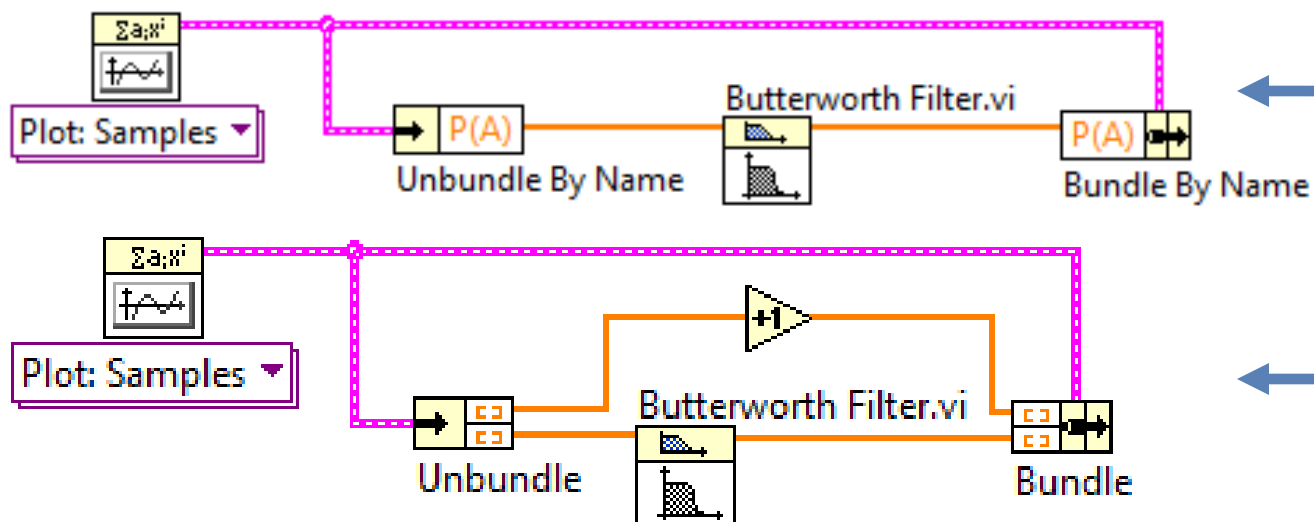
Disassembling a Cluster

- Use the Unbundle By Name function whenever possible.
- Use Unbundle function when some or all cluster elements are unnamed.



Modifying a Cluster

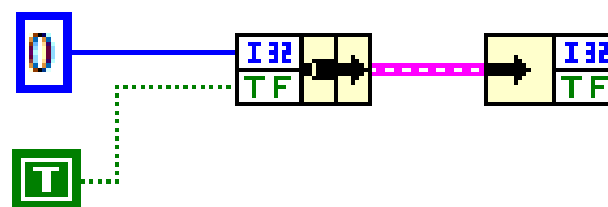
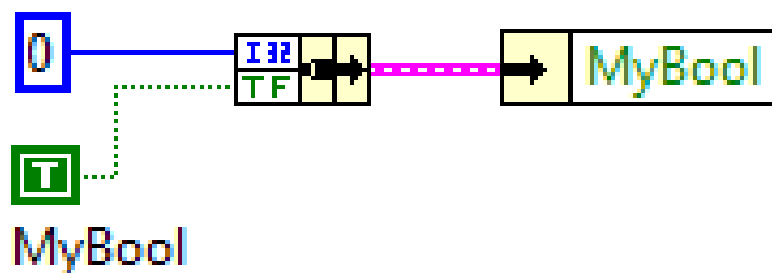
- Use Bundle By Name whenever possible to access elements in a cluster.
- Use Bundle when some or all cluster elements are unnamed.



You must wire the input cluster terminal.

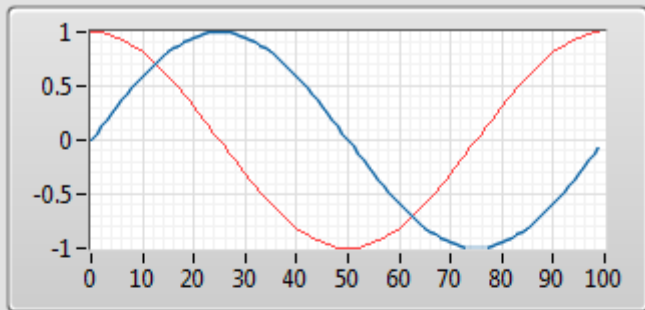
Creating a Cluster on the Diagram

- Use the Bundle function to programmatically create a cluster on a block diagram.
- If the elements that are bundled have labels, you can access them using the Unbundle By Name function. Otherwise use the Unbundle function.

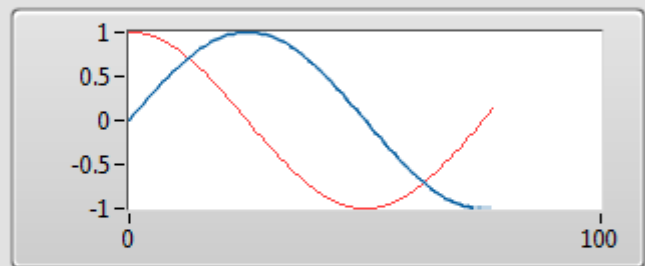


Charts vs. Graphs – Multi-plot and XY Graph

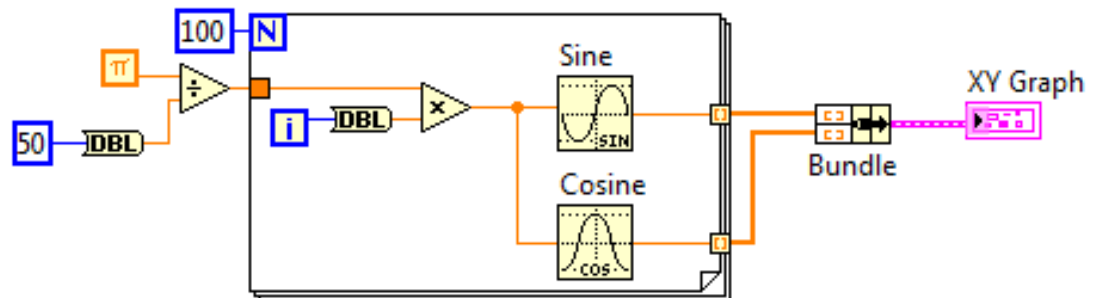
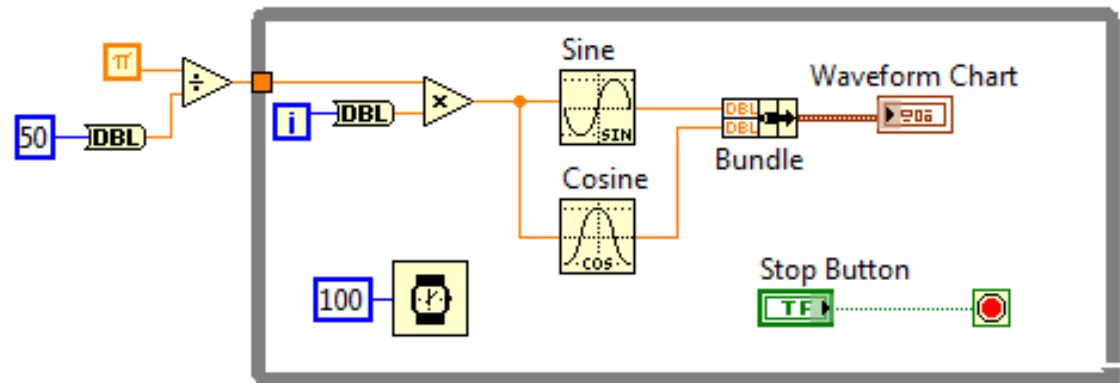
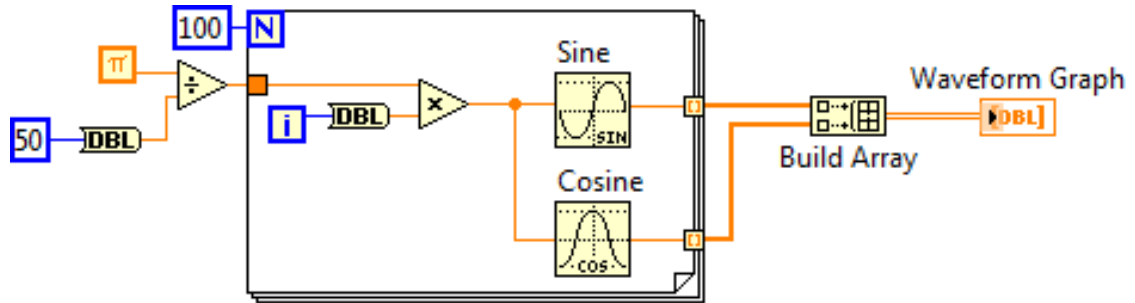
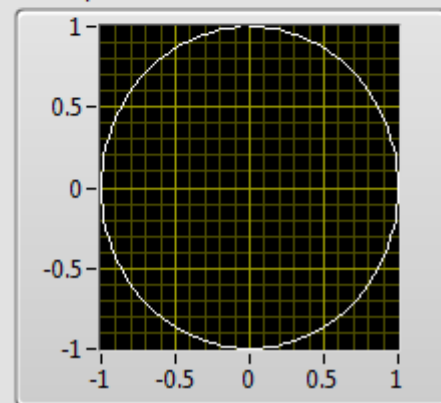
Waveform Graph



Waveform Chart

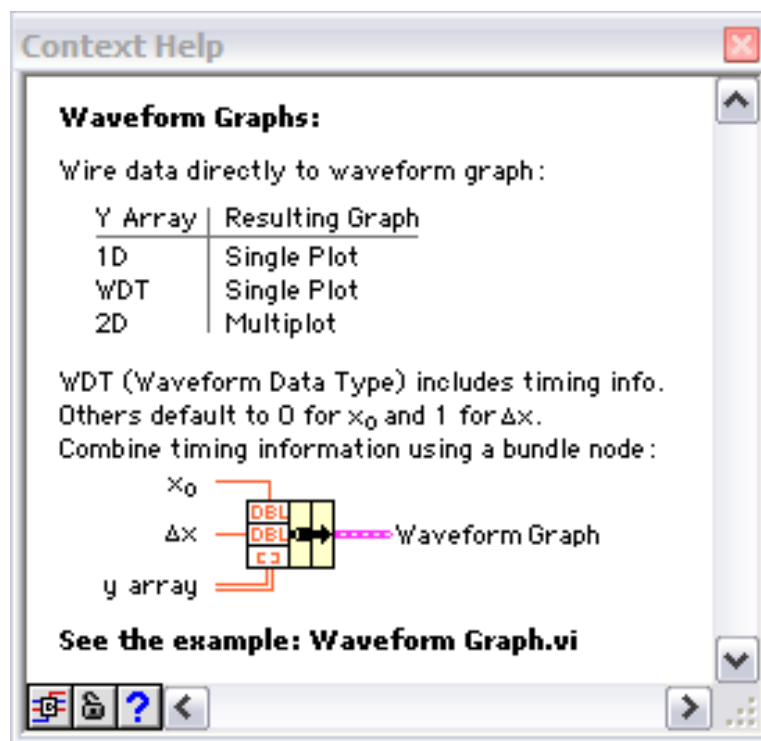


XY Graph



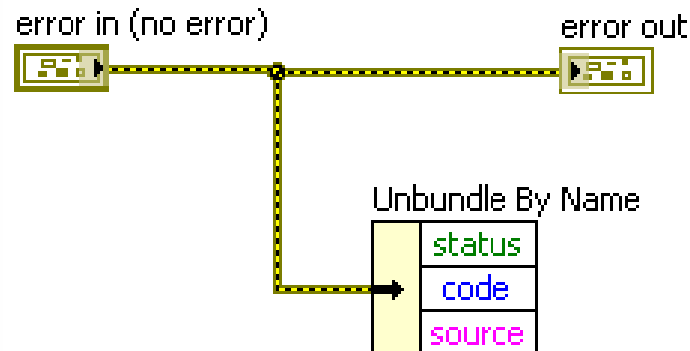
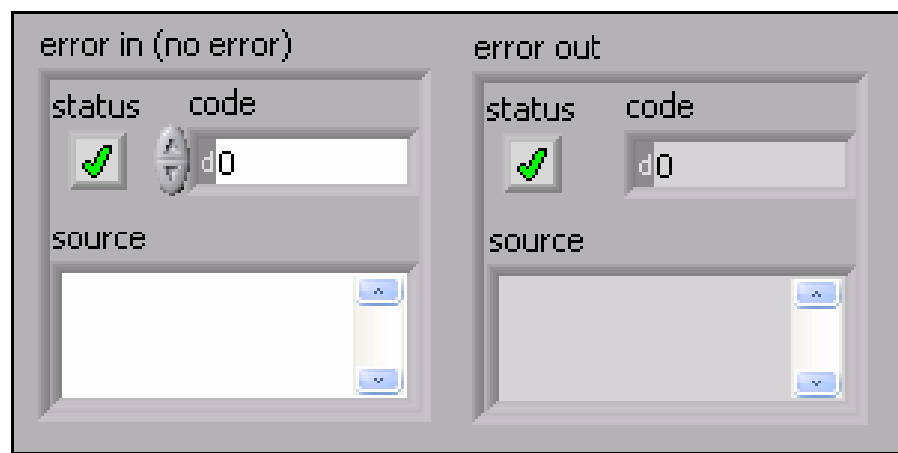
Plotting Data

Use the **Context Help** window to determine how to wire multi-plot data to Waveform Graphs, Charts and XY Graphs.



Error Clusters

- LabVIEW uses error clusters to pass error information.
- An error cluster contains the following elements:
 - **status**—Boolean value that reports True if an error occurs.
 - **code**—32-bit signed integer that identifies the error.
 - **source**—String that identifies where the error occurred.



Summary—Quiz

1. You can create an array of arrays.
 - a) True
 - b) False

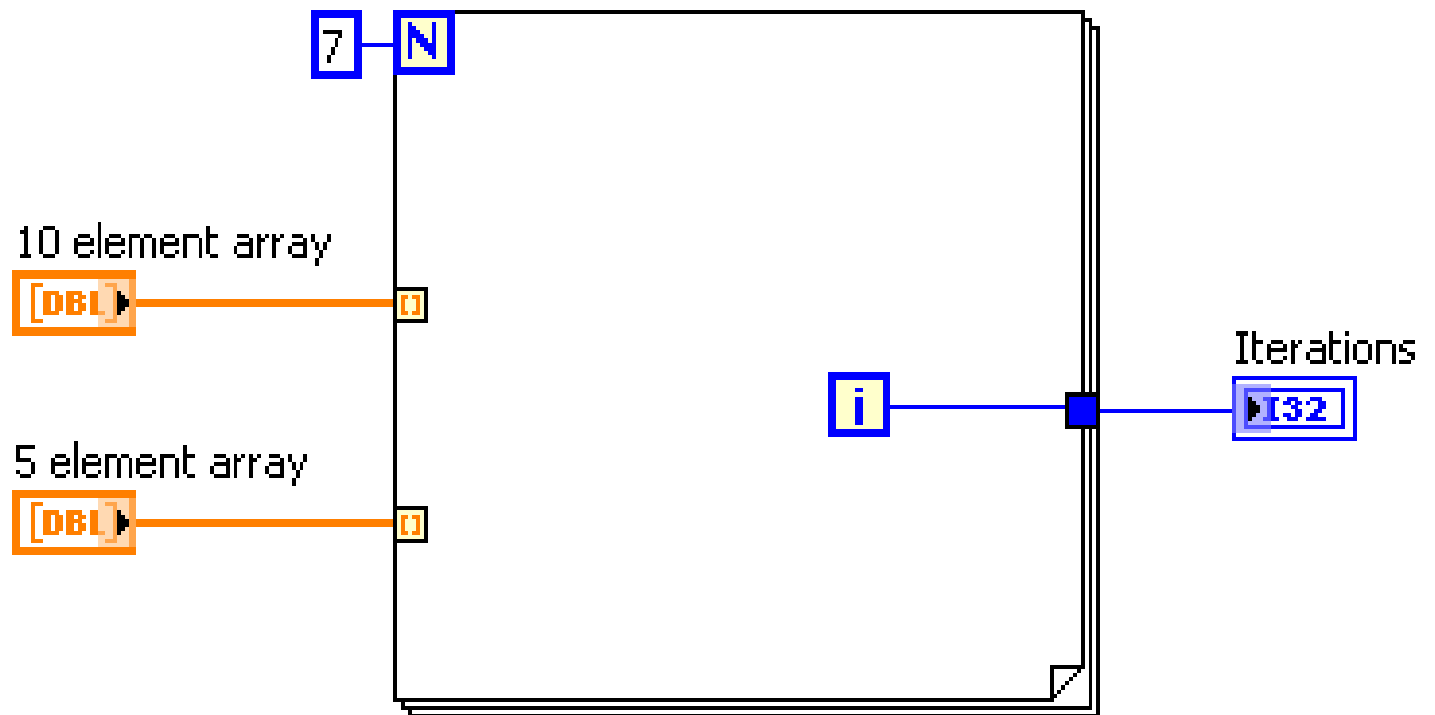
Summary—Quiz Answer

1. You can create an array of arrays.
 - a) True
 - b) False**

You cannot drag an array data type into an array shell. However, you can create two-dimensional arrays.

Summary—Quiz

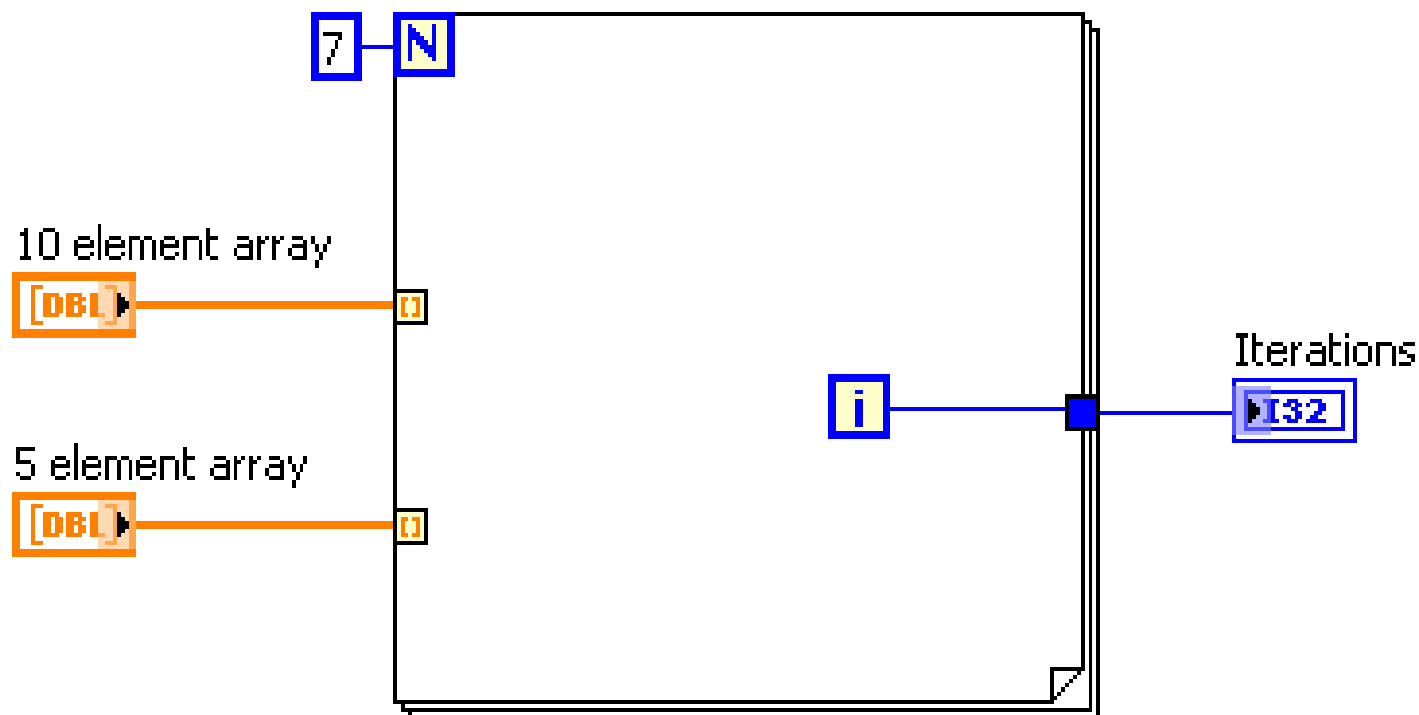
2. What is the value of the **Iterations** indicator after running this VI?



Summary—Quiz Answer

2. What is the value of the **Iterations** indicator after running this VI?

4



Managing File and Hardware Resources

TOPICS

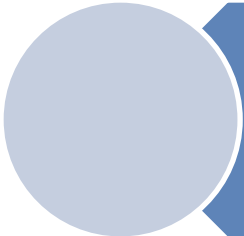
- A. Understanding File and Hardware Resources
- B. File I/O

A. Understanding File and Hardware Resources

Definition of Resources

Accessing Resources in LabVIEW

What are Resources?

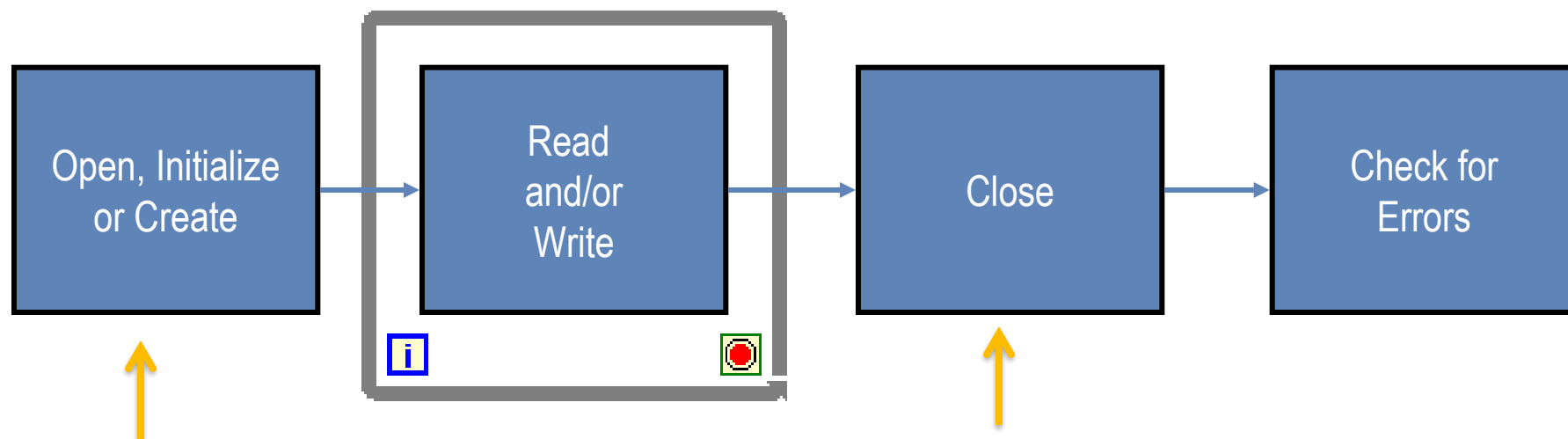


Resources – An addressable file, hardware device, object, or network connection available on your system

- LabVIEW includes VIs and functions to allow you to access your resources.
- Resources are known to the system by a path, name, port, or other identifier.

Accessing Resources in LabVIEW

A typical resource operation involves the following process:

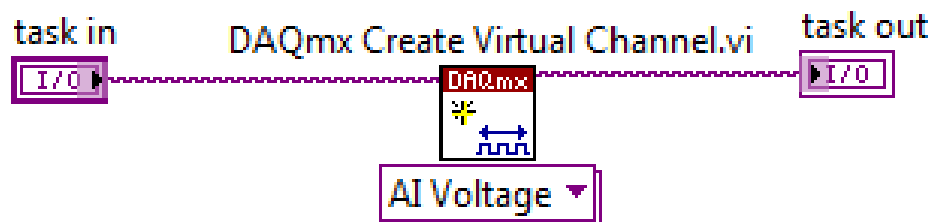


- Specify the path or device name.
- LabVIEW creates a refnum as a unique identifier to the resource.
- A refnum is a temporary pointer to the resource.

- The refnum becomes obsolete.

Accessing Resources in LabVIEW

Sample refnum wires:



B. File I/O

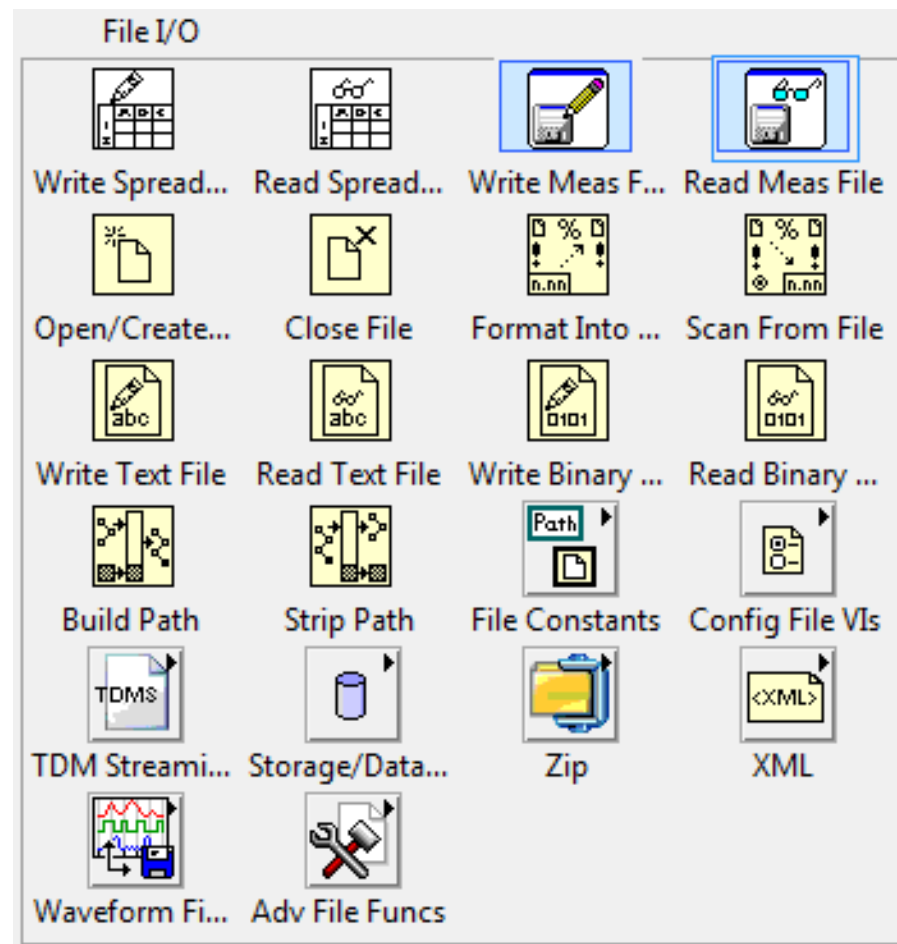
Understanding File I/O

File Formats

High-Level and Low-Level I/O

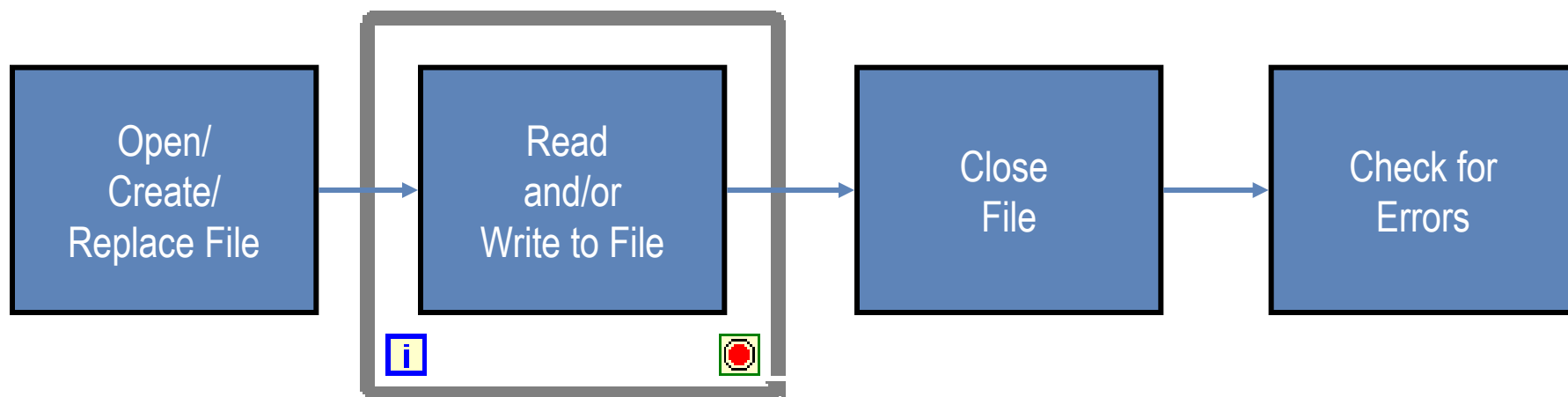
File I/O

- Understanding File I/O
- File Formats
- High-Level File I/O VIs
- Low-Level File I/O VIs



Understanding File I/O

- File I/O writes to or reads from a file.
- A typical file I/O operation involves the following process:



File Formats

LabVIEW can use or create the following file formats:

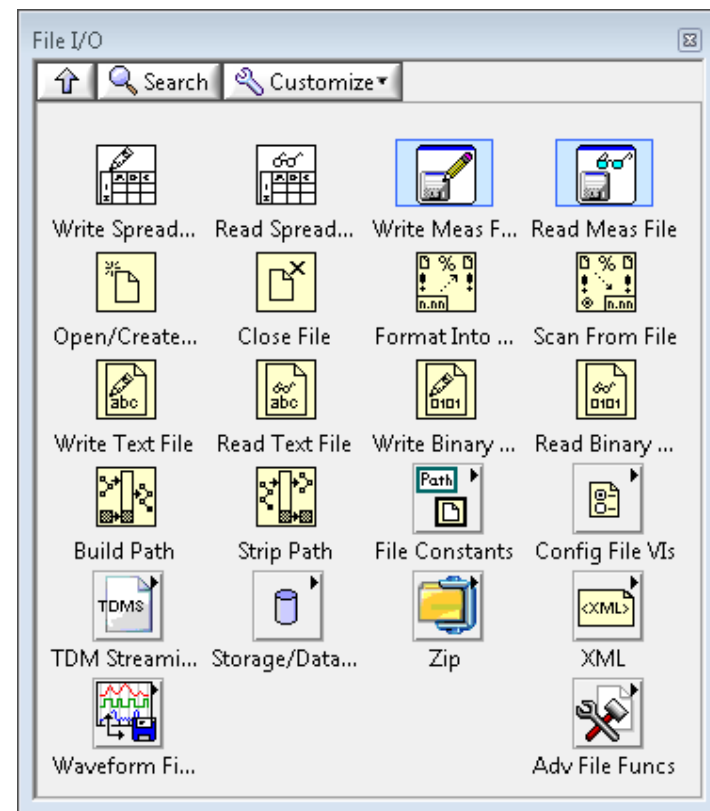
- **Binary**—Efficient, compact, and allows random access reading but not a human-readable file. Commonly used for high-speed and multi-channel DAQ.
- **ASCII**—Human-readable text file where data is represented as strings. Commonly used for low-speed DAQ.
- **LVM**—Format built on ASCII, the LabVIEW measurement data file (.lvm) is a tab-delimited text file you can open with a spreadsheet application or a text-editing application.
- **TDMS**—An NI-specific binary file format that contains data and stores properties about the data.

ASCII File Format

- In this course, you learn about creating text (ASCII) files.
- Use ASCII files in the following situations:
 - You want to access the file from another application.
 - Disk space and file I/O speed are not crucial.
 - You do not need to perform random access reads or writes.
 - Numeric precision is not important.

High-Level File I/O

- High-level File I/O functions:
 - Perform all three steps (open, read/write, close) for common file I/O operations.
 - Might not be as efficient as the functions configured or designed for individual operations.
- Low-level File I/O functions:
 - Provide individual functions for each step.
 - Are efficient for writing to a file in a loop.



High-Level File I/O

Write to Spreadsheet File

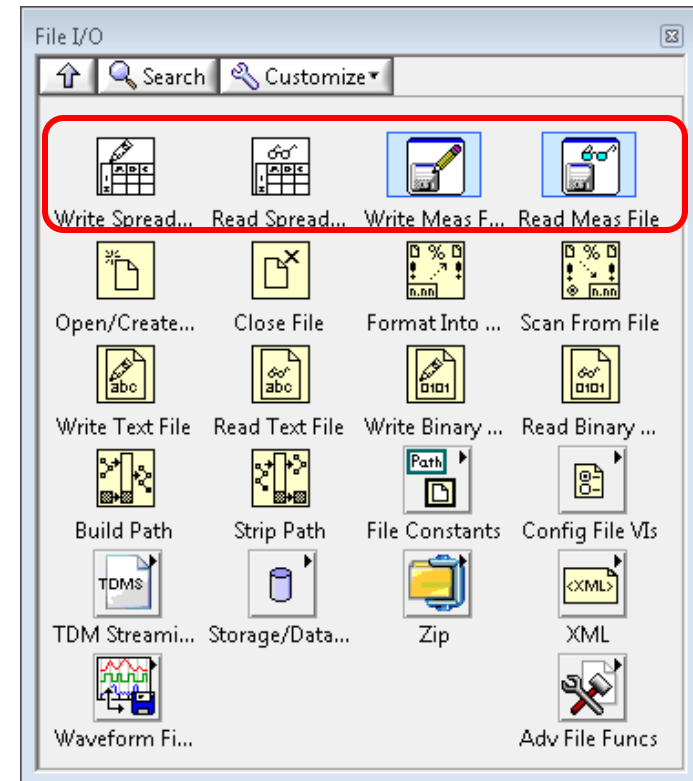
- Converts an array of double-precision numbers to a text string and writes the string to an ASCII file.

Read From Spreadsheet File

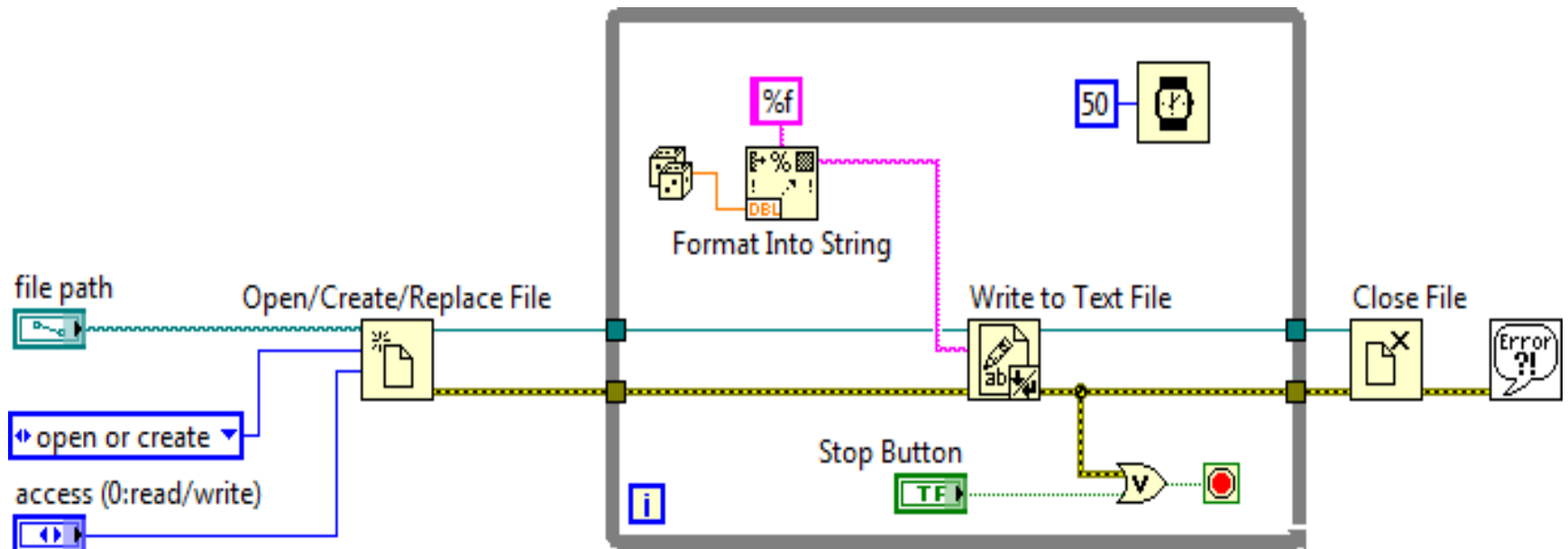
- Reads a specified number of lines or rows from a numeric text file and outputs a 2D array of double-precision numbers.

Write To/Read From Measurement File

- Express VIs that write data to or read data from an LVM or TDMS file format.



Understanding Low-Level File I/O VIs



Summary—Quiz

1. After opening a file, which output does the Open/Create/Replace File function return?
 - a) File path
 - b) File name
 - c) Refnum out
 - d) Task out

Summary—Quiz Answer

1. After opening a file, which output does the Open/Create/Replace File function return?
 - a) File path
 - b) File name
 - c) Refnum out**
 - d) Task out